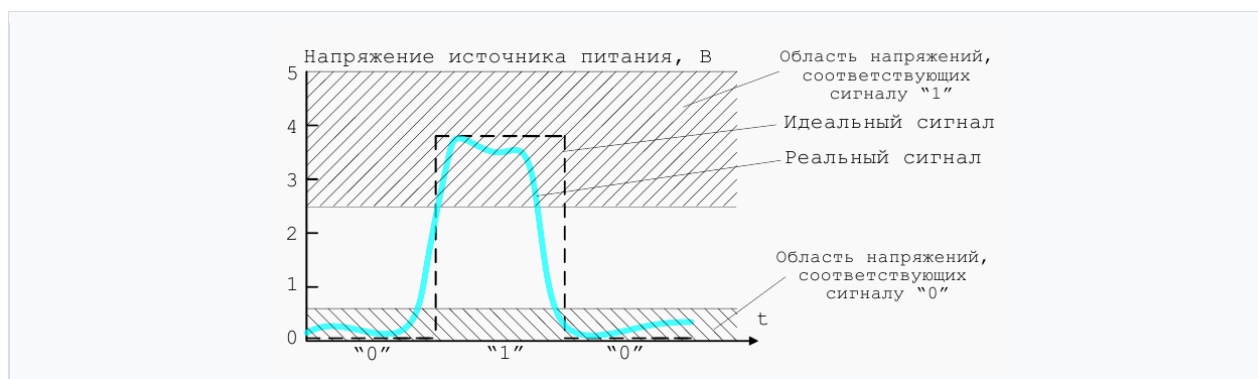


Содержание

1. Две формы представления информации. Способы представления дис...	3
2. Представление чисел с фиксированной точкой. Прямой, обратный...	5
3. Представление символьных и строковых данных. Принципы построе...	7
4. Базовые элементы вычислительной техники: ячейки, регистры, ши...	11
5. Структура и принцип функционирования ЭВМ. Порядок функциониро...	16
6. Операционная система Unix — ядро ОС и файловая система.	19
7. Операционная система Unix — интерпретаторы, стандартные поток...	21
8. Операционная система Unix — основные команды, права файлов и...	23
9. Состав и структура БЭВМ. Адресные пространства БЭВМ. Система...	27
10. Организация вычислений в БЭВМ. Сдвиги, арифметические и логич...	33
11. Организация массивов данных. Режимы адресации. Цикл выборки а...	34
12. Управление вычислительным процессом в БЭВМ. Команды ветвлений...	37
13. Подпрограммы в БЭВМ. Цикл исполнения команд перехода и возвра...	39
14. Организация ввода-вывода в вычислительных системах. Инициация...	45
15. Организация ввода-вывода в БЭВМ. Устройства ввода-вывода, ком...	48
16. Организация асинхронного обмена в БЭВМ. Пример программы. Вре...	57
17. Организация прерываний в БЭВМ. Вектора прерываний, контроллер...	59
18. Организация обмена по прерыванию программы в БЭВМ. Пример про...	68
19. Понятие многоуровневой ЭВМ. Понятие и пример программы на раз...	72
20. Микропрограммный уровень БЭВМ. Структура МПУ. Форматы микроко...	75
21. Структура и принципы работы арифметико-логического устройства...	78
22. Микропрограммное управление вентильными схемами. Схема управл...	84
23. Архитектура ЭВМ. Гарвардская и фон-Неймановская архитектура...	92
24. Архитектура многопроцессорных ЭВМ. Системный коммутатор. Архи...	98
25. Структура современных процессоров. Окружение процессора. CISC...	109
26. Адресуемая память, организация и временные диаграммы. Констру...	115
27. Память, ориентированная на записи (блочная память). Организац...	124
28. Характеристики запоминающих устройств. Пирамида памяти...	130
29. Ассоциативная память, Кэш-память. Влияние промахов кэш-памяти...	134
30. Предназначение и организация виртуальной памяти. Сегментно-ст...	141
31. Сетевые технологии, Понятие сети ЭВМ, классификация компьютер...	148
32. Модель TCP/IP: передающая среда, канальный и сетевой уровень...	154
33. Модель TCP/IP: выделение адресов (DHCP), сервисы имен, трансп...	164
34. Интерфейсы ввода-вывода. Контроллеры внешних устройств. Уровн...	168
35. Параллельная передача данных. Контроллеры параллельной переда...	172
36. Синхронные последовательные интерфейсы. Контроллеры последова...	176
37. Асинхронный обмен. Принципы деления частоты, формат кадра...	180
38. Контроллер передачи асинхронного последовательного интерфейса...	183
39. Контроллер приема асинхронного последовательного интерфейса...	185
40. Организация прямого доступа к памяти. Контроллер ПДП...	187

1. Две формы представления информации. Способы представления дискретной информации. Системы счисления, используемые в вычислительной технике: двоичная, 8-я, 10-я, 16-я, двоично-десятичная.

Первая форма представления информации называется аналоговой или непрерывной (с помощью сходной величины – аналога). Количество значений, которые может принимать величина, представленная в такой форме бесконечно велико, даже если величина изменяется в ограниченном диапазоне. Отсюда названия – непрерывная величина и непрерывная информация. Слово непрерывность выделяет основное свойство таких величин – отсутствие разрывов, промежутков между значениями, которые может принимать аналоговая величина. Величина представляется в виде одного сигнала, пропорционального этой величине. Эта форма представления используется в аналоговых вычислительных машинах



Вторая форма представления информации называется цифровой или дискретной (с помощью набора напряжений, каждое из которых соответствует одной из цифр представляемой величины). Такие величины, принимающие не все возможные, а лишь вполне определённые значения, называются дискретными (прерывистыми). В отличие от непрерывной величины количество значений дискретной величины всегда будет конечным. Величина представляется в виде нескольких сигналов, каждый из которых соответствует одной из цифр заданной величины. Эта форма представления используется в электронных вычислительных машинах (ЭВМ).

Каждое значение из набора исходных данных задачи, результатов её решения может быть представлено в ЭВМ в виде нескольких электрических сигналов, один из которых соответствует числу единиц в значении, другой – числу десятков, третий – числу сотен и т.д. Однако такое представление не является наилучшим с технических позиций. Устройство, предназначенное для обработки подобных сигналов, должно различать в каждом из них десять состояний. Значительно проще построить устройство, которое различало бы всего два состояния (его наличие или отсутствие). Это тем более целесообразно, т.к. существующие сейчас дешёвые устройства для ввода данных в ЭВМ также кодируют отдельные составляющие вводимой информации с помощью двух состояний.

Это толкнуло создателей первых ЭВМ к применению другой системы счисления при внутреннем представлении чисел в машинах: вместо привычной десятичной системы счисления была взята двоичная. 2СС также является позиционной СС, т.е. в ней значение каждой цифры зависит от позиции этой цифры в записи числа.

Существуют специальные термины, широко используемые в вычислительной технике: бит, байт и слово.

Двоичный разряд – бит

Восьмибитовая единица – байт

ЭВМ содержит большое количество ячеек памяти и регистров для хранения двоичной информации. Большинство этих ячеек имеет одинаковую длину n , т.е. они используются для хранения n бит двоичной информации. Информация, хранящаяся в такой ячейке, называется словом.

Удобная для использования в ЭВМ двоичная система счисления совсем неудобна для записи и чтения чисел человеком. Для сокращения трудоёмкости ручной обработки кодов чисел, команд широко применяют 8- и 16СС. В 8 СС используется 8 цифр (0-7), в 16СС – 10 цифр и 6 прописных букв (0-9, A-F). Т.к. основанием 8СС является $8=2^3$, то для перевода двоичных чисел в восьмеричные необходимо разделить двоичные числа на триады. Каждую из таких групп можно представить одной восьмеричной цифрой. Аналогичным образом осуществляется перевод двоичных чисел в шестнадцатеричные. Только в этом случае двоичное число разбивается на 4 тетрады, которые представляются одной шестнадцатеричной цифрой.

Наконец следует упомянуть о двоично-десятичной СС, которая используется в цифровых устройствах, где основная часть операций связана не с обработкой и хранением вводимой информации, а с самим её выводом на какие-либо индикаторы с десятичным представлением полученных результатов. В 2-10СС десятичные цифры от 0 до 9 представляют 4-разрядными двоичными комбинациями от 0000 до 1001. Две двоично-десятичные цифры составляют 1 байт (можно представлять значения от 0 до 99)

2. Представление чисел с фиксированной точкой. Прямой, обратный и дополнительный код. Формирование битовых признаков переноса, переполнения, отрицательного результата, нуля.

Целые двоичные числа без знака можно использовать для представления нуля и целых положительных чисел. При размещении таких чисел в одном 16-разрядном слове они могут изменяться от $(0000\ 0000\ 0000\ 0000)_2 = (000\ 0)_{16} = 0$ до $(1111\ 1111\ 1111\ 1111)_2 = (FFFF)_{16} = 2^{16} - 1 = 65535$. Такая запись называется прямым кодом числа. Подобные числа (так же как и рассмотренные ниже двоичные числа со знаком) относятся к числам с фиксированной запятой, разделяющей целую и дробную части числа. В числах, используемых в базовой ЭВМ, положение запятой строго фиксировано после младшего бита слова. Целые двоичные числа со знаком используются тогда, когда необходимо различать положительные и отрицательные числа. В современных ЭВМ для представления целых чисел со знаком используется дополнительный код, в котором старший бит формата определяет знак числа: 0 - для положительных чисел и 1 - для отрицательных чисел. При этом дополнительный код положительного числа совпадает с его прямым кодом.

А для представления отрицательного числа в дополнительном коде производится инвертирование прямого кода модуля числа (получение обратного кода числа) и добавление к результату единицы. Такая же операция используется при изменении знака числа, представленного в дополнительном коде. Использование дополнительного кода упрощает конструкцию ЭВМ, так как при сложении двух таких чисел, имеющих разные знаки, не требуется переходить к операциям вычитания меньшего (по модулю) числа из большего и присвоения результату знака большего числа.

Кроме того, одной и той же схемой сумматора можно воспользоваться для выполнения операций над знаковым и беззнаковым представлением числа. Признаком выхода за границы разрядной сетки для беззнакового представления числа является перенос в старший разряд (бит C - Carry). Например, при сложении:

$$\begin{array}{r} 1000\ 0000\ 0000\ 0000 \\ + 1000\ 0000\ 0000\ 0000 \\ \hline 1\ 0000\ 0000\ 0000\ 0000 \end{array}$$

В ответе должно получиться $32768+32768=65536$, но т.к. разрядность слова составляет лишь 16 бит, то в нем сохраняется только часть результата, т.е. 0. Единица, возникшая вследствие переноса оказалась в несуществующем 17 разряде. Признаком переполнения разрядной сетки для знакового представления является бит переполнения (OVerflow). Разные знаки слагаемых, или совпадение знаков слагаемых со знаком суммы свидетельствуют о том что результат корректен. в противном случае формируется сигнал – Переполнение. Признаком отрицательного результата N при знаковом представлении выставляется в случае когда в старшем разряде числа в доп. коде находится 1. Признаком нулевого результата Z выставляется в случае когда все разряды числа равны 0.

Целые двоичные числа без знака можно использовать для представления нуля и целых положительных чисел. При размещении таких чисел в одном 16-разрядном слове они могут изменяться от $(0000\ 0000\ 0000\ 0000)_2 = (0000)_{16} = 0$ до

$(1111\ 1111\ 1111\ 1111)_2 = (FFFF)_{16} = 2^{16} - 1 = 65535$. Такая запись называется прямым кодом числа.

совпадает с его прямым кодом. А для представления отрицательного числа в результате знака большего числа. Кроме того, одной и той же схемой сумматора представлением числа.

+

3. Представление символьных и строковых данных. Принципы построения кодовых таблиц ASCII, КОИ-8, ISO8859-5, Windows-1251, UTF-8, UTF-16.

Представление текстовой информации в вычислительных машинах основано на кодировании букв алфавитов для существующих на земле языков, которые традиционно используются человечеством. Символ - это графическое изображение, которое используется человеком для создания слов, текстов и другой значимой информации. Как известно, к символам относятся буквы, знаки препинания, символы валют, цифры и т.д.

В ЭВМ представление текстовой информации основывается на кодировании букв и символов при помощи кодовой таблицы. Кодовая таблица (или кодировка символов) является соглашением между разработчиками о соответствии каждому символу определенного порядкового номера или кода, чтобы его можно было сохранять в памяти ЭВМ или передавать по каналам связи.

Для хранения графических начертаний символов в ЭВМ существуют шрифты. Шрифты бывают векторные и растровые. В растровых шрифтах каждому коду символа соответствует изображение, определенного (в точках) размера. В векторных хранится принцип начертания символа в виде последовательности линий.

Кодировки стандарта ASCII

ASCII — таблицы кодировок, в которых содержатся основные символы (английский алфавит, цифры, знаки препинания, символы национальных алфавитов(свои для каждого региона), служебные символы) и длина кода каждого символа $n = 8$ бит.

7 бит:

ASCII — первая кодировка, пригодная для работы с текстом. Помимо маленьких букв английского алфавита и служебных символов, содержит большие буквы английского языка, цифры, знаки препинания и другие символы (при этом старший бит использовался для контроля четности битов передаваемого по каналу связи символа).

Кодировки стандарта ASCII (8 бит):

ISO 8859 — первая кодировка, в которой стало возможно использовать символы национальных алфавитов.

КОИ8-R — первая русская кодировка. Символы кириллицы расположены не в алфавитном порядке. Их разместили в верхнюю половину таблицы так, чтобы позиции кириллических символов соответствовали их фонетическим аналогам в английском алфавите. Это значит, что даже при потере старшего бита каждого символа, например, при проходе через устаревший семибитный модем, текст остается "читаемым" (отсюда и появилось понятие транслита).

Windows-1251 — русская кодировка, использовавшаяся в русскоязычных версиях операционной системы Windows в начале 90-х годов. Кириллические символы идут в алфавитном порядке. Содержит все символы, встречающиеся в типографике обычного текста (кроме знака ударения).

Кодировки стандарта UNICODE

Юникод — это промышленный стандарт, обеспечивающий цифровое представление символов всех письменностей мира и специальных символов.

Стандарт предложен в 1991 году некоммерческой организацией «Консорциум Юникода» (англ. Unicode Consortium). Применение этого стандарта позволяет закодировать очень большое число символов из разных письменностей. Стандарт состоит из двух основных разделов: универсальный набор символов (англ. UCS, universal character set) и семейство кодировок (англ. UTF, Unicode transformation format). Универсальный набор символов задаёт однозначное соответствие символов кодам — элементам кодового пространства, представляющим неотрицательные целые числа. Семейство кодировок определяет машинное представление последовательности кодов UCS.

Способы представления

Юникод имеет несколько форм представления (англ. Unicode Transformation Format): UTF-8, UTF-16 (UTF-16BE, UTF-16LE) и UTF-32 (UTF-32BE, UTF-32LE).

UTF-8 — представление Юникода, обеспечивающее наилучшую совместимость со старыми системами, использовавшими 8-битные символы. Текст, состоящий только из символов с номером меньше 128, при записи в UTF-8 превращается в обычный текст ASCII. И наоборот, в тексте UTF-8 любой байт со значением меньше 128 изображает символ ASCII с тем же кодом. Остальные символы Юникода изображаются последовательностями длиной от двух до шести байт.

UTF-16

Первая версия Юникода (1991 г.) представляла собой 16-битную кодировку с фиксированной шириной символа; общее число разных символов было 65 536. Во второй версии Юникода (1996 г.) было решено значительно расширить кодовую область; для сохранения совместимости с теми системами, где уже был реализован 16-битный Юникод, и была создана UTF-16. Область 0xD800—0xDFFF, отведённая для суррогатных пар, ранее принадлежала к области «символов для частного использования».

Поскольку в UTF-16 можно отобразить 1 112 064 символов, то это число и было выбрано в качестве новой величины кодового пространства Юникода.

ASCII		
Порядковые номера	Коды символов	Что за...
0 — 31	00000000 — 00011111	Управляющие символы <i>ДЛЯ</i> управления процессом вывода текста на экран или печать, подача звукового сигнала, разметка текста и т.п.
32 — 127	00100000 — 01111111	Сюда входят строчные и прописные буквы латинского алфавита, десятичные цифры, знаки препинания, всевозможные скобки, коммерческие и другие символы. (Символ 32 — пробел)
128 — 255	10000000 — 11111111	Кодовая страница <i>ДЛЯ</i> нелатинского алфавита

Кодирование UTF-8 1.

Если размер символа в кодировке UTF-8 = байт Код имеет вид (0aaa aaaa), где «0» — просто ноль, остальные биты «а» — это код символа в кодировке ASCII; 2. Если размер символа в кодировке в UTF-8 байт (то есть от до):

2.1 Первый байт содержит количество байт символа, закодированное в единичной системе счисления;

2 – 11

3 – 111

4 – 1111

5 – 1111 1

6 – 1111 11

2.2 «0» – бит терминатор, означающий завершение кода размера

2.3 далее идут значащие байты кода, которые имеют вид (10xx xxxx), где «10» – биты признака продолжения, а «х» – значащие биты.

В общем случае варианты представления одного символа в кодировке UTF-8 выглядят так:

(1 байт) 0aaa aaaa

(2 байта) 110x xxxx 10xx xxxx

(4 байта) 1111 0xxx 10xx xxxx 10xx xxxx 10xx xxxx

UTF-16

— символы кодируются двухбайтовыми словами с использованием всех возможных диапазонов значений (от 0000_{16} до $FFFF_{16}$). При этом можно кодировать символы Unicode Сдвигаем символ в 8в диапазонах $0000_{16}..D7FF_{16}$ и $E000_{16}..10FFFF_{16}$. Исключенный отсюда диапазон

6

$D800_{16}..DFFF_{16}$ используется как раз для кодирования так называемых суррогатных пар — символов, которые кодируются двумя 16-битными словами. Символы Unicode до $FFFF_{16}$ включительно (исключая диапазон для суррогатов) записываются как есть 16-битным словом. Символы же в диапазоне $10000_{16}..10FFFF_{16}$ (больше 16 бит) уже кодируются парой 16-битных слов. Для этого их код арифметически сдвигается до нуля (из него вычитается минимальное число 10000_{16}). В результате получится значение от нуля до $FFFF_{16}$, которое занимает до 20 бит. Старшие 10 бит этого значения идут в лидирующее (первое) слово, а младшие 10 бит — в последующее (второе). При этом в обоих словах старшие 6 бит используются для обозначения суррогата. Биты с 11 по 15 имеют значения 11011_2 , а 10-й бит содержит 0 у лидирующего слова и 1 — у последующего.

4. Базовые элементы вычислительной техники: ячейки, регистры, шины, вентили, тактовые генераторы, логические схемы, триггеры, регистры, счетчики, сумматоры.

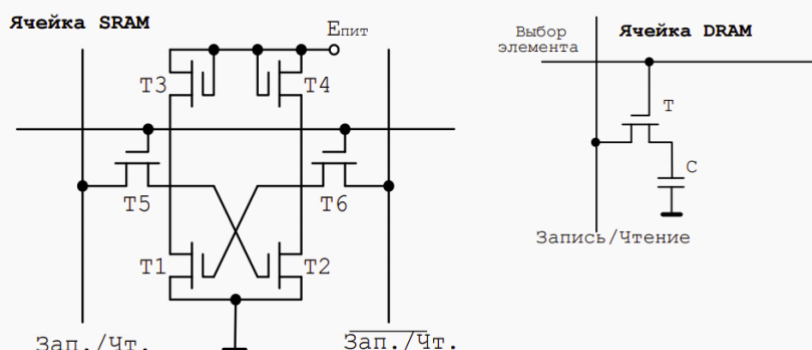
Ячейка памяти – минимальный адресуемый элемент запоминающего устройства ЭВМ. Ячейки имеют адрес (порядковый номер, число), по которому к ним могут обращаться команды процессора. Ячейки памяти состоят из элементов, которые могут

находиться в одном из двух устойчивых состояний: конденсатор заряжен или разряжен, транзистор находится в проводящем или непроводящем состоянии. Одно из таких физических состояний создает высокий уровень выходного напряжения элемента памяти, а другое – низкий. Первое обычно принимается за двоичную 1, а второе – за двоичный 0. Возможно и обратное кодирование. Память бывает статическая (SRAM - static random access memory) и динамическая (DRAM – dynamic ...)

Статическая память с произвольным доступом (SRAM, static random access memory) – полупроводниковая оперативная память, в которой каждый двоичный разряд хранится в схеме с положительной обратной связью, позволяющей поддерживать состояние без регенерации. Тем не менее, сохранять данные без перезаписи SRAM может, только пока есть питание. Используется в кэшах

Динамическая память с произвольным доступом – тип компьютерной памяти, отличающийся использованием полупроводниковых материалов, энергозависимостью и возможностью доступа к данным, хранящимся в произвольных ячейках памяти. Физически DRAM состоит из ячеек, созданных в полупроводниковом материале в виде емкости. Заряженная или разряженная емкость хранит бит данных. Каждая ячейка такой памяти имеет свойство разряжаться, поэтому их постоянно надо подзаряжать. Совокупность ячеек образует условный «прямоугольник», состоящий из определенного количества строк и столбцов. Один такой «прямоугольник» называется страницей, а совокупность страниц называется банком.

• Элементы хранения (DRAM/SRAM)



Регистр процессора – память внутри процессора, предназначенная для хранения адресов и промежуточных результатов вычислений или данных, необходимых для работы самого

процессора. Регистр характеризуется единственным числом: количеством битов, которые могут в нем храниться. Операция чтения информации, хранимой в регистре, сводится к созданию копии его содержимого, оригинал же сохраняется в регистре без изменений.

Шина - электрическая цепь, соединяющая регистр с другим регистром или иным устройством ЭВМ. Шина состоит из параллельных проводов, каждый из которых предназначен для передачи соответствующего регистра. Также шина содержит несколько дополнительных проводов, используемых для передачи сигналов синхронизации и управления. Шины служат для передачи информации лишь в направлении, обозначенном стрелкой на шине. Специальные схемы позволяют в одни моменты времени передавать информацию по шине в одну сторону, а в другие – в обратном направлении, т.е. организовать двунаправленную шину.

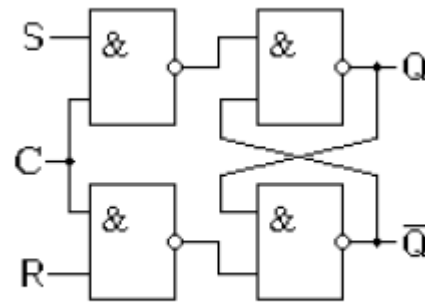
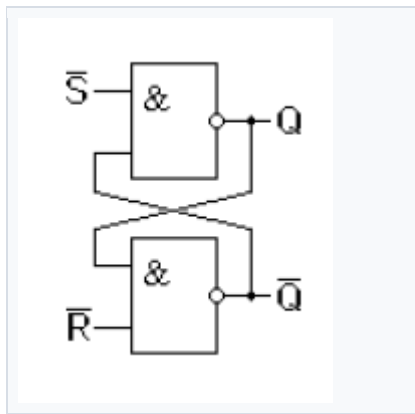
Вентильные схемы – это электронные ключевые схемы, предназначенные для управления потоком информации из регистров в шины и обратно. Такая схема содержит два входа и один выход. На один вход подается информационный сигнал (данные с регистра), а на другой (являющийся вентилем) – управляющий. Если управляющий сигнал равен 1, то данные проходят схему без препятствий, если 0 – никакая информация не пройдет через схему. Для подачи информационного сигнала на вход вентильной схемы обычно используется многопроводная шина. Для передачи выходного сигнала требуется шина с таким же количеством проводов. Если управляющий сигнал равен 1, то информационные сигналы на входной и выходной шинах совпадают.

Тактовый генератор – устройство, генерирующее электрические импульсы заданной частоты (обычно прямоугольной формы). Используется для синхронизации процессов передачи информации между устройствами ЭВМ.

Функциональная логическая схема - совокупность логических элементов (простейшее устройство ЭВМ, выполняющее одну определённую логическую операцию над входными сигналами согласно правилам алгебры логики) и связей между ними.

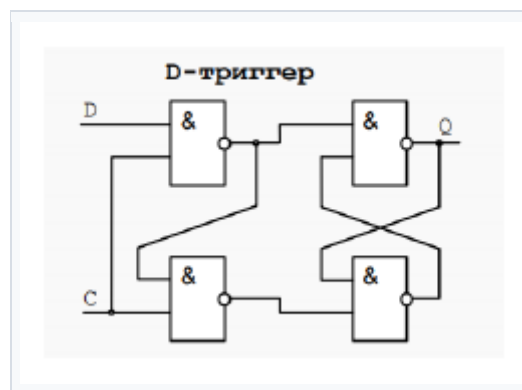
Триггер — это элемент, который имеет **два устойчивых состояния** (0 и 1) и **сохраняет** своё состояние, пока его специально не переключат. В отличие от логического вентиля (который просто вычисляет функцию от входов «здесь и сейчас»), у него есть память - способность оставаться в одном из двух состояний и после прекращения действия переключающего сигнала. У триггера обычно два выхода: прямой Q (само хранимое значение) и инверсный (отрицание Q) — они всегда противоположны.

RS-триггер получил название по названию своих входов. Вход S (Set — установить англ.) позволяет устанавливать выход Q в единичное состояние. (Устанавливать означает записывать логическую единицу). Вход R (Reset — сбросить англ.) позволяет сбрасывать выход Q (Quit — выход англ.) в нулевое. **Главный недостаток** — наличие **запрещённой комбинации** ($R = S = 1$) и то, что триггер реагирует на входы **в любой момент** (асинхронно), как только они меняются. В сложных схемах нужна согласованность по времени, а такой триггер срабатывает хаотично. Собирается из двух логических элементов (И-НЕ или ИЛИ-НЕ), у которых выход каждого заведён на вход другого — **перекрёстная обратная связь**. Именно эта петля обратной связи и создаёт «запоминание»: выходы поддерживают друг друга.



Синхронный RS-триггер. Схема, пропускающая входные сигналы только при наличии синхронизирующего сигнала. К обычному RS добавляется третий вход — **C (синхросигнал, такт)**: Пока **C = 0** — входы R и S **игнорируются**, триггер хранит состояние, что бы ни происходило на R и S. Триггер переключается (воспринимает R и S) **только когда C = 1** (или по фронту такта). Это убирает хаотичность и делает схему предсказуемой. Но **запрещённая комбинация** $R = S = 1$ у синхронного RS всё равно остаётся

D-триггер. При записи и хранении данных один бит может принимать значение, как нуля, так и единицы. Для его передачи достаточно одного провода. сигналы установки и сброса триггера не могут появляться одновременно, поэтому можно объединить эти входы при помощи инвертора. Нет запрещённой комбинации; ячейка хранения бита → регистры. D-триггер получают из синхронного RS **одним конструктивным приёмом**: вместо двух независимых входов оставляют **один вход D**, а внутренние сигналы S и R формируют из него так, что **R всегда равен инверсии S**.



T-триггер — это счетный триггер. У данного триггера имеется только один вход. Принцип работы T-триггера заключается в следующем. каждый входной импульс переключает триггер в противоположное состояние. поскольку триггер меняет состояние каждым импульсом, на его выходе сигнал меняется вдвое реже, чем на входе. Поэтому T-триггер делит частоту пополам, а каскад T-триггеров — на степень двойки

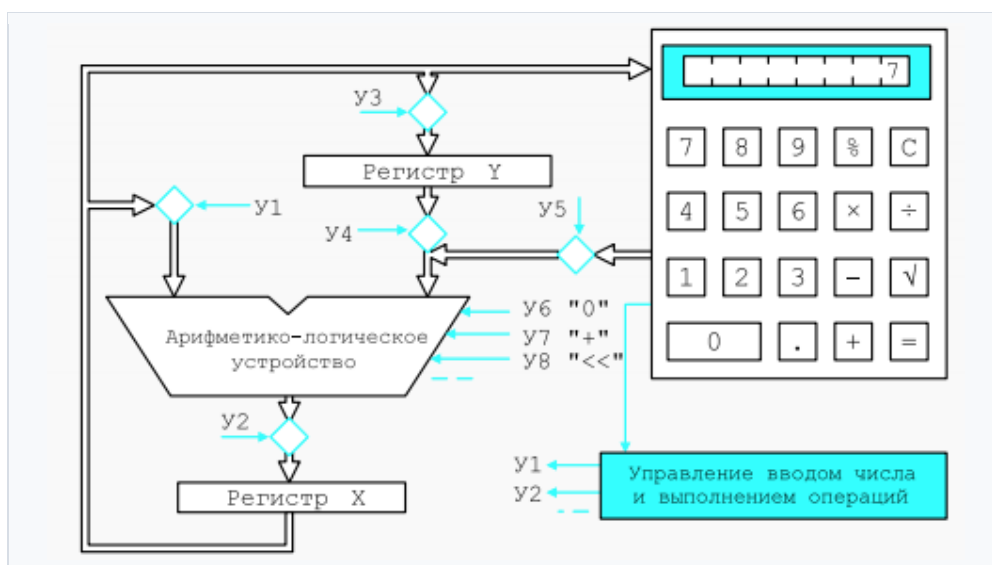
Счётчик числа импульсов — устройство, на выходах которого получается двоичный (двоично-десятичный) код, определяемый числом поступивших импульсов. Счетчики импульсов являются разновидностью регистров (счетные регистры) и строятся соответственно на триггерах и логических элементах. Основным параметр счётчика — модуль счёта — максимальное число

единичных сигналов, которое может быть сосчитано счётчиком. На рисунке представлена схема четырехразрядного счетчика на Т-триггерах, соединенных последовательно. Счетные импульсы подаются на счетный вход первого триггера. Счетные входы последующих триггеров связаны с выходами предыдущих триггеров.

Сумматор—устройство, преобразующее информационные сигналы (аналоговые или цифровые) в сигнал, эквивалентный сумме этих сигналов.

5. Структура и принцип функционирования ЭВМ. Порядок функционирования простого процессора на примере калькулятора.

Типичная ЭВМ состоит из процессора, памяти и устройств ввода-вывода. «Сердцем» ЭВМ является процессор, в состав которого входят устройство управления выборкой команд из памяти и их выполнением, арифметико-логическое устройство, производящее операции над данными, регистры, осуществляющие временное хранение данных и состояний процессора, схемы для управления и связи с подсистемами памяти и ввода-вывода. Устройство ввода обеспечивает считывание информации с определенных носителей информации и ее представление в форме электрических сигналов, воспринимаемых другими устройствами ЭВМ. Устройства вывода представляют результаты обработки информации в форме, удобной для визуального восприятия. Память ЭВМ включает устройство, обеспечивающее хранение команд и данных. Это устройство состоит из блоков одинакового размера – ячеек памяти, предназначенных для хранения одного слова информации. Ячейка памяти состоит из элементов памяти, состояние каждого из которых соответствует одной двоичной цифре. Совокупность нулей и единиц, хранящихся в элементах одной ячейки, представляет собой содержимое этой ячейки памяти. В микро ЭВМ используются безадресные, одноадресные и реже двухадресные команды. В одноадресных командах один из операндов выбирается из специального регистра – аккумулятора. В него же заносится и результат операции. Безадресные команды или задают какое-либо действие с устройствами ЭВМ, или используются для работы с операндами, имеющими фиксированное расположение (чаще всего с аккумулятором). В процессе работы ЭВМ последовательно выполняет набор достаточно простых операций: выборку команды, определение ее типа, исполнение команды и определение адреса следующей команды.



Рассмотрим принципы функционирования простейшей ЭВМ — калькулятора. Он состоит из двух регистров — X и Y, хранящих результаты ввода пользователя и промежуточных вычислений, АЛУ, которая может выполнять простейшие арифметические и логические операции, шин и управляющих вентилях, осуществляющих передачу данных между функциональными блоками калькулятора, устройства управления (УУ), клавиатуры и дисплея.

Дисплей постоянно отображает содержимое регистра X. Клавиатура передает значение нажатой клавиши на вентиль У5, каждое нажатие на клавишу запускает УУ, которое в зависимости от текущего состояния ЭВМ формирует последовательность импульсов для выполнения требуемой операции, которая называется циклом импульсов. Каждая группа импульсов выдается последовательно, в моменты, совпадающие с импульсами тактового генератора.

Предположим пользователь вводит первую цифру необходимого ему числа (7). Так как это новая операция, УУ, после своей активации нажатием кнопки 7, выдаст последовательность управляющих импульсов для первой цифры числа.

В первую очередь необходимо сохранить предыдущее значение регистра X в регистре У. Для этого должен быть открыт вентиль, управляющий записью в регистр У. Он открывается управляющим сигналом У3.

После этого необходимо обнулить регистр X, подготовив его для новой цифры числа, которая была введена с клавиатуры. Для этого должны быть закрыты все вентили, кроме У2 — который осуществляет передачу данных из АЛУ в регистр У и У6 — который сформирует в АЛУ значение 0.

Далее необходимо сложить значение 0 с цифрой с клавиатуры. Для этого содержимое регистра X поступает на правый вход АЛУ (У1), цифра с клавиатуры на правый вход АЛУ (У5), и выбирается операция сложения (У7).

В конце цикла необходимо передать результат сложения в регистр X (У2), отобразив его, при этом, на дисплее.

Для ввода второй и последующих цифр необходимо осуществлять поразрядный сдвиг регистра X после каждой введенной цифры и складывать введенную цифру со сдвинутым содержимым регистра X. Разберем этот цикл по тактам:

Содержимое регистра X через вентиль, управляемый У1 подается на левый вход АЛУ, при этом вентили У4, который управляет передачей из регистра У и У5 (ввод с клавиатуры), должны быть закрыты, при этом на правый вход АЛУ подается 0. Управляющий сигнал У8 вызовет сдвиг информации, которая поступает на левый вход АЛУ. В двоично-десятичной системе счисления (в которой обычно выполняют вычисления калькуляторы) сдвигу на один десятичный разряд соответствует умножение на 10.

Результат сдвига записывается (управляющий сигнал У2) в регистр X.

Затем необходимо сложить результат сдвига с новой цифрой с клавиатуры. Для этого содержимое регистра X поступает на правый вход АЛУ (У1), цифра с клавиатуры на правый вход АЛУ (У5), и выбирается операция сложения (У7).

Результат сдвига и сложения записывается (управляющий сигнал У2) в регистр X.

Когда пользователь нажимает кнопку «+» или «=», в зависимости является ли это сложение промежуточной или конечной операцией, необходимо сложить содержимое регистра X и регистра У. Для этого:

Содержимое регистра X подается на левый вход АЛУ (У1), содержимое регистра У подается на правый вход АЛУ (У4) и выполняется операция сложения (У7). Все остальные вентили при этом закрыты (в первую очередь должен быть закрыт У5). Результат записывается в регистр X и показывается на экране калькулятора.

6. Операционная система Unix – ядро ОС и файловая система.

UNIX – семейство переносимых, многозадачных и многопользовательских операционных систем. Идеи, заложенные в основу UNIX, оказали огромное влияние на развитие компьютерных операционных систем. В настоящее время UNIX-системы признаны одними из самых исторически важных ОС.

Основное отличие UNIX-подобных систем от других операционных систем заключается в том, что это изначально многопользовательские многозадачные системы. То есть в один и тот же момент времени сразу множество людей может выполнять множество вычислительных задач (процессов). Даже популярную во всём мире систему Microsoft Windows нельзя назвать полноценной многопользовательской системой, так как кроме как на некоторых серверных версиях, в один и тот же момент за одним компьютером с Windows может работать только один человек. В Unix может работать сразу много людей, при этом каждый из них может выполнять множество различных вычислительных процессов, которые будут использовать ресурсы именно этого компьютера.

Вторая колоссальная заслуга Unix в её мультиплатформенности. Ядро системы написано таким образом, что его легко можно приспособить практически под любой микропроцессор.

UNIX имеет и другие характерные особенности:

- использование простых текстовых файлов для настройки и управления системой;
- широкое применение утилит, запускаемых из командной строки;
- взаимодействие с пользователем посредством виртуального устройства – терминала;
- представление физических и виртуальных устройств и некоторых средств межпроцессового взаимодействия в виде файлов;
- использование конвейеров из нескольких программ, каждая из которых выполняет одну задачу.

Файловая система UNIX

В операционной системе UNIX файл является хранилищем двоичных и символьных данных, хранимых как поток байтов. Понятие файла является одним из наиболее важных для ОС UNIX. Все файлы, с которыми могут манипулировать пользователи, располагаются в файловой системе, представляющей собой дерево, промежуточные вершины которого соответствуют каталогам, и листья – файлам и пустым каталогам. Каждый каталог и файл файловой системы имеет уникальное полное имя. Каталог, являющийся корнем файловой системы (корневой каталог) имеет имя / . Коротким или относительным путем называется путь к файлу от текущего рабочего каталога. В каждом каталоге содержатся два специальных файла-ссылки, файл "." – ссылка на текущий каталог, и ссылка ".." на родительский каталог.

inode – Index-node – описатель файла, его уникальный номер. Он содержит всю информацию о файле, за исключением имени файла, и собственно данных файла. В inode хранятся тип файла, права, время модификации/создания файла и другая служебная информация под общим названием «метаданные».

7. Операционная система Unix – интерпретаторы, стандартные потоки ввода вывода, фильтры.

Командный интерпретатор – программа, предоставляющая пользователю интерфейс для общения с командной строкой; эта программа «переводит» введенные пользователем команды на понятный операционной системе язык. Интерпретатор более известен как оболочка (англ. shell). Наиболее распространенными оболочками являются sh, bash (стандарт в Unix), c shell. Пользователь может вводить команды как по отдельности, так и с помощью набора команд (скриптов). Команды могут задаваться как напрямую в командной строке, так и поступать из стандартного ввода или указанного файла. В качестве команд могут приниматься вызовы системных или прикладных утилит или управляющие конструкции. Кроме того, оболочка отвечает за перенаправление потоков ввода-вывода. В совокупности с набором утилит, она представляет собой операционную среду, язык программирования и средства решения как системных, так и прикладных задач, особенно по части автоматизации выполняемых последовательностей команд. Для взаимодействия и обмена информацией с пользователем используются файлы, именуемые стандартными потоками ввода (для чтения из него) и вывода (для записи в него). Вывод на экран представляется тоже как запись в файл, а ввод – как чтение из файла. Кроме потоков ввода и вывода существует так же стандартный поток ошибок, на который выводится вся служебная информация, которая не должна попадать в поток вывода (сообщения об ошибке или ходе работы программы). Стандартные потоки привязаны к файловым дескрипторам с номерами: 0 для ввода (stdin) 1 для вывода (stdout) 2 для ошибок (stderr).

Потоки по умолчанию связаны с терминалом (командной строкой), но их можно подключить к чему угодно – к файлам, программам или устройствам. В интерпретаторе такая операция называется перенаправлением. Таким образом, стандартные потоки можно перенаправлять не только в файлы, но и на вход других программ. Для осуществления перенаправления используются следующие операции:

- **> file** — перенаправить stdout в **file**
- **>> file** — добавить stdout к **file**
- **2> file** — перенаправить stderr в **file**
- **2>> file** — добавить stderr к **file**
- **< file** — взять stdin из **file**
- **<< EOF** — записать в stdin из терминала до символов «**EOF**»
- **ls | sort** — перенаправить stdout команды **ls** на stdin команды **sort**

Файл т.н. «пустое устройство» - /dev/null – перенаправление в него позволяет избавиться от ненужных сообщений об ошибке или игнорирования вывода. С помощью него также можно создавать пустые файлы, используя в качестве источника ввода. При записи в него может вместить любое количество информации, он работает в качестве «черной дыры».

8. Операционная система Unix – основные команды, права файлов и способы их задания.

touch файл Создает пустой файл, а если он уже есть – обновляет время последней модификации.

mkdir каталог Создает пустой каталог.

rm файл Удаляет файл.

-r Рекурсивно стирает каталоги. Если этого флага нет, файл не может быть каталогом.

rmdir каталог Стирает только пустые каталоги.

Echo Выводит строку текста.

cat файл Выводит содержимое файла.

Pwd Выводит имя текущего каталога.

ls файл Выводит список файлов в каталоге или информацию о файле, если это не каталог.

-l Длинный формат. Выводится с подробной информацией о каждом файле.

-a Вывод вместе со скрытыми файлами.

-FK имени файла добавляется его тип.

-R Рекурсивно выводит подкаталоги.

cd каталог Переходит в каталог.

cp файл1 файл2 Копирует файл в другой файл.

mv файл каталог Перемещает файл в каталог.

ln файл1 файл2 Создает новую жесткую ссылку на файл. Жесткая ссылка может ссылаться только в пределах одного диска. Файл не будет удален, пока на него есть хоть одна жесткая ссылка.

-s Создает символическую ссылку. Может ссылаться куда угодно. Если переместить/удалить файл, симв. ссылка будет недействительна.

head/tail файл Выводит первые/последние 4 строки файла

-n Первые/последние n строк.

-с Первые/последние с байт.

wc файл Выводит количество строк, слов и байт в файле.

-l Только кол-во строк.

-w Только кол-во слов.

-с Только кол-во байт.

-m Кол-во символов.

find выражение Ищет файлы в иерархии каталогов по заданным параметрам.

man команда Выводит справку по команде.

Для каждого файла существуют следующие категории пользователей:

u (user) Владелец файла. g (group) Члены группы, владеющей файлом. o (others) Все остальные. a (all) Все категории. Не рассматривается как отдельная категория.

Каждая из этих категорий может иметь любую комбинацию из следующих прав:

r (read) Право на чтение файла/просмотр каталога. w (write) Право на запись в файл/добавление или удаление каталога. x (execute) Право на исполнение файла/поиск и переход в каталог.

Права представляют собой последовательность из 9 бит – по 3 бита на категорию: владелец, группа, прочие; в следующем порядке – чтение, запись, исполнение. В случае отсутствия какого-либо из прав у категории, ставится символ «-». Вторым способом записи прав является запись этой последовательности в 8-ричной системе счисления, где праву на чтение (r) соответствует цифра 4, праву на запись (w) – цифра 2, а праву на исполнение (x) – цифра 1. Цифра 0 означает отсутствие прав. Для получения конечной цифры, нужные права суммируются. Таким образом, запись занимает всего 3 бита: по 1 биту на категорию.

Для выставления прав файлу (каталогу) используется команда `chmod`. Существует 3 способа задания прав доступа:

`chmod [ugoa][+|=][rwx] файл` Добавляет, удаляет или устанавливает выбранную комбинацию прав для выбранной комбинации категорий. `chmod число файл` Устанавливает права на основе восьмиричной записи. `chmod категория1=категория2 файл` Копирует права одной категории и присваивает их другой.

Сначала рассмотрим какими бывают права доступа linux и как они устанавливаются. Пред этим рекомендую прочитать статью про права, ссылка на которую есть выше. Есть три основных вида прав:

r - чтение;

w - запись;

x - выполнение;

s - выполнение от имени суперпользователя (дополнительный);

Также есть три категории пользователей, для которых вы можете установить эти права на файл linux:

u - владелец файла;

g - группа файла;

o - все остальные пользователи;

Синтаксис настройки прав такой:

группа_пользователей действие вид_прав

В качестве действий могут использоваться знаки "+" - включить или "-" - отключить. Рассмотрим несколько примеров:

u+x - разрешить выполнение для владельца;

ugo+x - разрешить выполнение для всех;

ug+w - разрешить запись для владельца и группы;

o-x - запретить выполнение для остальных пользователей;

ugo+gwx - разрешить все для всех;

Но права можно записывать не только таким способом. Есть еще восьмеричный формат записи, он более сложен для понимания, но пишется короче и проще. Я не буду рассказывать, как считать эти цифры, просто запомните какая цифра за что отвечает, так проще:

0 - никаких прав;

1 - только выполнение;

2 - только запись;

3 - выполнение и запись;

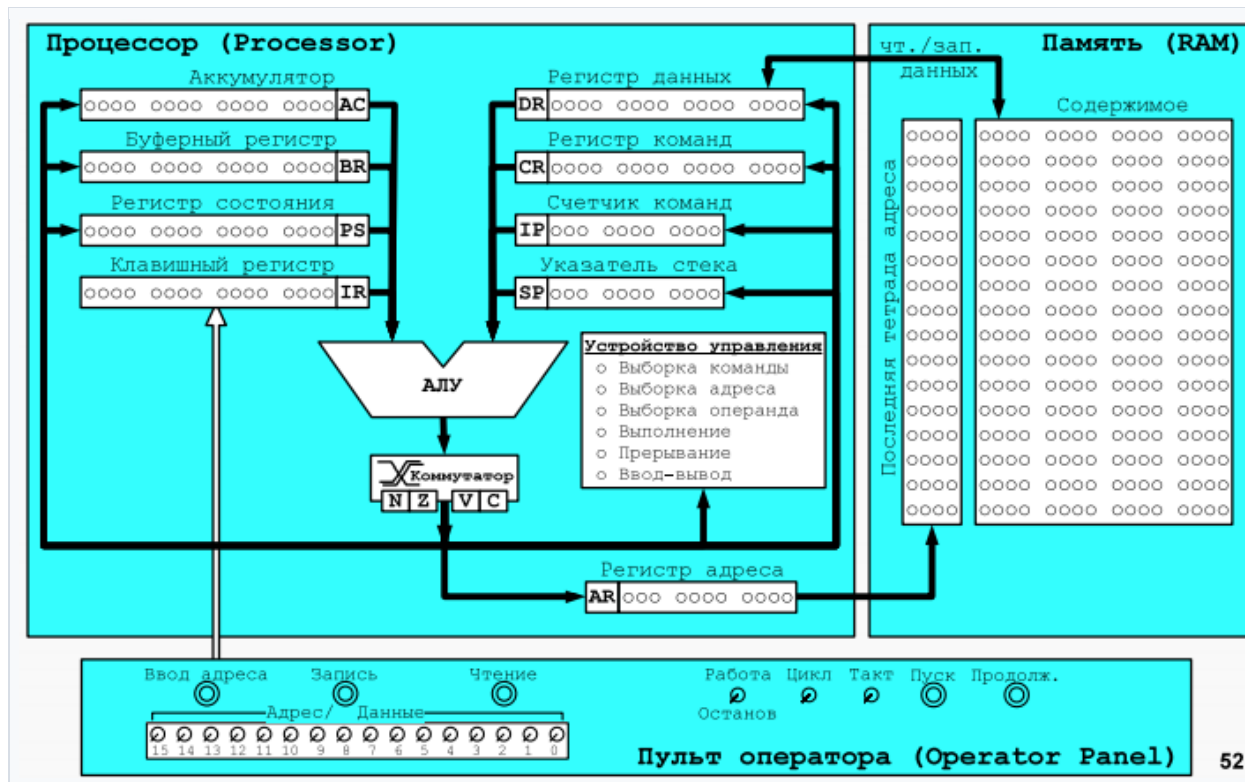
4 - только чтение;

5 - чтение и выполнение;

6 - чтение и запись;

7 - чтение запись и выполнение.

9. Состав и структура БЭВМ. Адресные пространства БЭВМ. Система команд БЭВМ, форматы команд. Машинные циклы.



52

БЭВМ включает в себя нескольких функциональных блоков и регистров:

Память - состоит из 2048 ячеек. Каждая ячейка занимает 16 разрядов. Для обращения к памяти существует два регистра: 11-разрядный регистр адреса (AR – Address Register), в который нужно поместить адрес прежде чем обратиться к памяти; 16-разрядный регистр данных (DR – Data Register), который предназначен для чтения или записи данных в/из ячеек памяти. Чтение данных и запись данных реализуется по шинам, которые подключаются к ячейке памяти.

11-разрядный счетчик команд (IP – Instruction Pointer). Хранит в себе адрес следующей исполняемой команды.

Арифметико-логическое устройство или АЛУ (ALU – Arithmetic-n-Logic Unit) может выполнять несколько операций: сложение, логическое умножение, инверсия и прибавление единицы. При операций «сложение» возможен выход за пределы разрядной сетки и формирование битов переполнения и переноса. Выход из АЛУ через коммутатор подключается к шине, по которой информация может быть передана в любой другой регистр БЭВМ.

Буферный регистр (BR – Buffer Register) это 16-разрядный регистр, который используется для организации промежуточного хранения данных во время работы.

Регистр команд (CR – Command Register) - используется для хранения кода команды и декодирования операций, происходящих во время работы.

Акумулятор (AC - ACcumulator). БЭВМ относится к ЭВМ, которые называются ЭВМ аккумуляторного типа, где все вычисления с данными производятся через этот регистр.

Указатель стека (SP – Stack Pointer), как и IP и АЯ 11-ти разрядный, и всегда указывает на вершину стека - особого участка памяти, который предназначен для хранения адресов возвратов и параметров подпрограмм и прерываний.

16-разрядный клавишный регистр (IR - Input Register) - находится в составе пульта оператора ЭВМ и предназначен для ввода адреса программы, кодов программы и данных, запуска программы на выполнение и управления режимами работы БЭВМ.

16-ти разрядный регистр состояния (PS – Program State) хранит биты управляющие работой БЭВМ (работа, прерывание и пр.) и признаки результата.

Устройство управления разработано в виде микропрограммного устройства управления (МПУ, MCU – Microprogram Control Unit) – простейшего компьютера, программа которого непосредственно состоит из микроопераций - т. е. по-тактного изменения значений вентилях БЭВМ, которые задают атомарные операции: вычисления в АЛУ, пересылки данных между регистрами и простейшие проверки. Код программы для МПУ называется микрокодом.

МПУ выполняет все машинные команды БЭВМ. Исполнение в МПУ машинной команды называется циклом команды. Цикл команды логически разбит на пять циклов:

Цикл выборки команды. Осуществляет загрузку исполняемой команды в регистр команды и частичное ее декодирование. Выполняется для каждой исполняемой команды.

Цикл выборки адреса. Предназначен для обработки адресных команд и выборки адреса операнда с учетом режимов адресации.

Цикл выборки операнда. Для тех команд, где это необходимо, размещает в БЯ второй операнд команды. Первым, напомним, является аккумулятор.

Цикл исполнения. Производится исполнение команды.

Цикл прерывания. Цикл выполняется в том случае, если разрешены прерывания и устройство ввода-вывода готово к обмену (то есть, требует прерывания – будет обсуждено позднее).

Для обеспечения работы оператора БЭВМ в ней предусмотрена микропрограммная реализация циклов пультовых операций:

Ввод адреса – адрес из клавишного регистра вводится в счетчик команд.

Чтение – информация по адресу в 1Р читается из памяти в БЯ, 1Р увеличивается на единицу.

Запись – информация из 1Я записывается в память по адресу в ЕР, 1Р увеличивается на единицу. Используется для ввода программы и данных в режиме оператора.

Пуск – осуществляет сброс состояния БЭВМ и переход к выполнению программы.

На панели оператора, кроме того, расположены другие органы управления – переключатель «Работа/Останов», который вызывает останов программы после каждой команды; переключатель «Такт», который может выполнить микрокод по одному такту, кнопка «Продолжение» - возобновляющая работу остановленной БЭВМ.

10. Организация вычислений в БЭВМ. Сдвиги, арифметические и логические операции. Цикл выборки команды.

Целые двоичные числа без знака можно использовать для представления нуля и целых положительных чисел. В 16-разрядном слове они могут изменяться от 0 до 65535. Это числа с фиксированной запятой. Целые двоичные числа со знаком используются, когда необходимо различать положительные и отрицательные числа. Отрицательные числа представляются в дополнительном коде. Это упрощает конструкцию ЭВМ. Сложение целых двоичных чисел со знаком и без знака выполняется в базовой ЭВМ с помощью команды ADD. По команде INC к содержимому аккумулятора прибавляется единица, а по команде DEC – единица вычитается. Если при этом возникает перенос из старшего разряда A, то в регистр переноса заносится 1, в противном случае в него заносится 0. Вычитание может выполняться путем сложения уменьшаемого и дополнительного кода вычитаемого. В базовой ЭВМ нет команд для выполнения умножения и деления (АЛУ не выполняет таких операций), поэтому произведение и частное необходимо получать программным путем. Для изменения знака числа необходимо его инвертировать, а затем прибавить единицу к младшему разряду (NEG). Побитовая обработка данных обеспечивается командами логического умножения, циклических сдвигов, а также командами инвертирования и очистки регистра переноса.

Команда AND выполняет над каждым разрядом аккумулятора и содержимым ячейки булевой операцию «И». Результат выполнения команды для каждой пары битов операндов равен 1 только тогда, когда оба бита равны 1, а в остальных случаях бит результата равен 0, т.е. команда позволяет выделять или очищать определенные биты слова.

Команды ROL и ROR замыкают аккумулятор и регистр переноса в кольцо и сдвигают все биты кольца влево или вправо. Арифметическими сдвигами числа (ASL и ASR) можно реализовать операции умножения или деления на 2 (один сдвиг), 4 (два сдвига), 8 (три сдвига) и т.д.

Цикл выборки команды:

```
IP ? BR, AR
BR + 1 ? IP; MEM(AR) ? DR
DR ? CR
if CR(15) = 1 then GOTO CHKBR @ 09
if CR(14) = 1 then GOTO CHKABS @ 0C
if CR(13) = 1 then GOTO CHKABS @ 0C
if CR(12) = 0 then GOTO ADDRLESS @ 78
GOTO IO @ C2
```

11. Организация массивов данных. Режимы адресации. Цикл выборки адреса и операнда БЭВМ.

Символьные данные представляются в БЭВМ в виде 8-разрядного знакового числа, которое формируется с использованием той или иной кодировки. Строковые данные формируются из массива символьных данных: т.к. аккумулятор и ячейки в памяти БЭВМ 2-х байтовые, то максимально мы можем уместить всего 2 символа в каждую ячейку памяти.

Есть два типа массивов, реализуемых в БЭВМ:

- 1) Массив, хранящий в себе количество обрабатываемых ячеек - выбирается индексная ячейка, содержащая адрес первого элемента массива. Дальше либо записывается число обрабатываемых ячеек в доп. коде в любую выбранную ячейку, либо первый элемент массива переводится в доп. код, пересылается в какую-нибудь ячейку, а потом мы работаем с ней через LOOP
- 2) Массив, содержащий в себе некоторый заданный стоп-символ - выбирается индексная ячейка, содержащая адрес первого элемента массива. Дальше поэлементно смотрим на каждый элемент массива, пока не встретим стоп-символ

Режимы адресации в бэвм. Цикл выборки адреса:

Код				Мнемоника	Описание	Реализация машинных циклов AF, OF
11	10	9	8			
0	M	M	M	ADD 0ADDR ADD \$L	Прямая абсолютная	CR → DR, DR → AR; MEM(AR) → DR
1	0	0	0	ADD (L)	Косвенная относительная	SXT_CR(0..7) → BR, BR + IP → AR, MEM(AR) → DR, DR → AR; MEM(AR) → DR
1	0	0	1		Резерв	
1	0	1	0	ADD (L) +	Косвенная автоинкрементная (постинкремент)	SXT_CR(0..7) → BR, BR + IP → AR, MEM(AR) → DR, DR + 1 → DR, DR → MEM(AR), DR - 1 → DR, DR → AR; MEM(AR) → DR
1	0	1	1	ADD - (L)	Косвенная автодекрементная (предекремент)	SXT_CR(0..7) → BR, BR + IP → AR, MEM(AR) → DR, DR - 1 → DR, DR → MEM(AR), DR → AR; MEM(AR) → DR
1	1	0	0	ADD &N ADD (SP+N)	Косвенная относительная, со смещением (SP)	SXT_CR(0, 7) → BR, BR + SP → DR, DR → AR; MEM(AR) → DR
1	1	0	1		Резерв	
1	1	1	0	ADD L ADD (IP+N)	Прямая относительная	SXT_CR(0..7) → BR, BR + IP → DR, DR → AR; MEM(AR) → DR
1	1	1	1	ADD #N	Прямая загрузка	CR(0, 7) → BR, BR → DR

Где здесь
AF и OF?

12. Управление вычислительным процессом в БЭВМ. Команды ветвлений, цикл исполнения команды LOOP.

Задача управления вычислительным процессом, т.е. требуемой последовательностью выполнения команд, решается в базовой ЭВМ при помощи команд перехода (BCS, BPL, BMI, BEQ, BR), команды «Приращение и пропуск» (LOOP) и «Останов» (HLT).

Ветвления в программах организуются с помощью команд перехода. Они не изменяют содержимое аккумулятора и регистра переноса, а лишь изменяют содержимое IP, помещая в него адрес, определяемый адресной частью команды.

 Команды ветвления			
Наименование	Мнемон.	Код	Описание
Переход, если равенство	BEQ D	F0XX	IF Z==1 THEN IP+D+1 → IP
Переход, если неравенство	BNE D	F1XX	IF Z==0 THEN IP+D+1 → IP
Переход, если минус	BMI D	F2XX	IF N==1 THEN IP+D+1 → IP
Переход, если плюс	BPL D	F3XX	IF N==0 THEN IP+D+1 → IP
Переход, если ниже/перенос	BLO D BCS D	F4XX	IF C==1 THEN IP+D+1 → IP
Переход, если выше/нет переноса	BHIS D BCC D	F5XX	IF C==0 THEN IP+D+1 → IP
Переход, если переполнение	BVS D	F6XX	IF V==1 THEN IP+D+1 → IP
Переход, если нет переполнения	BVC D	F7XX	IF V==0 THEN IP+D+1 → IP
Переход, если меньше	BLT D	F8XX	IF N⊕V==1 THEN IP+D+1 → IP
Переход, если больше или равно	BGE D	F9XX	IF N⊕V==0 THEN IP+D+1 → IP
Безусловный переход	BR D JUMP D	CEXX	IP+D+1 → IP

Циклические программы используются в тех случаях, когда требуется несколько раз выполнить набор одинаковых действий над некоторым изменяющимся набором данных. Для организации циклических программ часто используют команды перехода и команду LOOP, которая служит для увеличения на 1 содержимого адресуемой ячейки памяти и перехода к одному из двух путей продолжения программы в зависимости от знака этого содержимого (если содержимое меньше 0, то выполняется команда, следующая за LOOP, в противном случае, эта команда пропускается, т.е. выполняется команда через одну за LOOP).

Также для циклов часто используют косвенную адресацию индексных ячеек. Если произвести косвенную адресацию какой-либо из индексных ячеек, то сначала ее содержимое будет использовано в качестве адреса операнда, а затем оно автоматически увеличится на 1.

Цикл исполнения LOOP:

```

~0 + DR ? DR
~0 + DR ? BR; DR ? MEM(AR)
if BR(15) = 0 then GOTO INT @ C4
IP + 1 ? IP
GOTO INT @ C4

```

13. Подпрограммы в БЭВМ. Цикл исполнения команд перехода и возврата из подпрограммы. Стек, передача параметров. Позиционно-независимый код. Загрузчик и библиотеки.

Достаточно часто встречаются ситуации, когда отдельные части программы должны выполнить одни и те же действия по обработке данных. В подобных случаях повторяющиеся части программы выделяют в подпрограмму. В базовой ЭВМ для этой цели используется команда CALL. При оформлении подпрограммы перед ее первой командой следует разместить команду PUSH, в которую будет пересылаться адрес возврата из подпрограммы. В команде обращения к подпрограмме указывается адрес именно этой ячейки, и команда CALL M выполняет следующие действия:

передает управление команде, расположенной по адресу M+1

Последней командой подпрограммы должна быть команда выхода (команда RET). По ней осуществляется выход из подпрограммы.



ИТМО ВТ

Цикл исполнения CALL

CALL: DR после м.ц. OF содержит адрес перехода

- **DR** → **BR** ; Адрес перехода записать в BR
- **IP** → **DR** ; Подготовить адрес возврата для записи в стек
- **BR** → **IP** ; Переход на подпрограмму
- **~0 + SP** → **SP, AR** ; Уменьшить стек на 1
- **DR** → **MEM(AR)** ; Записать адрес возврата
- **ГОТО INT** ; Завершение цикла



Цикл исполнения RET

- **SP** → **AR** ; Вершину стека поместить в AR
- **MEM(AR)** → **DR** ; Прочитать адрес возврата
- **DR** → **IP** ; Вернуться из подпрограммы
- **SP + 1** → **SP** ; Увеличить стек на 1
- **ГОТО INT** ; Завершение цикла

СТЕК – память типа LIFO. «Последним вошел - первым вышел» - **LIFO** - "Last In - First Out" Обмен информацией между ЭВМ и стеком всегда выполняется через верхнюю ячейку – вершину стека. Стековая память очень удобна для упрощения решения многих задач, возникающих при работе с подпрограммами, обслуживания прерываний и т.д.

В БЭВМ нет аппаратной реализации стека, но его можно смоделировать. При этом в качестве стека используют просто часть адресной памяти. LIFO обеспечивается с помощью указателя стека (SP), который содержит адрес плавающей вершины стека.

Стек предусматривает 2 операции: загрузка нового слова (уменьшается содержимое SP, и слово записывается в ячейку, на которую он указывает) и извлечение слова из стека (читается содержимое ячейки, на которую указывает SP, а потом он увеличивается).

Push: --SP, MOV
Pop: MOV, SP++

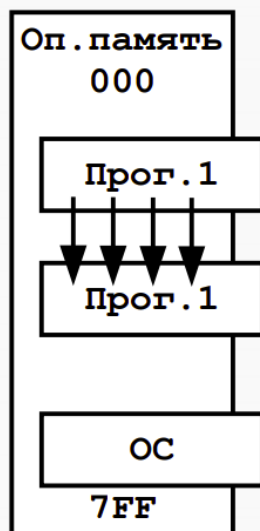
Варианты передачи данных в подпрограмму:

- Аккумулятор (Регистры Общего Назначения)
- Адресуемые ячейки памяти
- Стек – нет в БЭВМ, но можно реализовать (делать этого я, конечно, не буду)



PIC - Position Independent Code (перемещаемый код)

- Код, который работает относительно того адреса на который загружен



- Необходим для, например, модулей ядра (даже при наличии виртуальной памяти!)
- Внутри программы только относительные адреса (смещения)
- Внешние ссылки только абсолютные
- В БЭВМ

Длина перемещаемой программы в БЭВМ?

зации!

39

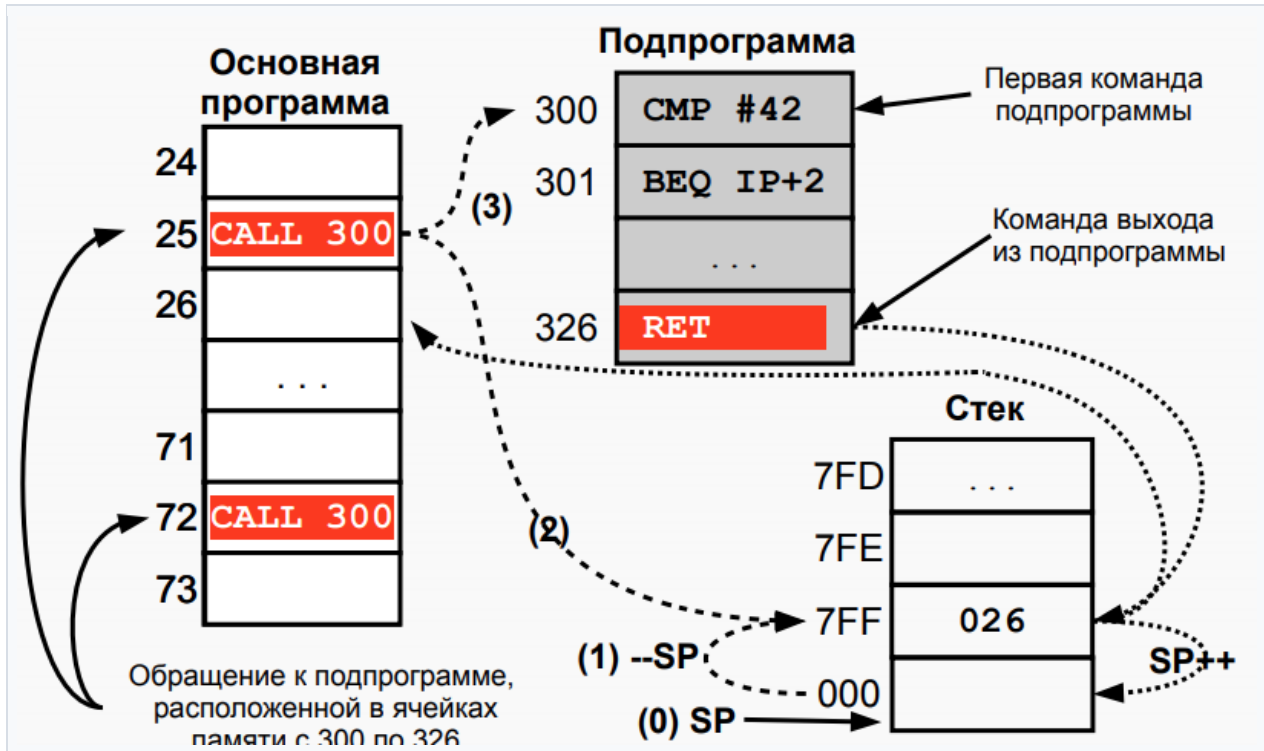


Загрузчик и динамический линковщик программ

- Любая ОС имеет соответствующую программу или часть ядра
 - Загрузка по выбранному ОС адресу (даже в виртуальной памяти)
 - Изменение константных частей адресов в программе
 - Загрузка базовых значений регистров
 - Динамическая загрузка разделяемых библиотек
 - Связывание адресов основной программы с вызываемыми библиотеками

Достаточно часто встречаются ситуации, когда отдельные части программы должны выполнить одни и те же действия по обработке данных. В подобных случаях повторяющиеся части программы выделяют в подпрограмму. В базовой ЭВМ для этой цели используются команды CALL и RET.

<структура подпрограммы>



Стек

Регистровые окна

Цикл исполнения CALL

- DR → BR; Адрес перехода записать в BR
- IP → DR; подготовить адрес возврата для записи в стек
- BR → IP; Переход на подпрограмму
- $\sim 0 + SP \rightarrow SP, AR$; уменьшить стек на 1
- DR → MEM(AR); записать адрес возврата
- GOTO INT; Завершение цикла
- SP → AR; Вершину стека поместить в AR
- DR → IP; вернуться из подпрограммы
- $SP + 1 \rightarrow SP$; увеличить стек на 1

- SP — stack pointer (указатель стека)

- **PUSH** — положить на стек

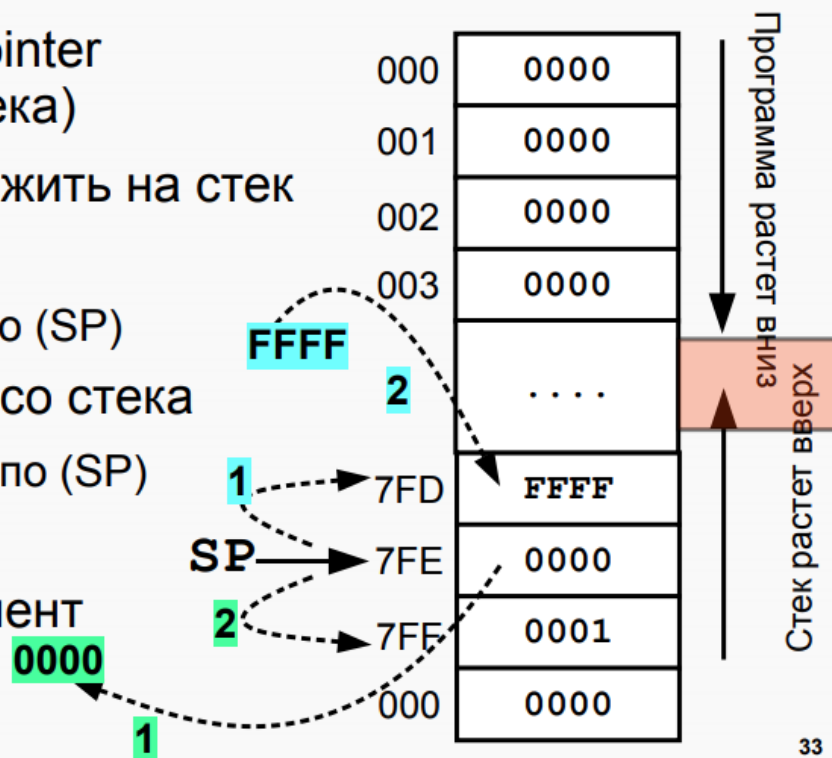
1. --SP
2. Записать по (SP)

- **POP** — снять со стека

1. Прочитать по (SP)
2. SP++

- Взять i-й элемент

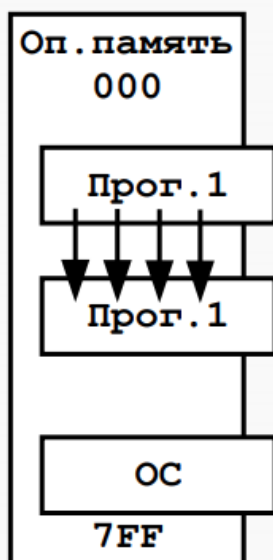
1. (SP+/-i)



33

PIC - Position Independent Code

- Код, который работает относительно того адреса на который загружен



- Необходим для, например, модулей ядра (даже при наличии виртуальной памяти!)

- Внутри программы только относительные адреса (смещения)

- Внешние ссылки только абсолютные

- В БЭВМ

Длина перемещаемой программы в БЭВМ?

39

Загрузчик и динамический линковщик программ:

Любая ОС имеет соответствующую программу или часть ядра

- Загрузка по выбранному ОС адресу (даже в виртуальной памяти)
- Изменение константных частей адресов в программе
- Загрузка базовых значений регистров
- Динамическая загрузка разделяемых библиотек
- Связывание адресов основной программы с вызываемыми библиотеками

Библиотеки:

- Набор стандартных библиотечных функций
- Разделяемые (динамически линкуемые) и архивные (статически линкуемые)
- `# find /lib /usr/lib -name "*.so" |wc -l 3510`
- Статические связывают вызовы функций с телом функции в процессе компиляции
- Динамические — в момент загрузки
- Если вам нужна функция — см. в библиотеки

14. Организация ввода-вывода в вычислительных системах. Инициация обмена, передача информации и завершение обмена. Драйверы.



Организация ввода-вывода в вычислительных системах

В вычислительных системах организация ввода-вывода реализуется за счёт шины ВУ, к которой подключаются контроллеры ВУ. Это сделано так потому, что ВУ гораздо медленнее, чем процессор, поэтому, если подключить ВУ напрямую к процессору, процессор будет тратить всё своё время на управление ВУ и ожидание его готовности, не выполняя более никакой полезной работы. Шина ВУ подразделяется на шину данных, шину управления и шину адреса. По шине адреса на контроллер приходят адреса регистров, по шине данных - данные для обмена, а по шине управления – управляющие сигналы, или приказы, которые указывают контроллеру, что именно нужно делать с полученной информацией. Обычно шину данных и шину управления не нужно использовать одновременно, поэтому их можно мультиплексировать – использовать одну и ту же шину для передачи управляющих сигналов в один момент времени и данных для обмена в другой.

Для обмена информацией используются драйверы.

Драйвер

Драйвер – это специальная программа в памяти ЭВМ, которая реализует взаимодействие ВУ, процессора и ОЗУ. Она “знает” принцип работы ВУ, адреса регистров и поддерживаемые режимы

работы. Эта программа содержит алгоритм обращения к ВУ и обработки данных поступающих с ВУ в процессор или с процессора в ВУ. Все драйверы управляются определённым единым программным интерфейсом, который задан операционной системой. Драйверы реализуют три стадии обмена: инициацию обмена, передачу данных и завершение обмена.

Инициация обмена

Начало обмена информацией. На этом этапе драйвер должен вывести на шину управления приказ для ВУ. Есть три вида инициации обмена:

- Синхронный – драйвер отдаёт приказ на шину данных в определённый момент времени.
- Асинхронный – драйвер отдаёт приказ на шину данных только если устройство явно просигнализировало, что оно готово к обмену. Готовность проверяется самим процессором.
- Управляемая прерываниями – устройство сигнализирует процессору, что оно готово обмениваться данными. Отличие от асинхронного обмена состоит в том, что процессор сразу же приступает к обработке прерывания, то есть к обмену с ВУ, в то время как при асинхронном обмене процессор, получив готовность не обязан действовать в этот же момент.

Передача данных

На этом этапе данные поступают из регистров контроллера ВУ в регистры процессора через шину данных. Есть два вида передачи данных:

- Асинхронная – процессор должен подтверждать получения каждого сегмента данных.
- Синхронная – процессор получает все данные без каких-либо дополнительных действий.

Завершение обмена

На этом этапе на процессор приходит сигнал, что передача данных завершена. Есть два вида завершения обмена:

- Синхронный – оповещение о завершении обмена происходит сразу же после завершения работы драйвера.
- Асинхронный – оповещение о завершении обмена происходит в любой момент после завершения работы драйвера.

Ввод-вывод бывает программно-управляемый и управляемый аппаратурой.

Программно-управляемый ввод-вывод

Используется для обмена небольшими блоками информации с ВУ. Для обмена информацией с ВУ используются драйверы.

Управляемый аппаратурой ввод-вывод

Используется для обмена большим объёмом данных между ВУ и ОЗУ. Так же используются драйверы, однако они нужны только для управления режимов работы контроллера, всё остальное делает сам контроллер. В процессе обмена процессор участия не принимает. Реализован при помощи специального контроллера прямого доступа в память (DMA), который самостоятельно иницирует обмен и транспортирует информацию с ВУ напрямую в предварительно подготовленные для этого ячейки памяти.

К ЭВМ можно подключать большое число разнообразных устройств ввода-вывода или внешних устройств (ВУ). Эти устройства передают в ЭВМ и получают из нее большой объем информации, который не может быть размещен только в регистрах процессора. Поэтому информация передается из ВУ в память ЭВМ и поступает на ВУ из ее памяти. При этом обмен может идти под управлением программы ЭВМ через регистры процессора (программно-управляемая передача данных) или под управлением специального внешнего устройства (контроллера прямого доступа в память), минуя процессор (передача данных при прямом доступе к памяти).

Программно-управляемый обмен осуществляется малыми порциями, при прямом доступе к памяти информация передается большими блоками. При использовании программно-управляемого обмена должна быть составлена программа, обеспечивающая пересылку данных из памяти ЭВМ в аккумулятор и далее в регистр памяти контроллера ВУ (вывод данных) или из регистра данных контроллера ВУ в аккумулятор и затем в память ЭВМ (ввод данных). В этой программе можно реализовать один из трех типов обмена: синхронный, асинхронный и по прерыванию.

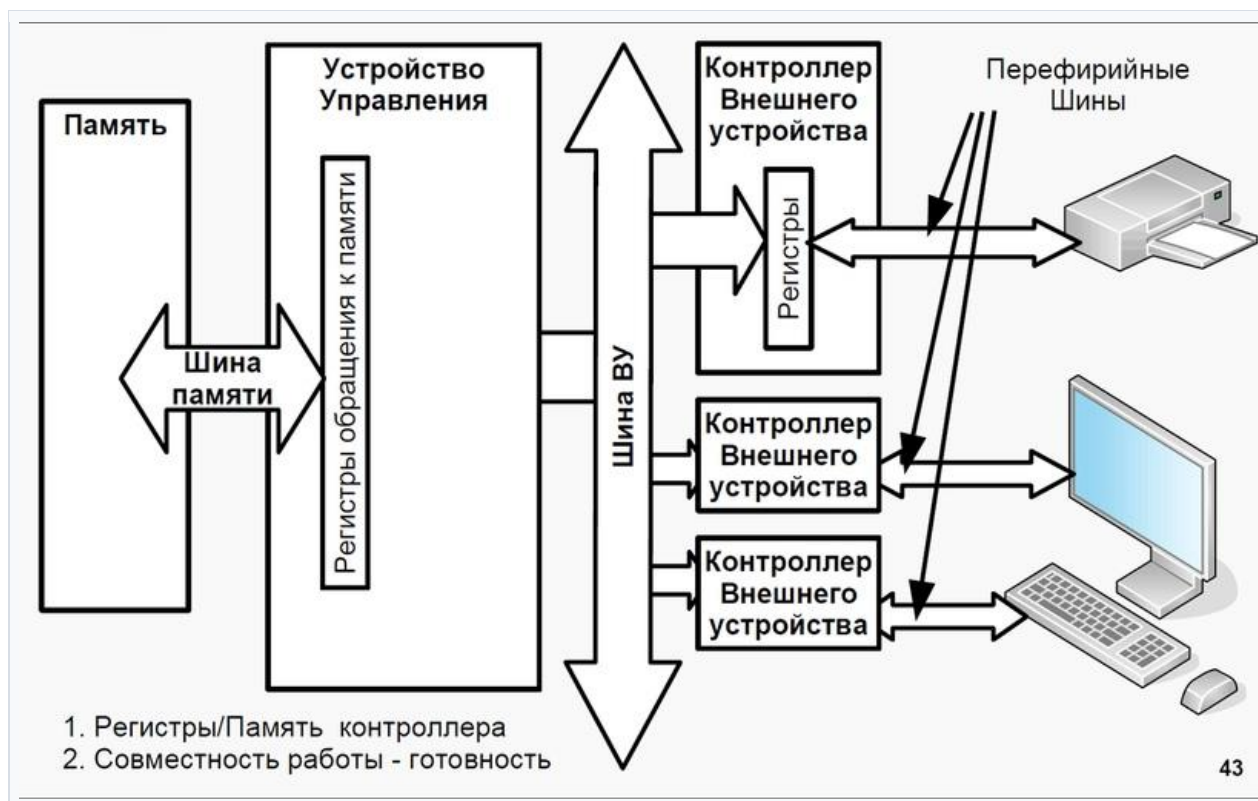
Передача данных: Синхронная/Асинхронная

Завершение обмена и получение драйвером (программой) результата: Синхронное/асинхронное

Драйверы:

- Организуют совместную работу с устройством
- «Знают» о принципах работы устройства, адресах регистров, поддерживаемых режимах работы
- Управляются единообразным программным интерфейсом

15. Организация ввода-вывода в ЭВМ. Устройства ввода-вывода, команды.



К ЭВМ можно подключать большое число разнообразных устройств ввода-вывода или внешних устройств (ВУ). Эти устройства передают в ЭВМ и получают из нее большой объем информации, который не может быть размещен только в регистрах процессора. Поэтому информация передается из ВУ в память ЭВМ и поступает на ВУ из ее памяти. При этом обмен может идти под управлением программы ЭВМ через регистры процессора (программно-управляемая передача данных) или под управлением специального внешнего устройства (контроллера прямого доступа в память), минуя процессор (передача данных при прямом доступе к памяти).

Программно-управляемый и управляемый прерываниями ввод-вывод, прямой доступ к памяти.

Преимущества и недостатки

Программно-управляемый ввод-вывод - все действия, связанные с вводом-выводом управляются процессором. То есть ВУ работает с памятью через процессор. Главный минус подобной стратегии ввода-вывода: процессор простаивает в ожидании готовности ВУ. Процессор будет простаивать до тех пор, пока ВУ не подаст сигнал о готовности передачи информации. Преимущества - легкость реализации.

Управляемый прерываниями ввод-вывод - работа ввода и вывода также происходит через процессор. Но теперь во время ожидания получения сигнала готовности ВУ передавать информацию, процессор может выполнять какую-либо полезную работу. Как только ВУ готова к обмену - она посылает сигнал готовности и просит прерывание (interruption) работы процессора. Такой подход с точки зрения производительности более правильный, но он требует больших усилий на обработку прерываний

Прямой доступ к памяти (Direct Memory Access или DMA) — ВУ работает с памятью напрямую, избегая процессор. Такой способ не нагружает процессор, следовательно, намного быстрее. Но для реализации данного подхода требуются более сложные контроллеры, что значительно удорожает конструкцию.

Какие режимы передачи данных и управления вводом-выводом реализуемы в БЭВМ?

Почему не возможно реализовать другие?

В БЭВМ реализованы

1.программно-управляемый ввод-вывод и управляемый прерываниями.

Программно-управляемый ввод-вывод достигается с помощью ожидания непосредственно в аккумуляторе бита статуса. Управляемый прерываниями достигается с помощью системы прерываний.

Прямой доступ к памяти не реализован в БЭВМ, так как нет соответствующих схем в контроллере ВУ!!!

Регистры контроллера

SR — однобитовый статусный регистр, в котором регистрируется готовность ВУ к обмену данными. Содержит 1 когда готов, 0 — не готов. При извлечения его содержимого в АС будет находится на месте 6 бита (то есть IN SR вернет либо 0000 0000, либо 0100 0000).

MR — 4-битовый регистр управления. Содержит в себе номер вектора прерывания и бит разрешения прерывания.

DR — 8-битовый регистр данных. Через него идет обмен данными. При извлечении его содержимого в АС занимает младший байт, не трогая старший.

Команды

Команда ввода-вывода

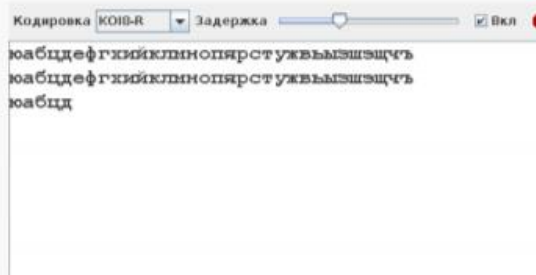
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
0	0	0	1	Приказ				Регистр устройства							

Наименование	Мнемон.	Код	Описание
Запрет прерываний	DI	1000	
Разрешение прерываний	EI	1100	
Ввод	IN REG	12XX	REG → AC
Вывод	OUT REG	13XX	AC → REG
Прерывание	INT NUM	18XX	Програмное прерывание с вектором NUM
Возврат из прерывания	IRET	0B00	(SP)+ → PS, (SP)+ → IP



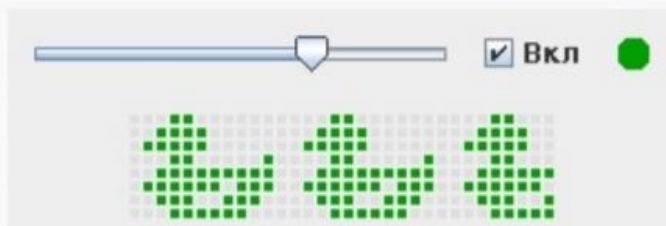
БЭВМ: ВУ-5 Текстовый принтер (регистры 0xC - 0xF)

- Печатает символы из РДВУ в заданной кодировке
- Регулируемая задержка (время печати) от 100 мс до 10 с
- Перевод строки по символу CR (0A₁₆)
- NUL (0) — очистка листа
- Остальные - неопределенное поведение



БЭВМ: ВУ-6 Бегущая строка (регистры 0x10 - 0x13)

- Размер матрицы: 32x8
- Сдвиг при записи нового значения в РДВУ
- Новое значение — справа
- Младший бит - нижний





БЭВМ: ВУ-7 8-ми разрядный 7-сегм. индикатор (0x14 - 0x17)

- Формат РДВУ:

7	6	5	4	3	2	1	0
0	Поз	Симв.					
- Симв==(A₁₆) — установка в разряде знака «-»
- Симв==(B₁₆-F₁₆) — сброс разряда



БЭВМ: ВУ-8 клавиатура (регистры 0x18-0x1C)

- Код нажатой клавиши в выбранной кодировке устанавливается в РДВУ
- Автоматически устанавливается готовность





БЭВМ: ВУ-9 Цифровая клавиатура (0x1C-0x1F)

- При нажатии клавиши ее код помещается в РДВУ
- Клавиша 0-9 код 0-9
- Клавиша «-» код A
- «+» код B
- «/» код C
- «*» код D
- «.» код E
- «=» код F

7	8	9	/
4	5	6	*
1	2	3	-
0	.	=	+



БЭВМ: ВУ-0 Таймер (регистры 0x0, 0x1)

- Устанавливает готовность (и вызывает прерывание) раз в $100 \cdot DR$ миллисекунд
 - Смещенная десятичная фиксированная точка
- Если $DR == 0$ готовность не устанавливается
- Можно использовать для организации синхронного обмена (Как?)



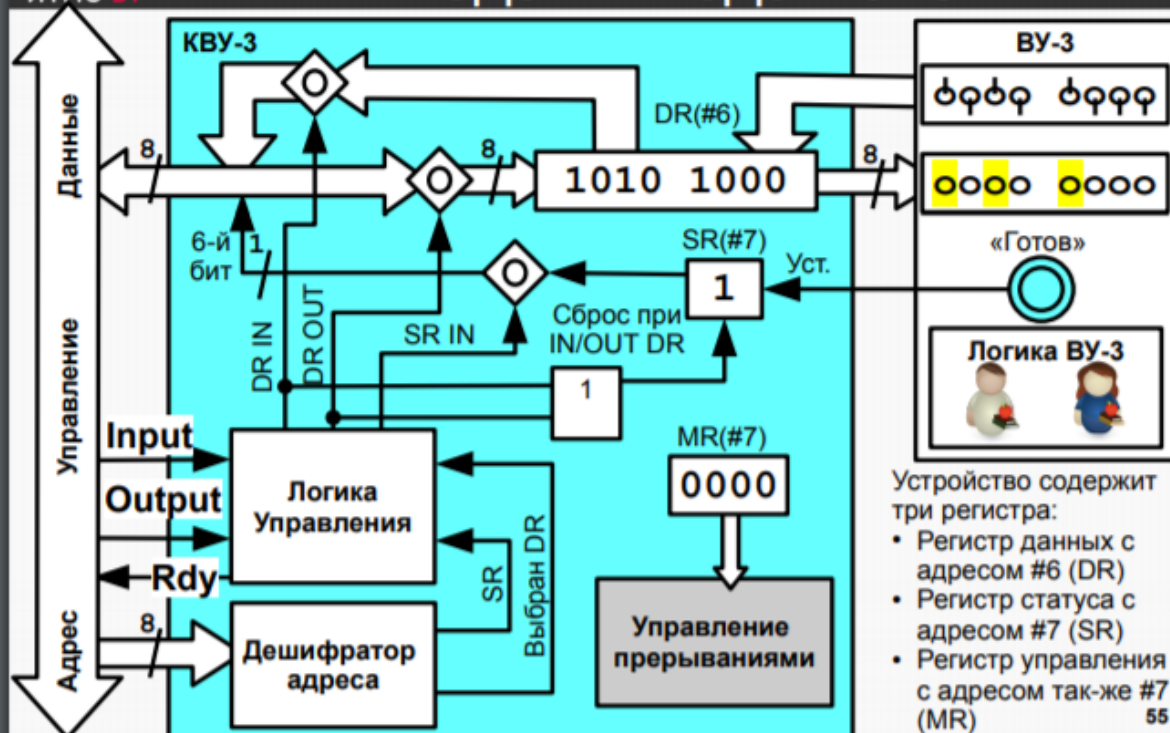
БЭВМ: у-во ввода-вывода ВУ-4 (регистры 0x8 - 0xB)

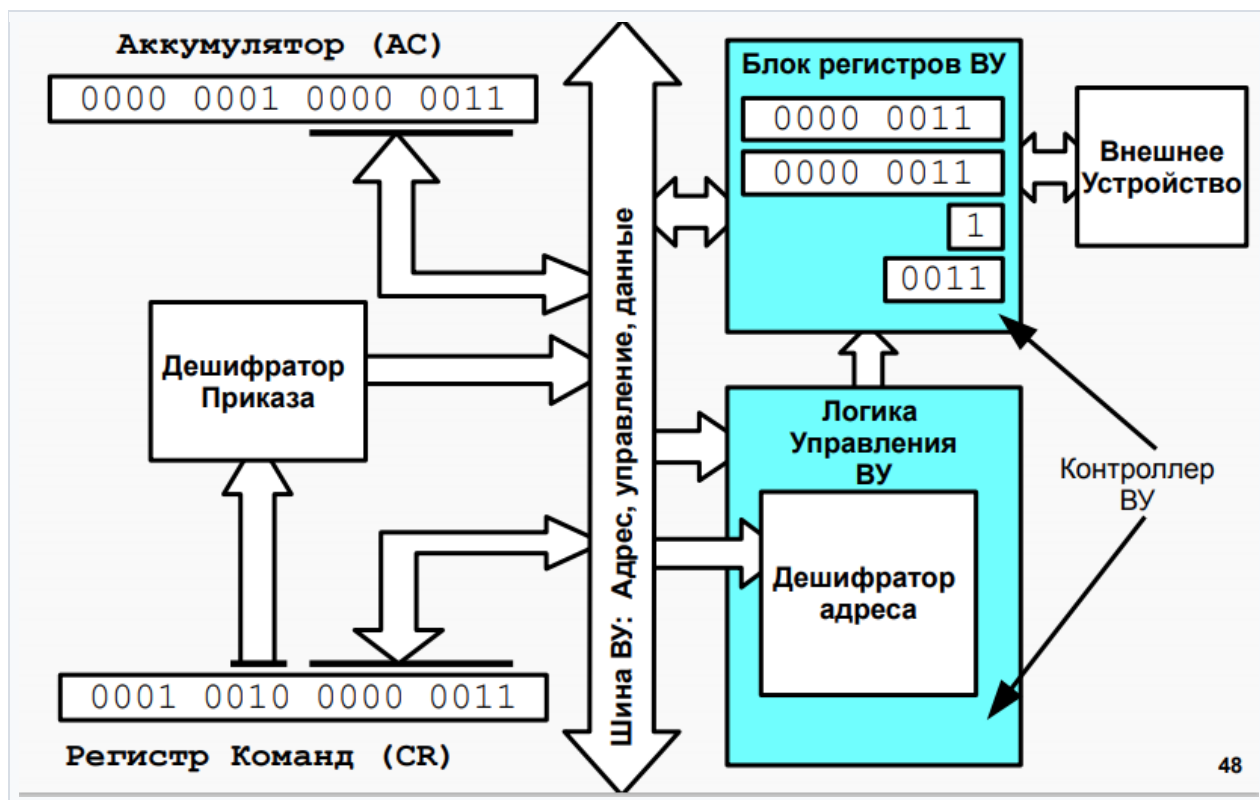
- По функционалу похоже на ВУ-3
- Адресуется 4-мя регистрами
- Отдельные регистры для входных (0x8 и выходных (0x9) данных
- Регистр состояния по адресу (0xA)
 - Бит #6 все также отвечает за готовность ввода-вывода
- Регистр управления по адресу (0xB)
- Все регистры доступны для чтения записи
- Позволяет реализовать сложные конфигурации подключения

56



Контроллер и устройство ввода-вывода ВУ-3





48

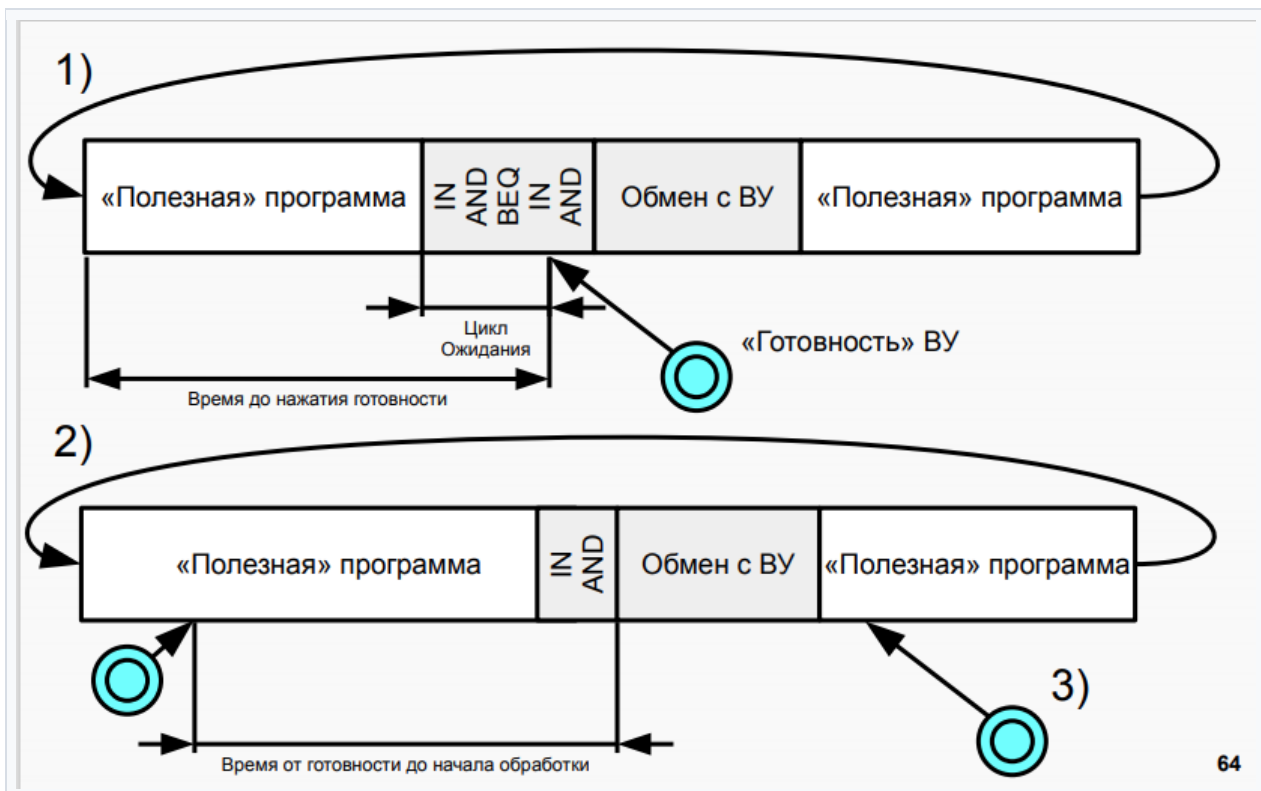
Между ВУ и процессором установлены простейшие контроллеры, каждый из которых содержит: регистр данных (для обмена данными между ВУ и процессором), дешифратор адреса (позволяющий выделить обращение к данному ВУ среди всех обращений у устройств ввода-вывода, подключенным к процессору), логику управления (декодирующий приказ от процессора на выполнение тех или иных операций) и регистр состояния (в котором хранится информация о готовности ВУ к обмену данными с процессором). В контроллерах простейших ВУ обычно используется однобитовые регистры состояния, которые часто называют флагом. Контроллеры ВУ связаны с процессором шинами ввода и вывода информации, шиной адреса и шиной управления, по которым передаются приказы от процессора и сведения о состоянии ВУ.

Устройства ВУ: вывода, ввода, ввода-вывода, ввода-вывода PRO, таймер, принтер, бегущая строка, 7 разр. индикатор, клавиатура, numrad

16. Организация асинхронного обмена в БЭВМ. Пример программы. Временные издержки асинхронного обмена.

Алгоритм программы асинхронного обмена: сначала проверяется готовность ВУ к обмену и, если оно готово, дается команда на обмен (ввод или вывод). ВУ сообщает о готовности установкой в единицу флага в контроллере ВУ. При асинхронном обмене ЭВМ должна тратить время на ожидание момента готовности, а так как готовность проверяется командным путем, то в это время ЭВМ не может выполнять никакой другой работы по преобразованию данных.

	ORG	0x10	
START:	CLA		
S1:	IN	5	; Ожидание ввода первого символа
	AND	#0x40	; Бит 6 SR == 0 («Готов» нажата?)
	BEQ	S1	; Нет - "Спин-луп"
	IN	4	; Ввод первого символа
	SWAB		; Сдвиг первого символа
	ST	RES	; Сохранение его в ячейке RES



Асинхронный обмен - БЭВМ запрашивает готовность у КВУ, пока КВУ не ответит, что оно готово и не начнется передача данных.

Ввод двух символов с устройства ввода ВУ-2 (DR#4, SR#5)				
	ORG	0x10		
START:	CLA			
S1:	IN	5	; Ожидание ввода первого символа	
	AND	#0x40	; Бит 6 SR == 0 («Готов» нажата?)	
	BEQ	S1	; Нет - "Спин-луп"	
	IN	4	; Ввод первого символа	
	SWAB		; Сдвиг первого символа	
	ST	RES	; Сохранение его в ячейке RES	
S2:	IN	5	; Ожидание ввода второго символа	
	AND	#0x40	; Бит 6 SR == 0 («Готов» нажата?)	
	BEQ	S2	; Нет - "Спин-луп"	
	LD	RES		
	IN	4	; Ввод второго в младшие 8 разрядов A	
	ST	RES		
	HLT			
RES:	WORD	?	; Ячейка для записи слова "ДА"	

Сколько циклов команд БЭВМ будет ждать ввода второго символа?

17. Организация прерываний в БЭВМ. Вектора прерываний, контроллер прерывания.

Прерывание — сигнал от программного или аппаратного обеспечения, сообщающий процессору о наступлении какого-либо события, требующего немедленного внимания. В контексте БЭВМ это сигнал о готовности обмена данными с некоторым ВУ.

Также стоит отметить преимущества над асинхронным режимом передачи данных (если Вы помните, то реализовывать его можно двумя способами: через spin-loop или одну проверку с последующим окном ввода-вывода в циклической полезной программе):

процессор не простаивает в ожидании готовности ВУ

организованная работа сразу со всеми нужными нам ВУ

если все ВУ не готовы к обмену, то процессор занят полезной работой

Ярким отличием прерываний является обязательное сохранение состояния процессора в момент прерывания, чтобы когда прерывание было обработано, мы смогли вернуться в основную программу и смогли продолжить ее выполнение без разнообразных коллизий.

Адр .	Сод .
0x0	Адр 0
0x1	PS 0
0x2	Адр 1
0x3	PS 1
0x4	Адр 2
0x5	PS 2
0x6	Адр 3
0x7	PS 3
0x8	Адр 4
0x9	PS 4
0xA	Адр 5
0xB	PS 5
0xC	Адр 6
0xD	PS 6
0xE	Адр 7
0xF	PS 7

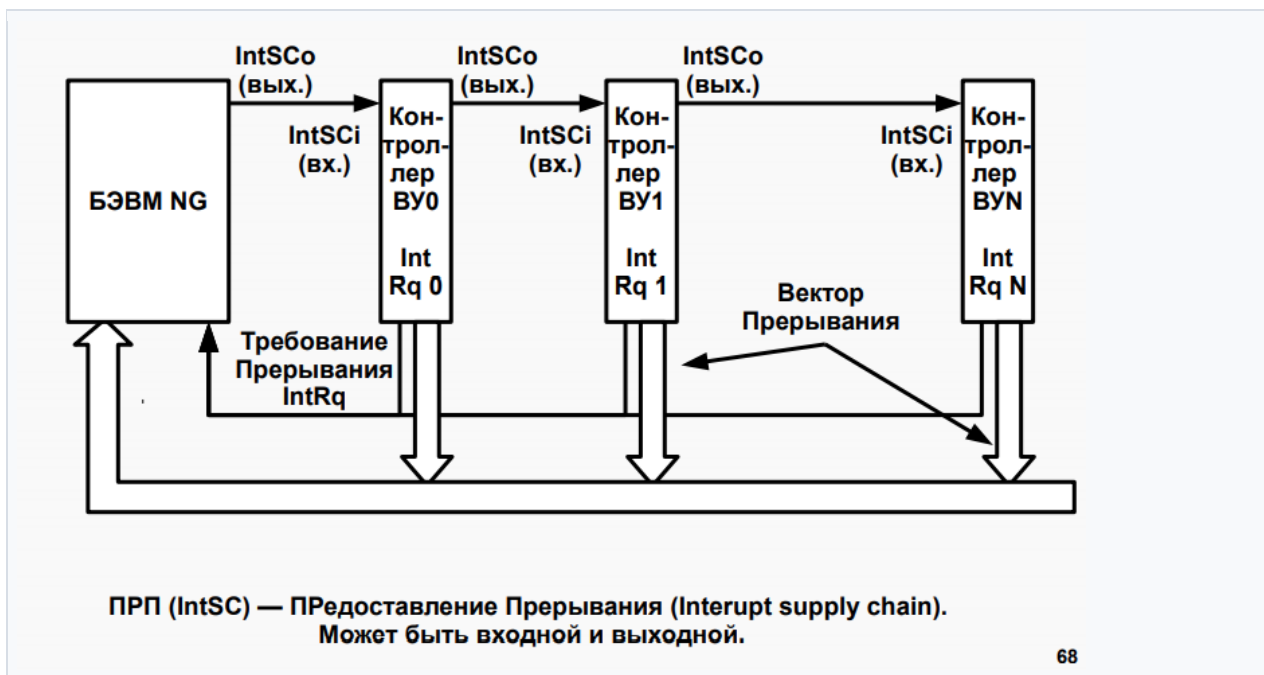
Вектор прерывания:

- Совокупность адреса программы обработки прерывания и регистра состояния (PS)

- Необходимо инициализировать перед началом обработки прерывания
- Хотя бы установить на подпрограмму, которая ничего не делает
- Ответственность OS и БИОС
- В БЭВМ-NG ячейки с 0x000 по 0x10
- Всего 8 векторов, по два слова на вектор
- На одном векторе может быть несколько прерываний

Регистр управления или Management Register — регистр, хранящий в себе информации о разрешении/запрете прерывания от данного ВУ и вектора прерывания, к которому привязано ВУ.

В 3х младших битах содержится номер вектора прерывания, в 3 бите будет 1, если прерывания от данного ВУ разрешены и 0 — если запрещены.



68

Сигнал последовательно проходит через контроллеры 0, 1, 2 и останавливается в логике управления КВУ-3

Внутри ЛУ происходит магия и открывается вентиль у MR (переключаемся на 2 картинку)

MR подключается к CR через шину адреса и младшие три бита MR записываются в младшие 3 бита CR, при этом оставшиеся 5 бит младшего байта CR заполняются нулями.

Параллельно с этим ЛУ маскирует сигнал IntSCo и сигнал предоставления прерывания не передается в следующие контроллеры

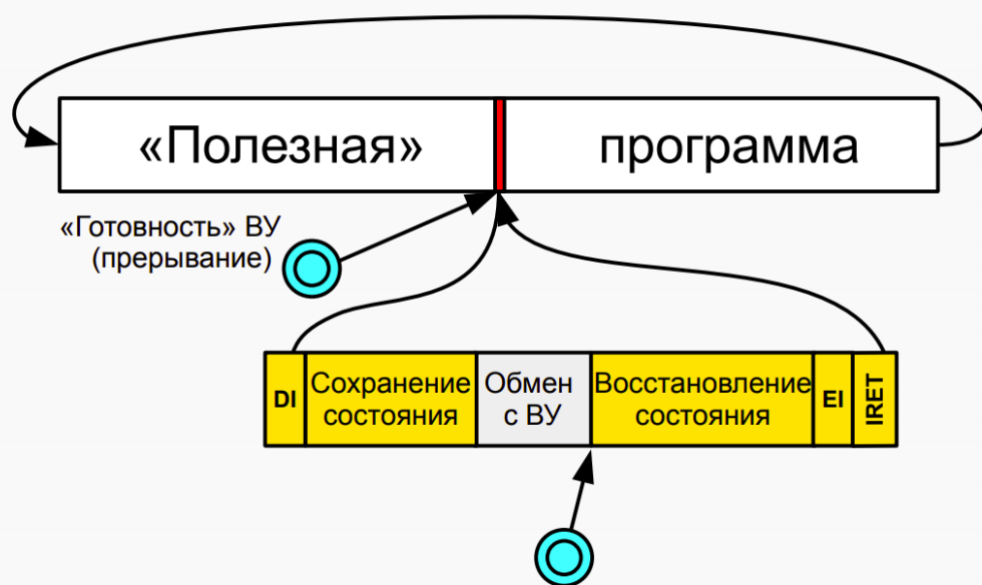
Также на шину управления поступает сигнал из ЛУ IntV (Interrupt Vector), который информирует о предоставлении данным контроллером номера вектора прерывания.

Вектор прерывания — это совокупность адреса обработчика прерывания и регистра состояния, с которым этот обработчик стартует (данное высказывание справедливо в контексте БЭВМ, в других ЭВМ количество нужных ячеек для вектора прерывания варьируется).

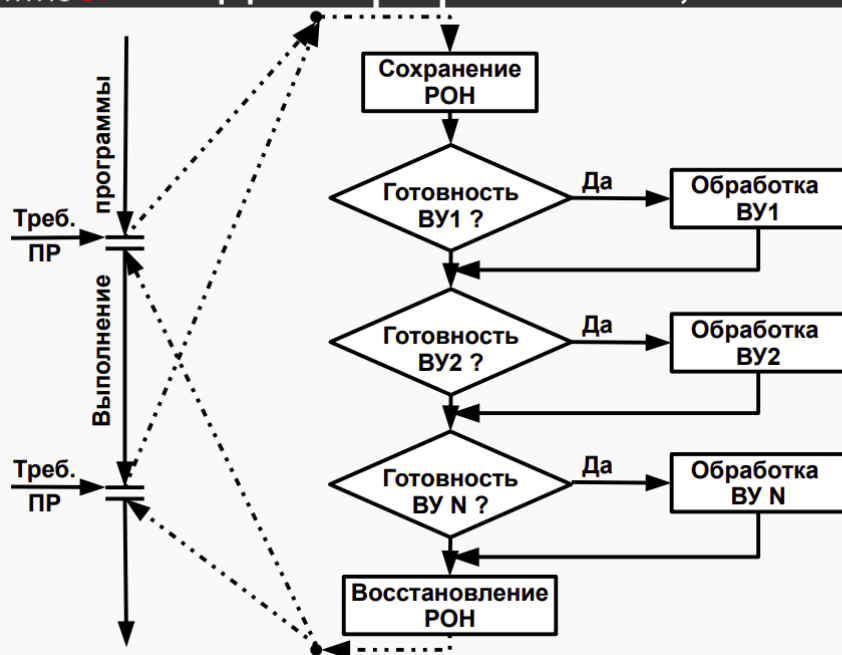
В БЭВМ доступно 8 векторов прерываний, и расположены они в ячейках памяти 0x0 — 0xF включительно. На один вектор прерывания может приходиться несколько прерываний.

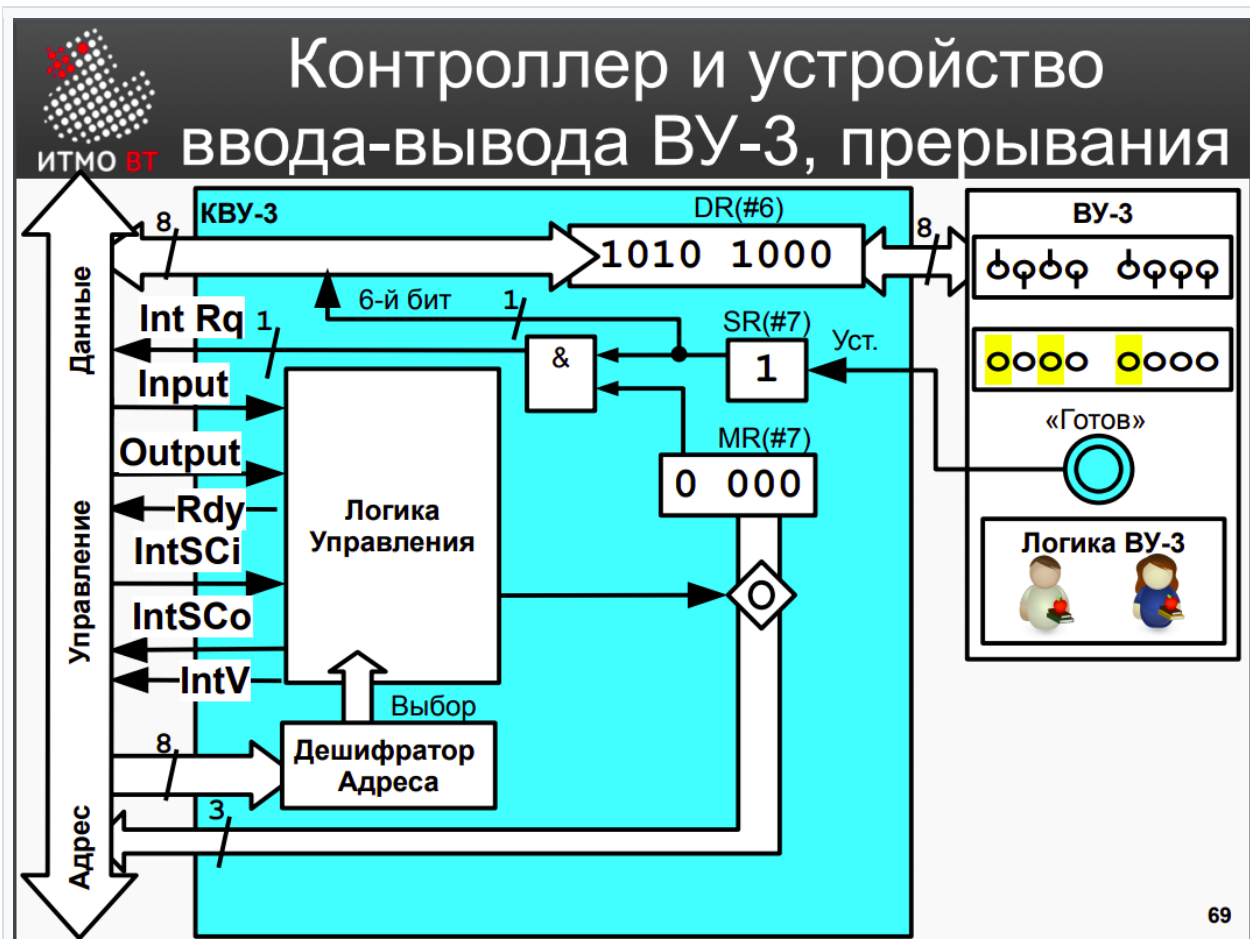


Инициация обмена по прерыванию



Логика обработки и приоритет: одно прерывание, много ВУ.





Сигнал (ы)	Направление CPU . . .CNTRL	Описание
Addr0...7	CR → Деш.адреса	Шина адреса контроллеров внешних устройств
Addr0...2	CR ← MR контр.	Шина номера вектора прерывания (НВП)
IntV	CPU ← л/у контр.	Сигнал подтверждения передачи в CR НВП
Input	Деш.команд → л/у	Сигнал управления "Ввод"
Output	Деш.команд → л/у	Сигнал управления "Вывод"
Rdy	CPU ← л/у	Сигнал подтверждения готовности
IntRq	PS (IRQ) ← SR контр.	Запрос прерывания
IntSCi IntSCo	CPU IRQSC → IntSCi0 IntSCo0 → IntSCi1 IntSCo1 → IntSCi2 ...	Цепочка сигналов предоставления прерывания. Передается далее в случае отсутствия требования прерывания в контроллере.
SYN	Такт.ген → л/у	Синхросигнал тактового генератора
Data0...7	АС ↔ рег. контр.	Двунаправленная шина данных связи АС с регистрами контроллера

Бит	Мнем.	Содержимое
0	C	Перенос
1	V	Переполнение
2	Z	Ноль
3	N	Знак
4	O	0 – используется для организации безусловных переходов в МПУ
5	EI	Разрешение прерываний
6	IRQ	Требование прерывания (логическое “И” шины запроса на прерывание и бита 5 PC – “разрешение прерываний”)
7	W	Состояние тумблеров РАБОТА/ОСТАНОВ (1 – РАБОТА)
8	P	Программа

Команды для работы с прерываниями

IRET (0x0B00) Возврат из прерывания Возвращает состояние процессора до прерывания Безадресная команда - Будут выполняться: IF, EHC, INT Не будут выполняться: AF, OF Потактово

DI (0x1000)

1. Запрет прерываний
2. Устанавливает 5 бит PS в 0
3. Команда ввода-вывода
4. -
5. Будут выполняться: IF, EHC, INT
6. Не будут выполняться: AF, OF
7. Потактово

› EI (0x1100)

1. Разрешение прерываний
2. Устанавливает 5 бит PS в 1
3. Команда ввода-вывода
4. -
5. **Будут выполняться:** IF, EXC, INT
6. **Не будут выполняться:** AF, OF
7. Потактово

1. Обрабатываются ли прерывания в пошаговом режиме (режиме останов) работы программы? Почему?

В режиме останов — нет, так как цикл INT требует для отработки 1 в 7 бите PS.

В какой момент вызывается следующее прерывание, если пришло несколько запросов на прерывание (при PS 0x180)?

Как только отработал обработчик прерывания для первого прерывания, вызывается в самом его конце IRET. В EXC он вернет все на круги своя и в PS в 5 бите снова окажется 1 (EI) и потом в 6 бите 1 (IRQ) и цикле INT команды IRET начнется обработка следующего прерывания и так будет до тех пор, пока есть запросы на прерывание. Приоритет обработки я описывал выше.

Сигнал предоставления прерывания проходит через все КВУ, пока не дойдет до того, чье ВУ вызвало прерывания. Как именно КВУ понимает что именно его ВУ вызывает прерывание — секрет и спрятано в логике управления данного КВУ.

В БЭВМ приоритет получения КВУ сигнала предоставления реализован аппаратно, как Вы можете увидеть по картинке. Отсюда вытекает очень интересный факт, что приоритет обработки прерываний от разных ВУ реализован АППАРАТНО.

После всех команд (кроме HLT (0100)) идет цикл прерывания (INT) — именно в нем проверяется есть ли запрос на прерывания (при этом мы пока что не знаем какое ВУ требует прерывания)

18. Организация обмена по прерыванию программы в БЭВМ. Пример программы. Цикл прерывания.

В БЭВМ доступно 8 векторов прерываний, и расположены они в ячейках памяти 0x0 — 0xF включительно. На один вектор прерывания может приходиться несколько прерываний. До начала необходимо загрузить вектора прерываний в ВУ и разрешить прерывания.

Цикл прерывания:

```

1) IF PS(W) == 0: GOTO STOP; Если тумблер РАБОТА\ОСТАНОВ установлен на ОСТАНОВ, то работа БЭВМ прекращается
2) IF PS(IRQ) == 0: GOTO INFETCH; Если запроса на прерывания нет (0 в 6 бите PS), то переходим к циклу выборки СЛЕДУЮЩЕЙ команды
3) IRQSC; Формируется сигнал предоставления прерывания

4) SP + ~0 -> SP, AR;
5) IP -> DR; ///IP -> -(SP)
6) DR -> MEM(AR);
7) SP + ~0 -> SP, AR;
8) PS -> DR; ///PS -> -(SP)
9) DR -> MEM(AR); LTOL(CR) -> BR; Выделили номер вектора прерывания
10) SHL(BR) -> BR, AR; Выделили адрес адреса обработчика прерывания (тавтология, привет)
11) MEM(AR) -> DR; в DR адрес обработчика прерывания
12) DR -> IP
13) LTOL(BR + 1) -> AR ; в AR адрес, в котором лежит PS для обработчика прерывания
14) MEM(AR) -> DR
15) DR -> PS

```

Готовность ВУ1: 2*А→РДВУ1, Готовность ВУ3: РДВУ3→ яч.3F

```

      ORG 0x0      ; Инициализация векторов прерывания
V0:   WORD $DEFAULT,0x180 ; Вектор прерывания #0
V1:   WORD $INT1,0x180   ; Вектор прерывания #1
V2:   WORD $DEFAULT,0x180 ; Вектор прерывания #2
V3:   WORD $INT3,0x180   ; Вектор прерывания #3
      ...
V7:   WORD $DEFAULT,0x180 ; Вектор прерывания #7
DEFAULT: IRET          ; Просто возврат

START: ORG 0x020 ; Загрузка начальных векторов прерывания
      DI
      CLA
      OUT 1      ; MR КВУ-0 на вектор 0
      OUT 5      ; MR КВУ-2 на вектор 0
      LD #9      ; разрешить прерывания и вектор №1
      OUT 3      ; (1000|0001=1001) в MR КВУ-1
      LD #0xB    ; разрешить прерывания и вектор №3
      OUT 7      ; (1000|0011=1011) в MR КВУ-3
      ...
      BR $PROG

```

Готовность ВУ1: 2*A→РДВУ1, Готовность ВУ3: РДВУ3→ яч.3F

PROG:
EI ; Установка состояния разр. прерывания
CLA ; Первоначальная очистка аккумулятора
INCLP: INC ; Цикл для наращивания
BR INCLP ; содержимого аккумулятора

ORG 0x03F
 ; Ячейка для хранения кодов, поступающих с ВУ-3
IO3: WORD ?

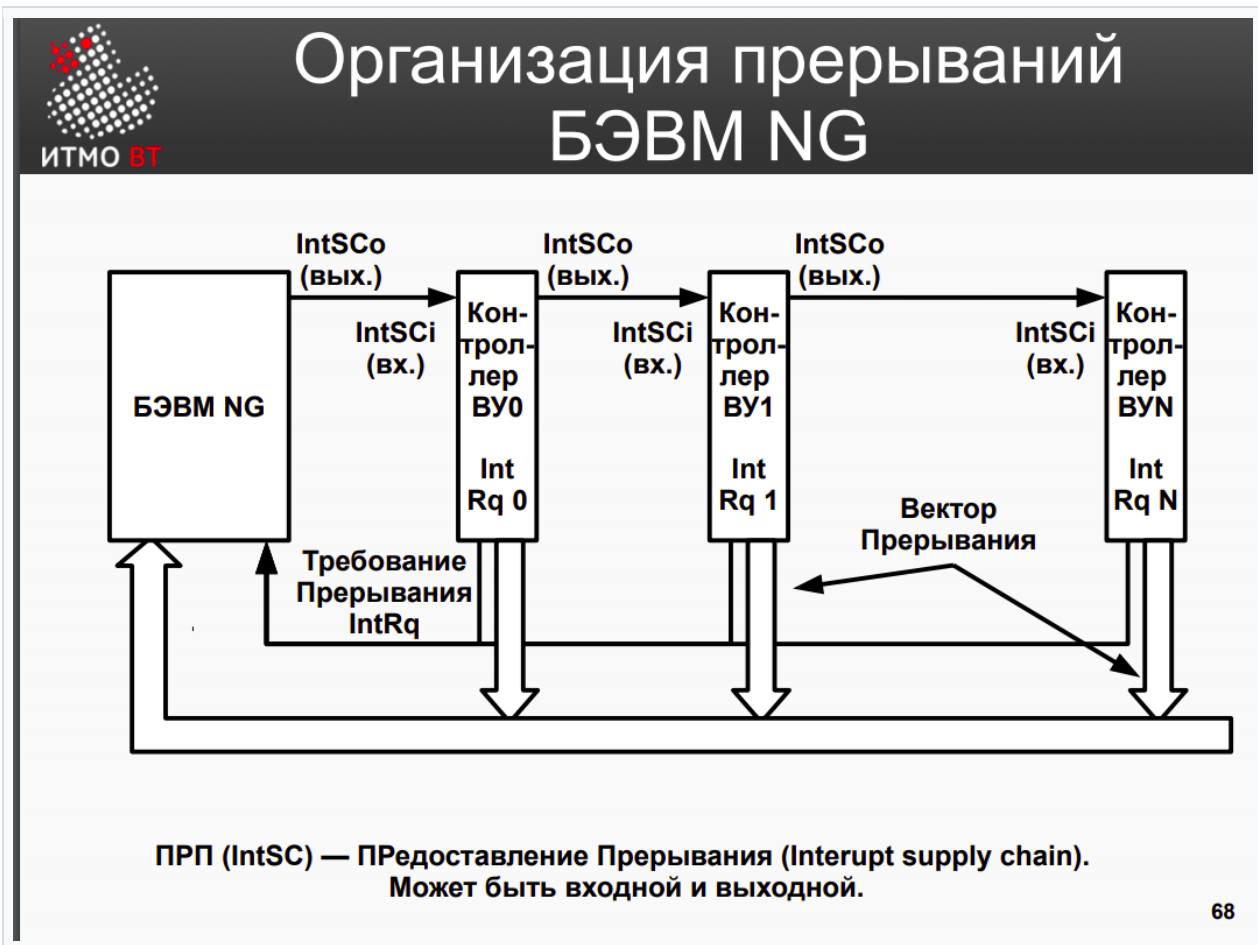
ORG 0x040
INT1: ; Прерывание сохранило содержимое PS
NOP ; отладочная точка останова (NOP/HLT)
PUSH ; Сохранили AC
ASL ; Умножили AC на 2
OUT 2 ; Записали в РДВУ-1 (DR#2)
POP ; Вернули AC назад
NOP ; отладочная точка останова (NOP/HLT)
IRET ; Возврат из обработки прерывания

Готовность ВУ1: 2*A→РДВУ1, Готовность ВУ3: РДВУ3→ яч.3F

ORG 0x040
INT3: ; Прерывание сохранило содержимое PS
NOP ; отладочная точка останова (NOP/HLT)
PUSH ; AC кладем в стек
CLA
IN 6 ; РДВУ-3
ST \$IO3 ; сохранение
NOP ; отладочная точка останова (NOP/HLT)
POP ; AC вынимаем из стека
IRET ; Возврат из обработки прерывания

Используется при работе с низкоскоростными ВУ или когда момент передачи данных заранее неизвестен. Обмен данными между ЭВМ и ВУ инициируется сигналом с ВУ. Для реализации данного типа обмена используется аппаратная проверка наличия внешнего прерывания, т. е. сигнала готовности по линии "Запрос прерывания". По завершении цикла исполнения текущей команды происходит переход к циклу прерывания, который есть у всех команд кроме EI (Разр. прер.), DI (Запр. прер.) и HLT (Останов). Если в этот момент на линии «Запрос прерывания» нет сигнала о готовности ВУ или прерывания запрещены, то выполняется следующая команда. Иначе, прерывания запрещаются, в ячейку с адресом 000 заносится содержимое СК, и управление передается команде, расположенной в ячейке 001, с которой начинается программа обработки прерываний. Программа обработки прерываний запоминает в памяти содержимое А в ячейку **SAVED_A** и содержимое С в ячейку **SAVED_C**. Т.е. минимальная информация о прерванной программе хранится в ячейках 000, **SAVED_A** и **SAVED_C**. Производится поиск источника

прерываний, и переход к последовательности действий по работе с конкретным ВУ. Затем выполняется передача данных и сброс флага готовности ВУ.



68

Цикл прерывания (2)

- $\sim 0 + SP \rightarrow SP, AR$
 - $PS \rightarrow DR$
 - $DR \rightarrow MEM(AR);$
- а также...
- $LTO L(CR) \rightarrow BR$; младшие 8 разрядов CR (номер вектора прерывания) записать в BR
- $SHL(BR) \rightarrow BR, AR$; Вычисляем адрес ячейки с переходом на подпрограмму обработки прерывания, как номер вектора * 2



Цикл прерывания (3)

- **MEM (AR) → DR**; адрес обработчика прерывания записать в DR ...
- **DR → IP** ; ... а затем в IP
- **LTO L (BR + 1) → AR** ; ... выбрать адрес следующей ячейки вектора прерывания, ограничивая результат 8-ю разрядами
- **MEM (AR) → DR**; содержимое PS обработчика прерывания записать в DR ...
- **DR → PS** ; ... а затем установить его в регистр

19. Понятие многоуровневой ЭВМ. Понятие и пример программы на разных уровнях.

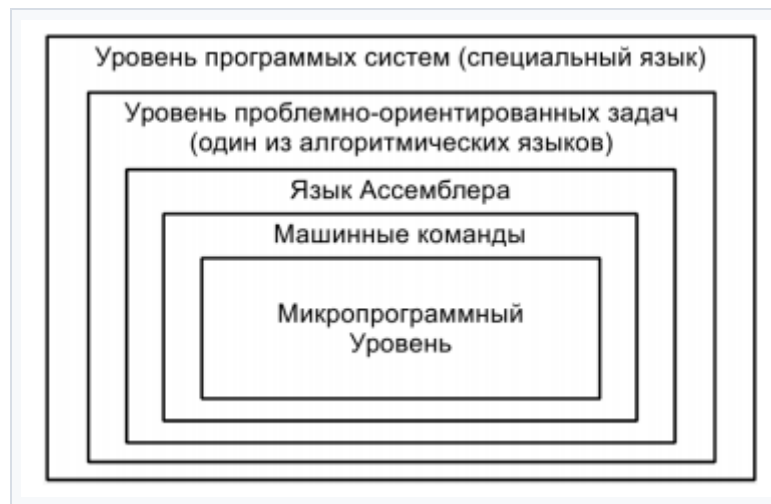
Возможность исполнения на ЭВМ программы, написанной на алгоритмическом языке, обеспечивается с помощью специальных системных программ: компиляторов и интерпретаторов. Компиляция, заключается в том, что процесс выполнения алгоритма осуществляется лишь после завершения процесса перевода исходной программы. В интерпретации же каждый оператор исходной программы заменяется программой-интерпретатором на эквивалентную последовательность машинных команд непосредственно перед исполнением. В отличие от компиляции, в интерпретации во время решения задачи машине нужны и исходная программа, и программа-интерпретатор.

Затраты на создание компиляторов (интерпретаторов) и время на процесс перевода программы в значительной мере определяются сходством компилируемого и получаемого языков. Поэтому алгоритмические языки не сразу переводят программу на язык машинных команд. Существует определенная иерархия языков программирования, в которой более сложный язык базируется на предшествующем.

Примером промежуточного языка служит язык символического кодирования команд, часто называемый языком ассемблера. Языки ассемблеров (разработанные для каждого типа ЭВМ) - это первые средства автоматизации программирования в вычислительной технике. В них допускается использование символических имен и меток. Компиляторы с таких языков называются ассемблерами. Они отводят определенные ячейки памяти для символических переменных, организуют связи между различными частями программы, что резко облегчает программирование по сравнению с программированием на уровне команд.

Человеку, работающему с ЭВМ на том или другом языке, чаще всего кажется, что язык, на котором он общается с ЭВМ, – это ее машинный язык. Следовательно, разным пользователям одной и той же ЭВМ может казаться, что они работают на разных вычислительных машинах. Отсюда появились понятия: виртуальная (кажущаяся) ЭВМ и многоуровневая ЭВМ.

Многоуровневая ЭВМ – это вычислительная машина, имеющая средства для работы с n различными уровнями языков программирования. Нижний язык, или уровень, является наиболее простым, верхний –наиболее сложным. Такую машину можно рассматривать как n различных виртуальных машин, каждая из которых имеет свой машинный язык. Сложность аппаратной реализации этих виртуальных машин возрастает по мере увеличения номера уровня.☒



Рассмотрим примеры программ разных уровней.

Уровень программных систем.

В качестве примера можно привести сложение в экселе (прикладная программа).

$X=Y+Z$

X и Z лежат в ячейках, формулу помещаем туда куда надо - сами знаете как это работает.

Эксель преобразует то что мы сделали в язык, понятный более низким уровням машины.

Уровень проблемно-ориентированных задач.

Java - один из алгоритмических языков. Думаю что все, кто это читают, и без меня в состоянии написать программу сложения двух чисел на джаве.

Язык Ассемблера.

```

ORG ...
X: WORD...

Y: WORD...

Z: WORD...
```

```

LD Y
ADD Z
ST X
HLT
```

Машинные команды.

228 ... (переменная X)

229 ... (переменная Y)

230 ... (переменная Z)

231 A229 (LD Y)

232 4230 (ADD Z)

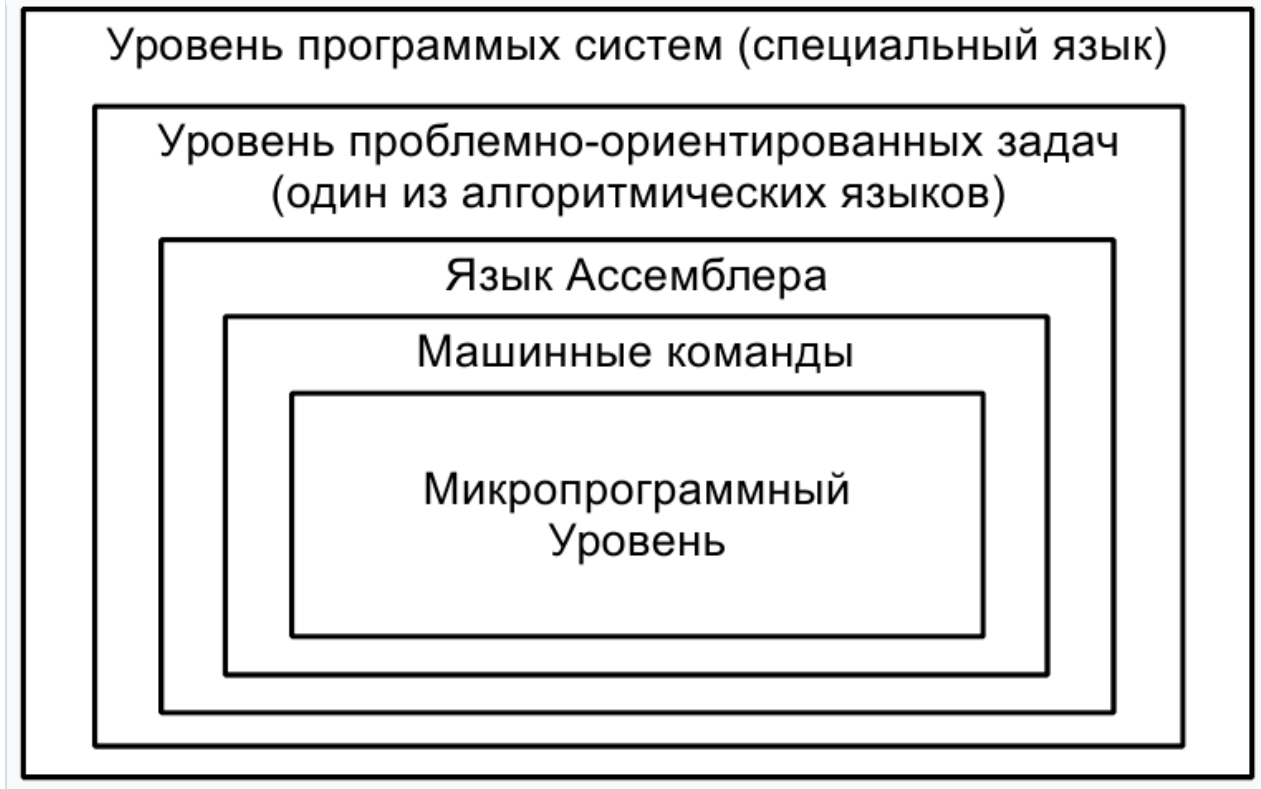
233 E228 (ST X)

234 0100 (HLT)

Микропрограммный уровень.

микрокоманда сложения:

32 0010E09011 ADD AC + DR ? AC, N, Z, V, C



20. Микропрограммный уровень БЭВМ. Структура МПУ. Форматы микрокоманд.

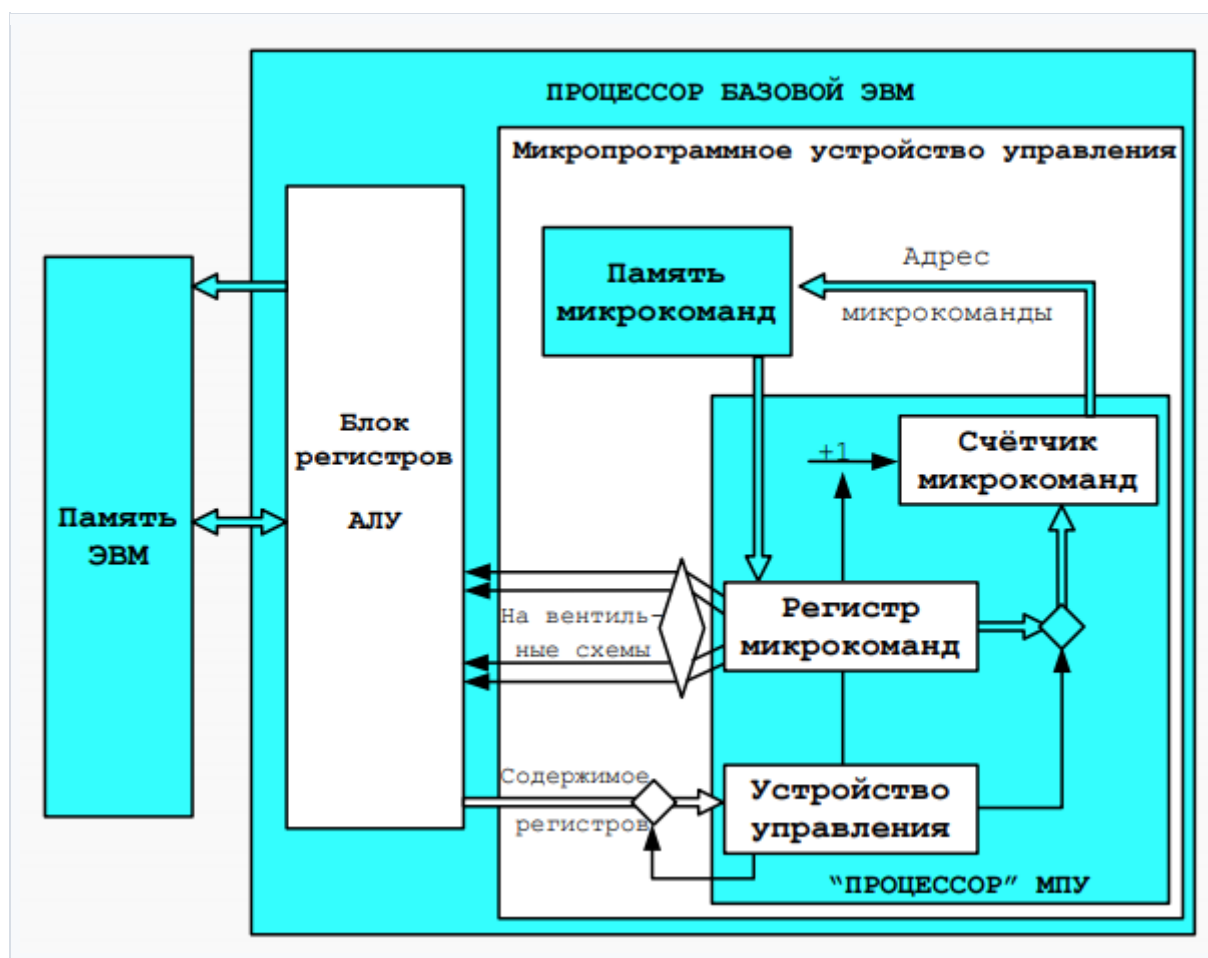
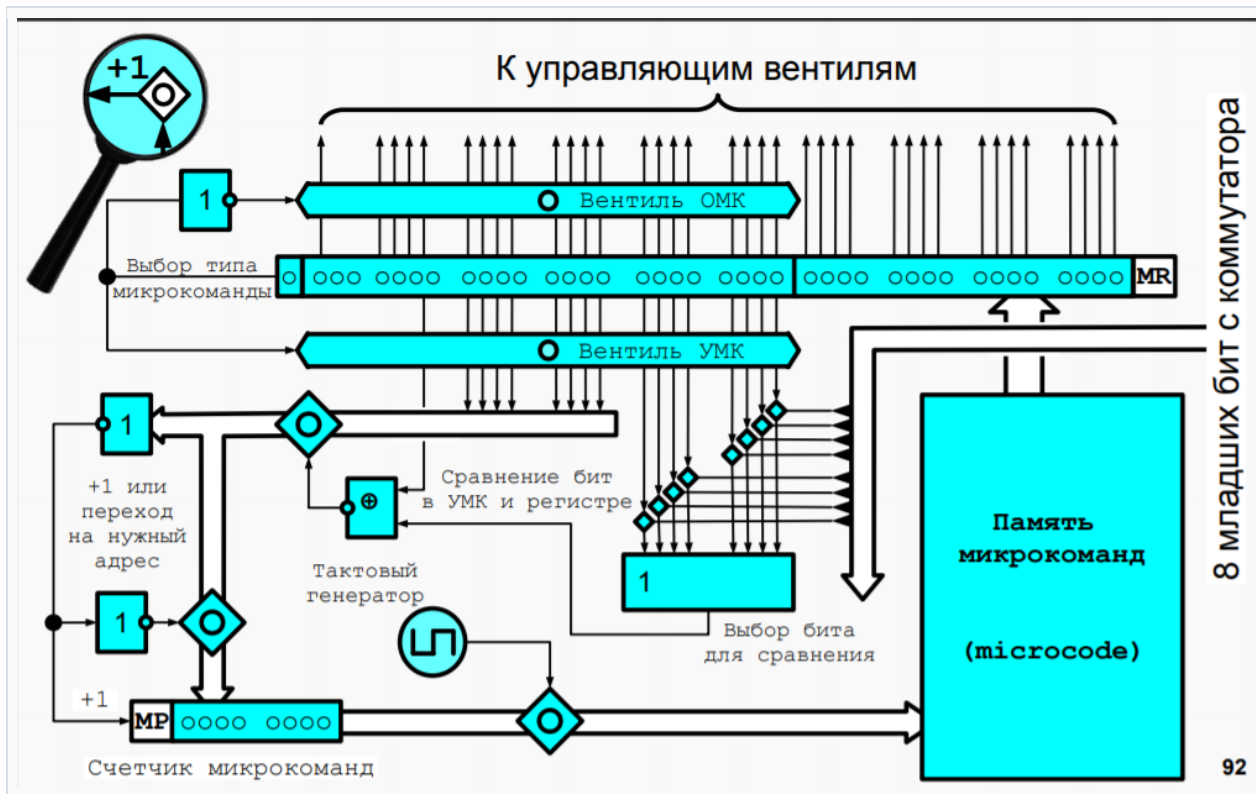
Если рассматривать БЭВМ как многоуровневую машину, то оказывается, что на её нижнем уровне выполняются элементарные действия (микрооперации) над словами информации. Управление порядком следования микроопераций осуществляется с помощью устройства управления БЭВМ, которое, в свою очередь, является очень простой ЭВМ. Для этой ЭВМ регистры и вентильные схемы базовой ЭВМ служат как бы устройствами ввода и вывода.

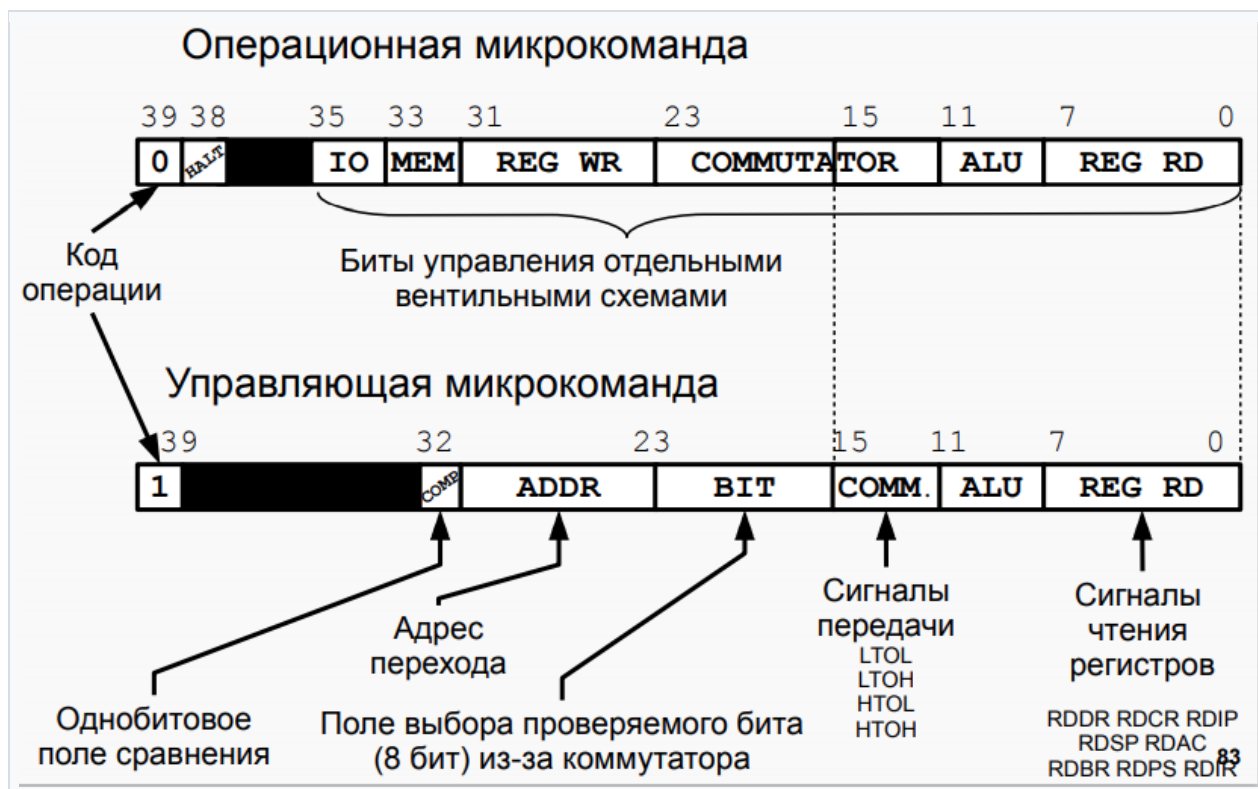
Программа работы такой ЭВМ – микропрограммного устройства управления (МПУ) – называется микропрограммой, а ее команды, содержащие информацию об элементарных действиях, подлежащих выполнению в течение одного рабочего такта ЭВМ, – микрокомандами. 29 Микропрограмма обычно хранится в постоянном запоминающем устройстве - памяти микрокоманд, но в эмуляторе БЭВМ реализована возможность изменения микропрограммы. В каждом такте работы ЭВМ из этой памяти в регистр микрокоманд (РМК) пересылается очередная микрокоманда, т.е. микрокоманда, на которую указывает счетчик микрокоманд (СЧМК), одновременно выполняющий функции регистра адреса микрокоманд.

МПУ Есть память микрокоманд, в счетчике +1, по его содержимому в памяти микрокоманд выбирается значение и поступает в регистр микрокоманд, в регистре выбирается тип команды и открывается определенный вентиль: 1) операционная все закодированные биты микрокоманды поступают на вентильные схемы 2) управляющая: часть в вентильные, значение регистра в устройство управления, там оно сравнивается с установленными в микрокоманде) если условие выполнено то загружается счетчик микрокоманд новым значением, если нет то добавляется единица. Общий принцип устройства упр микрокоманд. //Если проверяемый бит и бит из поля сравнения идентичны, то схема сравнения формирует единичный сигнал, который открывает вентильную схему ВА и на СЧМК пересылается адрес перехода (16-24-ый биты УМК). В противном случае на СЧМК передается нулевое значение, которое инициирует увеличение значения СЧМК на единицу (используется схема ИЛИ-НЕ для всех битов адреса перехода).

МР нужна для того: если команда не операционная то будут нули, и эта схема блокирует запись нулей в счетчик микрокоманд.

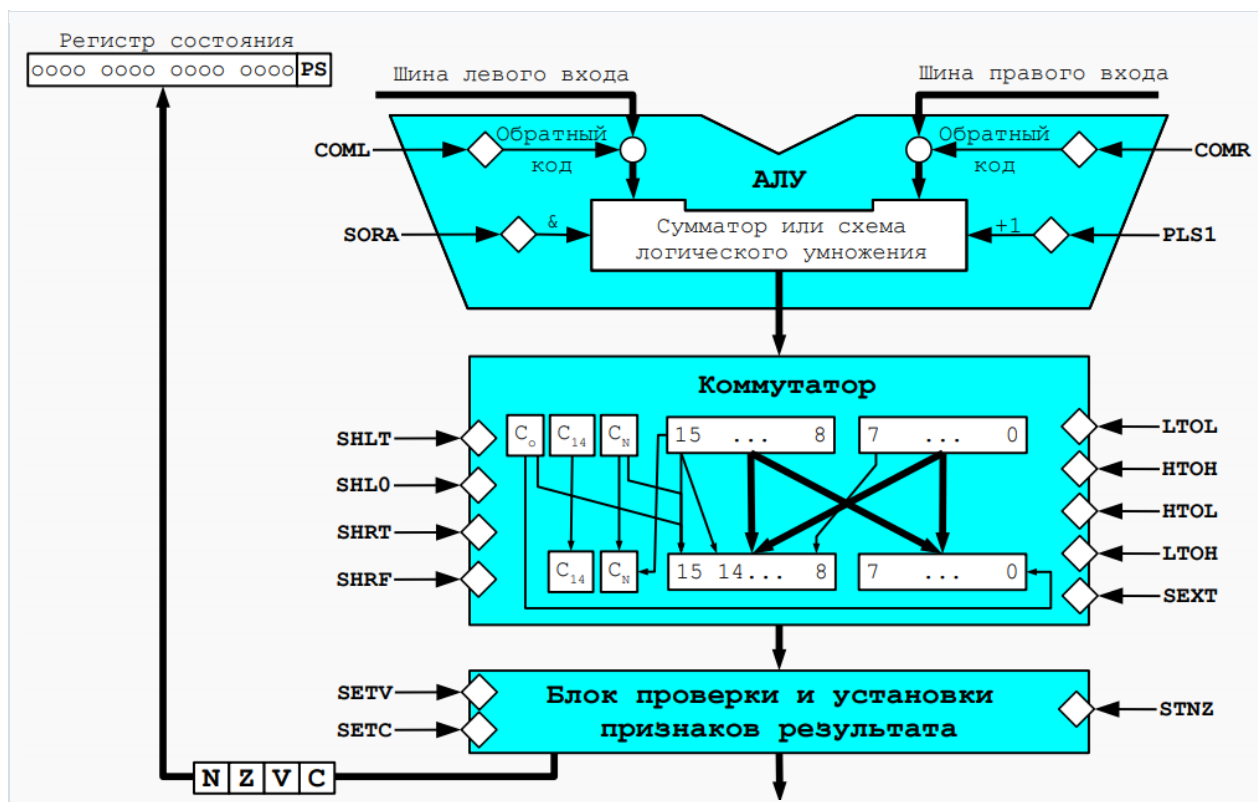
Microcode pointer используется для загрузки(выборки) микрокоманды из памяти и загрузки в регистр микрокоманд Вертикальная микрокоманда Так как в ЭВМ больших размеров могут быть сотни вентильных схем горизонтальная команда может быть неприемлемо длинной, поэтому используют кодируемые поля. Кодируемые поля, шифруют вентили, которые не могут быть открыты одновременно. В управляющей микрокоманде сокращение её разрядов можно обеспечить за счет кодирования полей выбора проверяемого регистра и проверяемого бита в этом регистре, что позволяет сократить УМК до 16 бит. Однако при декодировании таких сжатых полей требуется использовать дешифратор. Для декодирования операционных микрокоманд ОМК1 и ОМК2 потребуется ещё 8 дешифраторов. Но экономия затрат на память микрокоманд будет превышать стоимость этих дешифраторов.





21. Структура и принципы работы арифметико-логического устройства и коммутатора. Регистр состояния БЭВМ

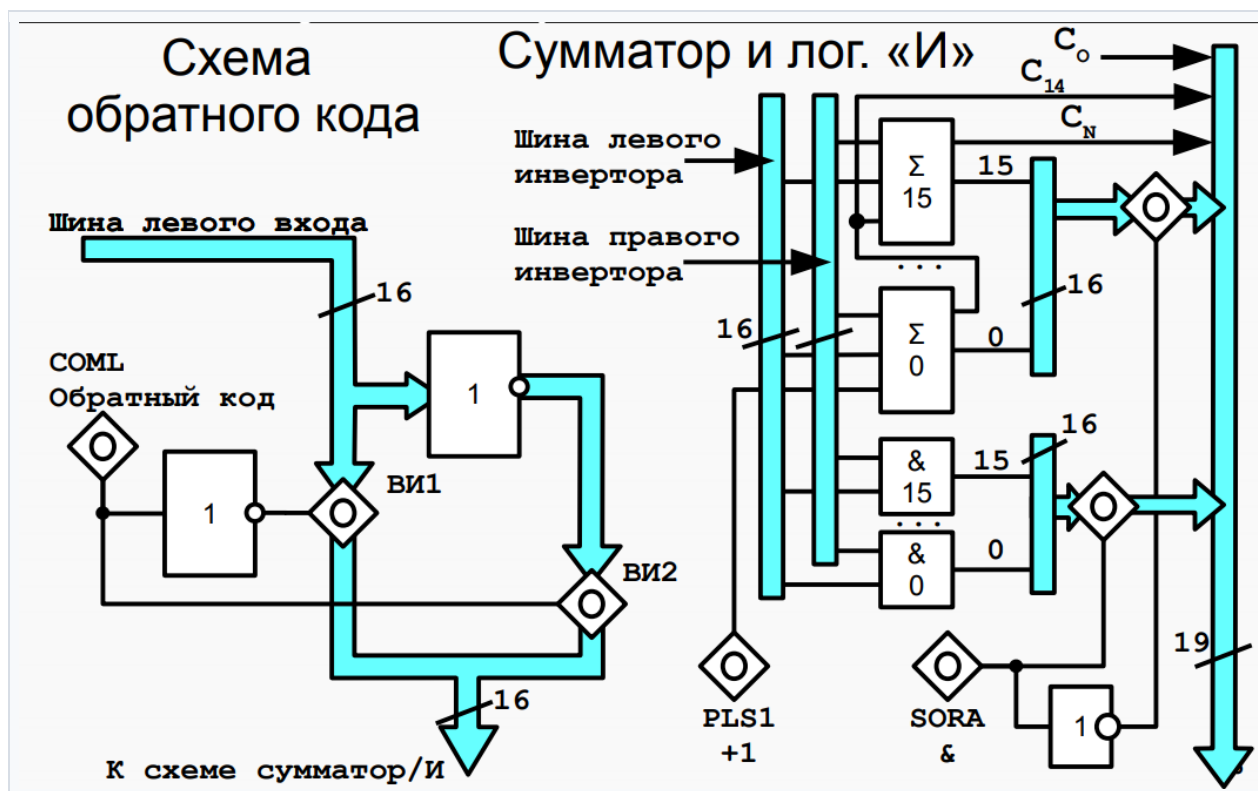
АЛУ использует А и РД в качестве операндов для получения результата, который помещается в аккумулятор. Все арифметические и логические команды БЭВМ и вспомогательные арифметические операции (например, увеличение на единицу содержимого СК) можно выполнить с помощью АЛУ.



Обратный код входных сигналов АЛУ получается по схеме, которая приведена слева снизу. В данной схеме изменение всех 0 на 1 и всех 1 на 0 осуществляется с помощью инверторов.

Сложение осуществляется типовой схемой сумматора, которая приведена снизу справа.

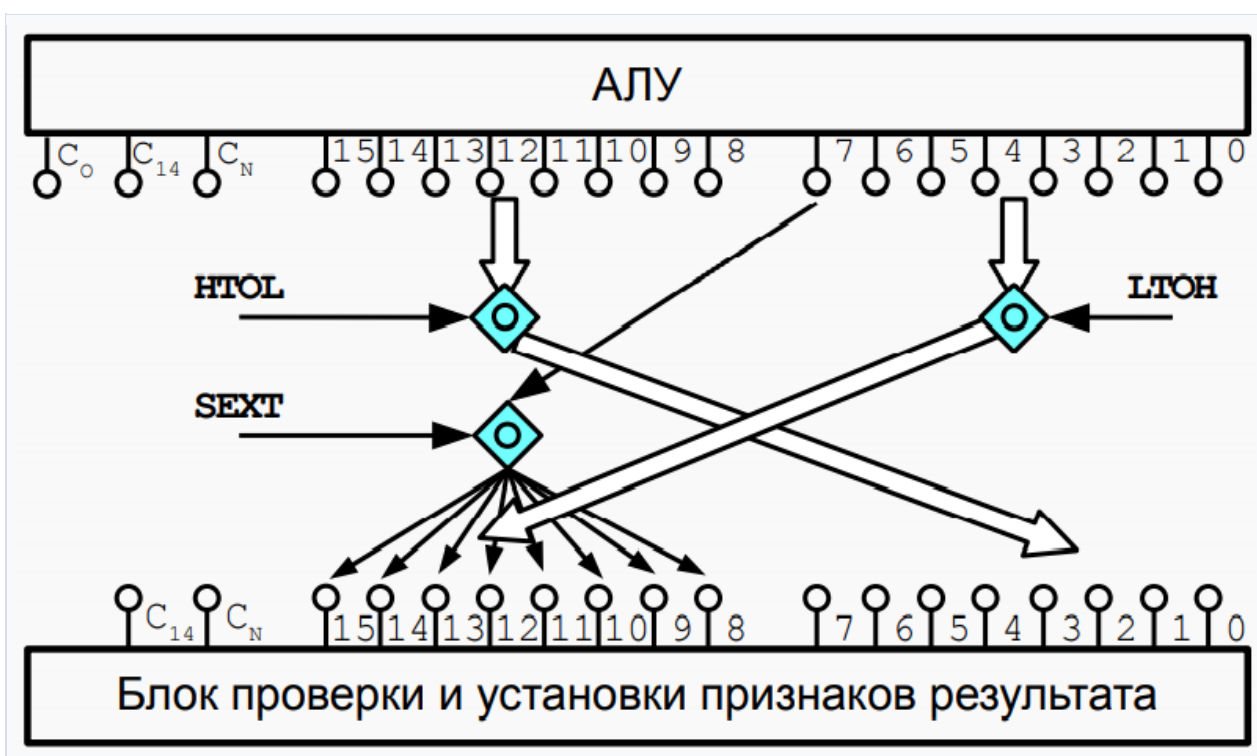
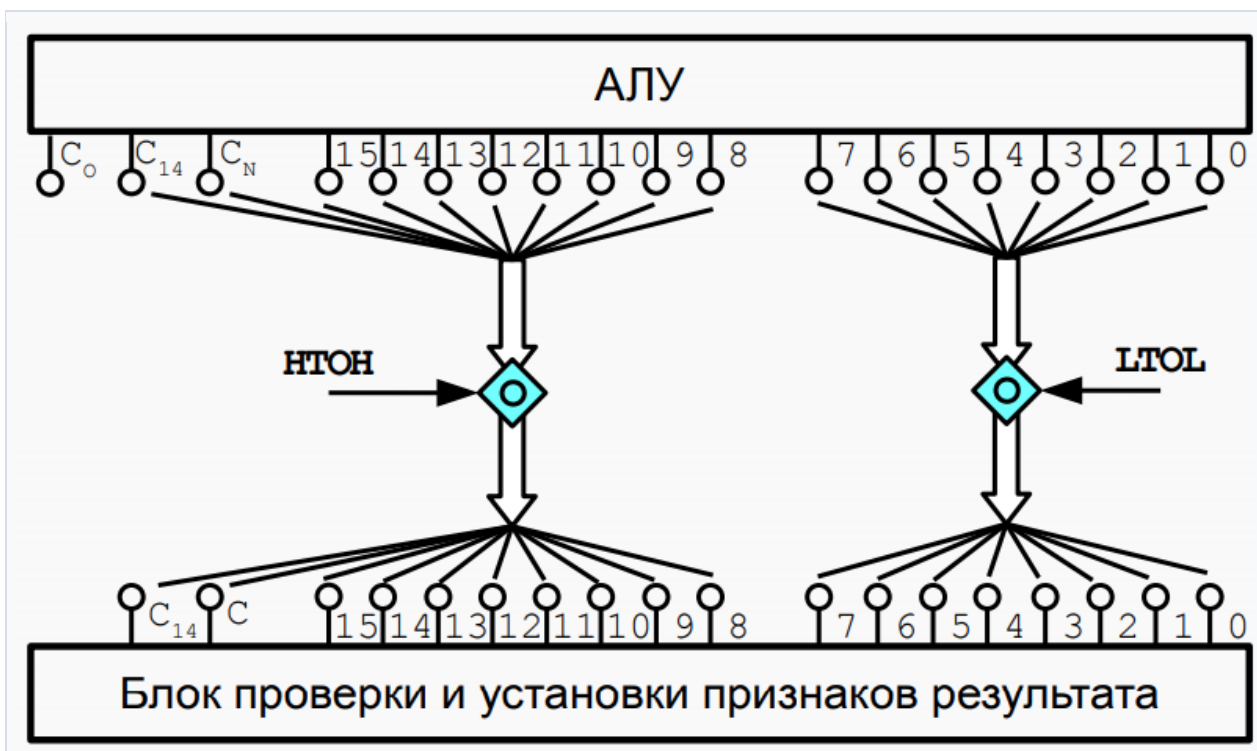
Логическое умножение осуществляется с помощью вентильной схемы, на входы которой подаются коды операндов, а на выходе образуется искомый результат.

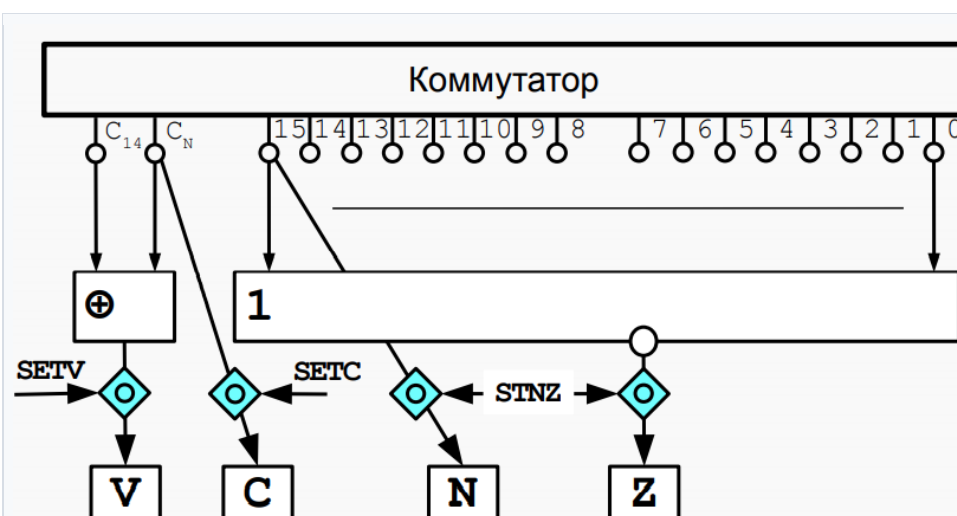
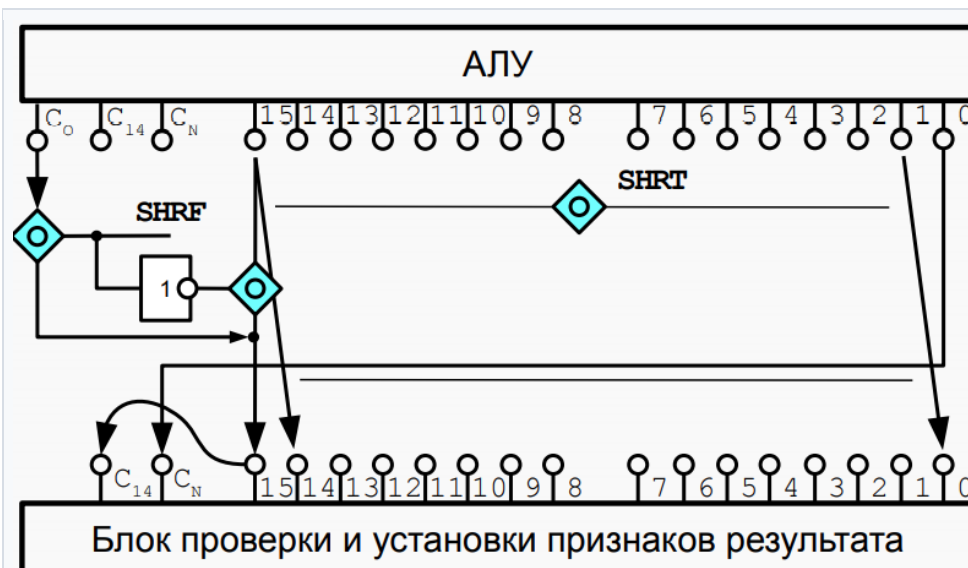
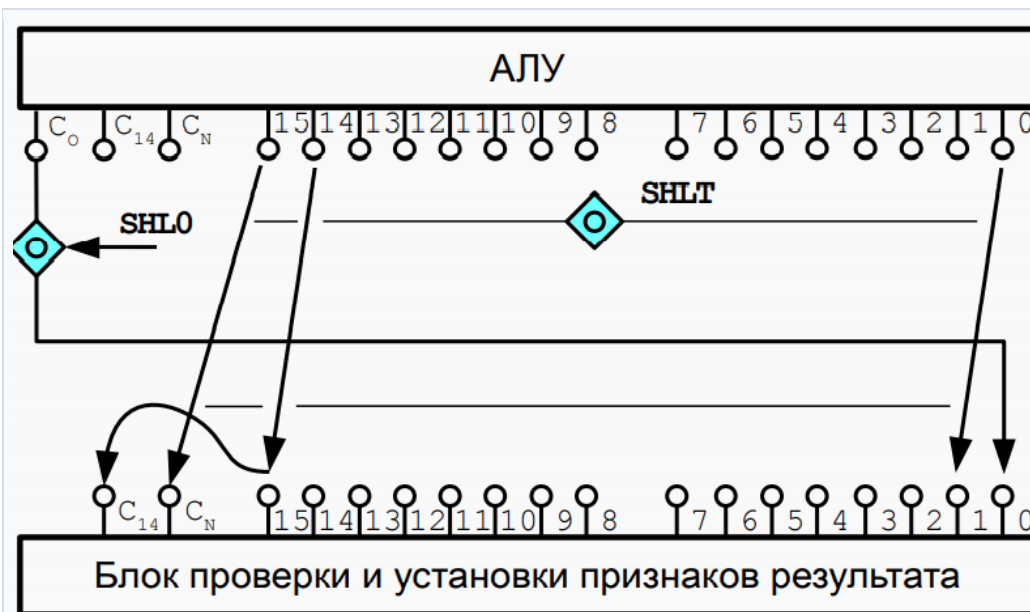


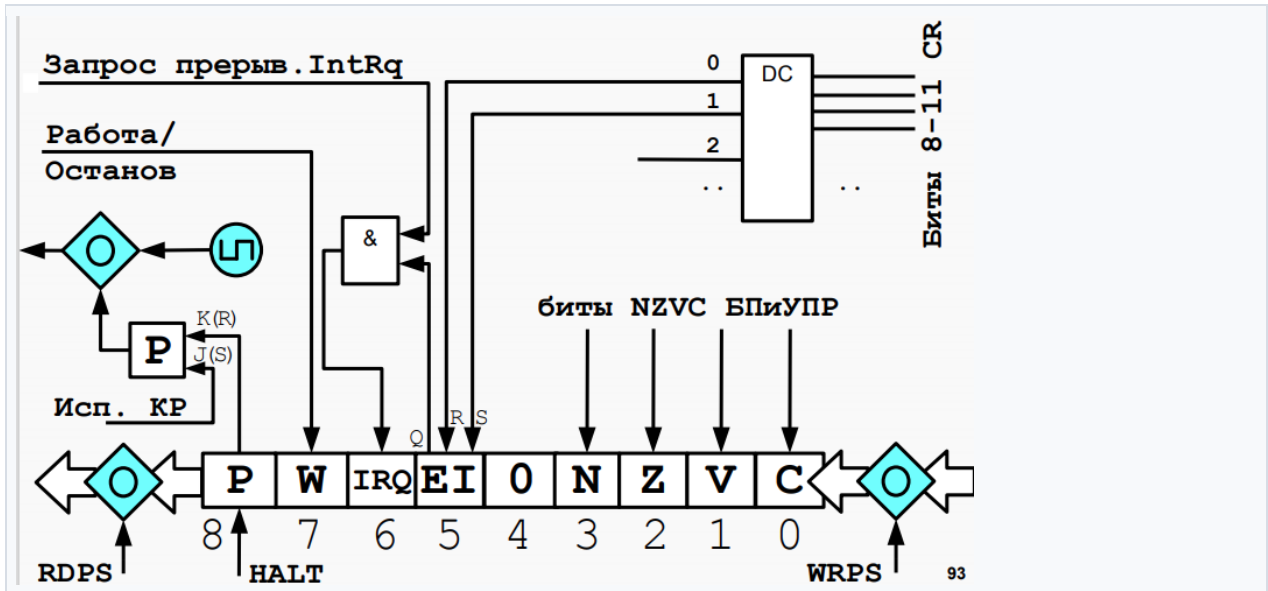
При организации ветвлений в микропрограмме используется содержимое РС, являющегося объединением однобитовых признаков результата и служебных битов состояния ЭВМ. Вентильные схемы – это электронные ключевые схемы, предназначенные для управления потоком информации из регистров в шины и обратно. На один вход подается информационный сигнал, на другой – управляющий.

Регистр состояния и команд(PS):

Бит	Мнем.	Содержимое
0	C	Перенос
1	V	Переполнение
2	Z	Ноль
3	N	Знак
4	O	0 – используется для организации безусловных переходов в МПУ
5	EI	Разрешение прерываний
6	IRQ	Требование прерывания (логическое “И” шины запроса на прерывание и бита 5 РС – “разрешение прерываний”)
7	W	Состояние тумблеров РАБОТА/ОСТАНОВ (1 – РАБОТА)
8	P	Программа

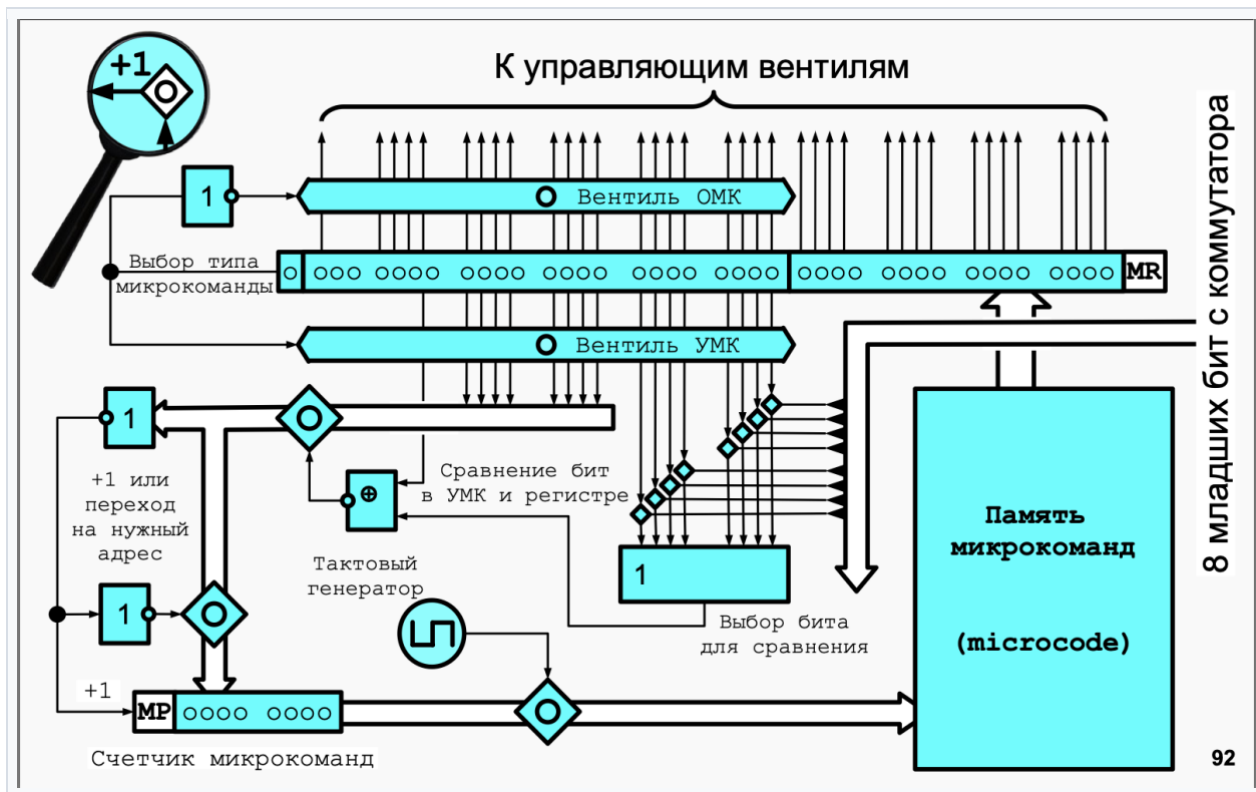






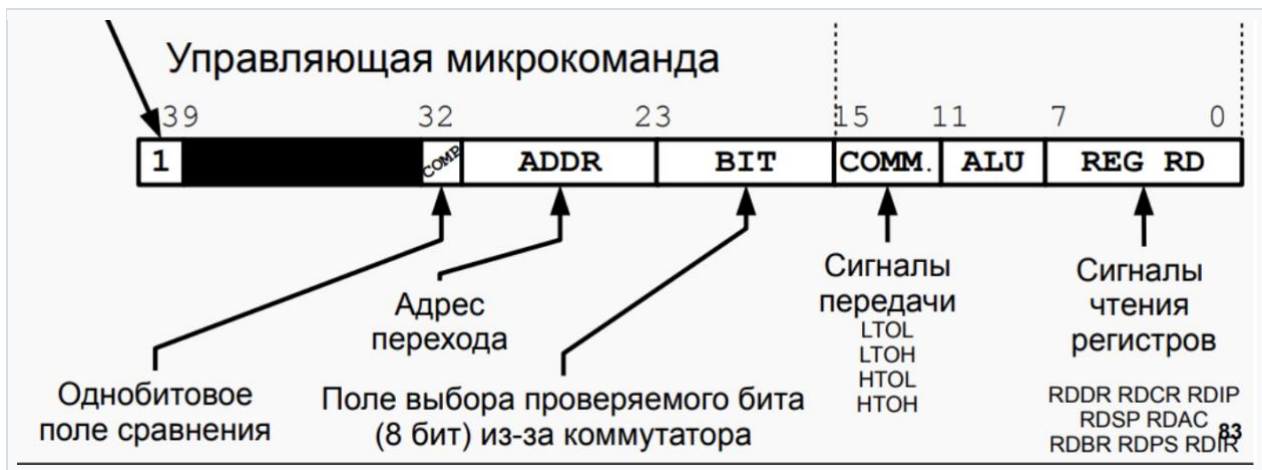
22. Микропрограммное управление вентильными схемами. Схема управления. Интерпретатор БЭВМ.

Для того, чтобы разобраться как происходит управление вентильными схемами, рассмотрим подробнее работу устройства управления.



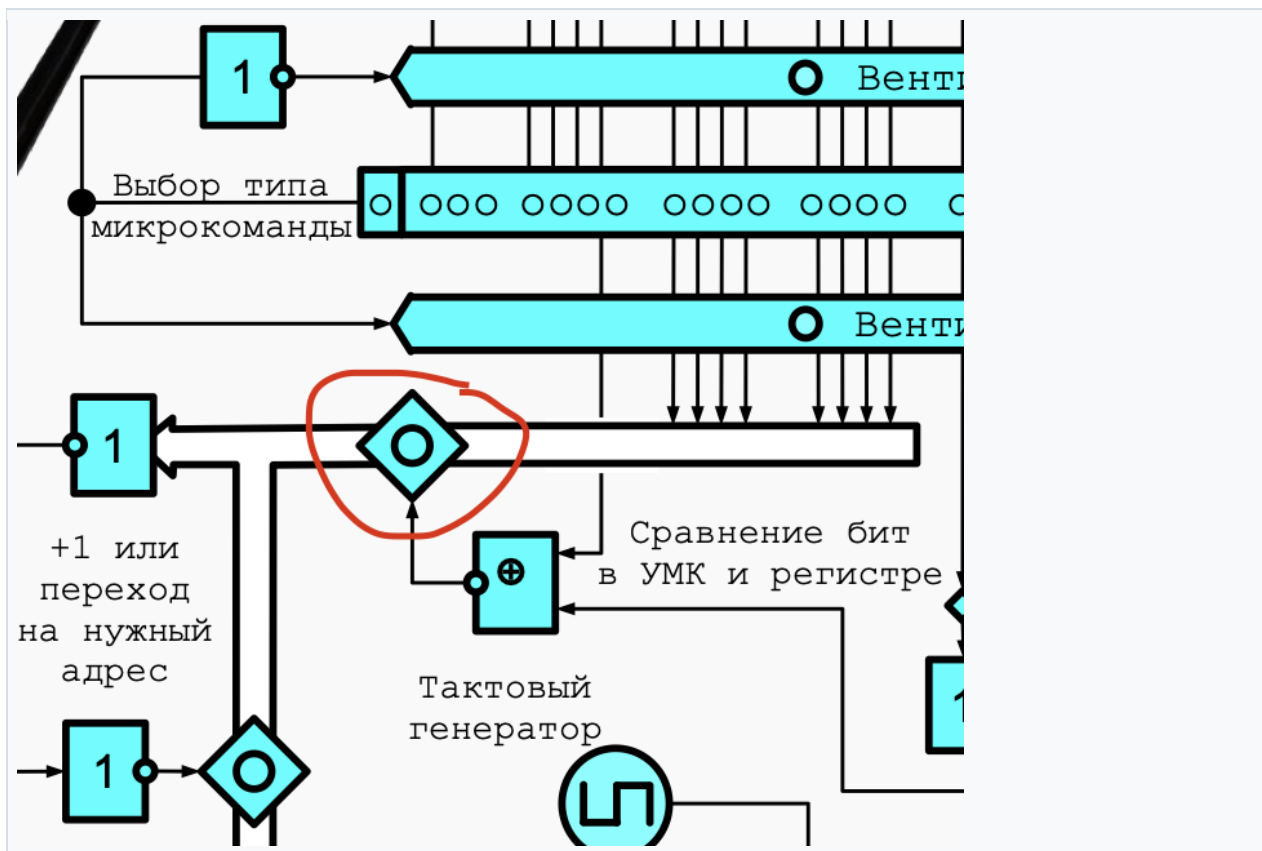
Содержимое памяти микрокоманд в каждом такте (внимание на тактовый генератор слева) передается в регистр микрокоманд (MR, 40 разрядный). Младшие 16 разрядов (ака одинаковая часть независимо от типа микрокоманды) сразу передаются на управляющие вентили. Внимание на бит 39. Это код операции микрокоманды. Если он НУЛЯМБА, то микрокоманда операционная. Сразу под лупой (не за лупой) вы можете видеть инвертор. Предположим, что наш 39 бит - 0. Тада он поступает на инвертор, на выходе инвертора получается единица. Открывается вентиль ОМК. Оставшиеся биты тоже сваливают на управляющие вентили. Все просто, все закончилось.

Предположим теперь, что код операции МК не нулямба, а единица. Откроется вентиль УМК. Среди тех бит, которые еще не ушли на вентильные схемы, рассмотрим младшие 8. Напомню, что это - поле выбора проверяемого бита.

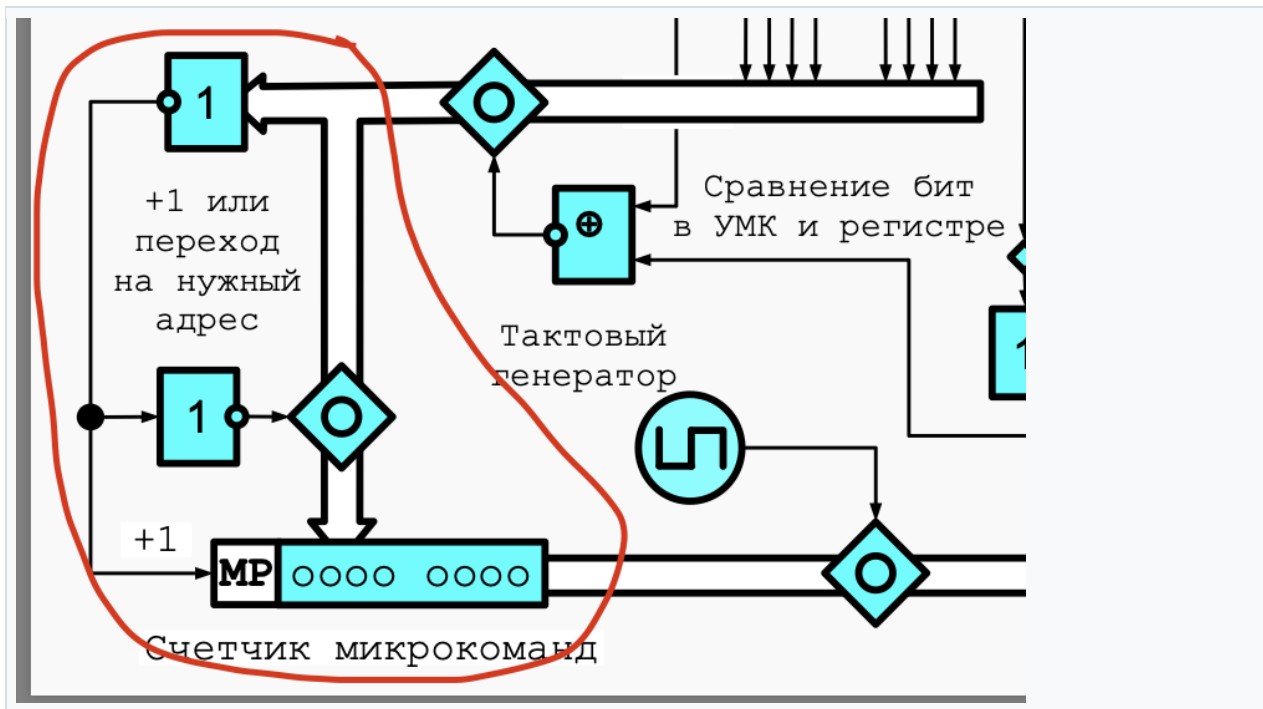


Видите белую стрелочку которая тупа с правого куска картинки появляется? По ней приходят 8 младших бит с коммутатора. Видите мелкие ромбики? Это 8 вентиляй. Если происходит совпадение одного из 8 битов МК и одного из 8 битов коммутатора, соответствующий ventиль открывается. Прямоугольник цвета циан (ну или тиффани) чуть ниже вентиляй - это одна большая схема ИЛИ. Результат идет на один из входов обратного ксора. На второй вход этого же самого обратного ксора идет бит номер 32. Посмотрев на картинку выше вспоминаем, что это "однобитовое поле сравнения".

Брат, ты помнишь что такое обратный ксор? Он дает единичку, если два его входа совпадают. Таким образом, если два входа совпали, то открывается ventиль. Какой? Этот.



Рассмотрим три случая. Случай первый: ventиль в самом деле открылся. Тогда адрес перехода попадает на вот эту схему:



Эта схемка нужна для того, чтобы понять, меняем ли мы МР, или просто прибавляем к нему АДЫН.

Адрес перехода не ноль? Не ноль. Смотрим в красную картошку. Верхний инвертор проверяет, не равен ли нулю наш адрес перехода. Если не равен, то открывается вентиль в красной картошке, адрес перехода записывается в MP. Ура!

Рассмотрим второй случай. Тот самый инвертированный ксор выдал на своем выходе ноль. Это значит, что адрес перехода не попал никуда дальше. Верхний инвертор нашей красной картошки получил на вход 0, выдал на выходе 1 и просто прибавил к МР единицу. Вентиль внутри картошки (если вы не догадались, так я называю красную “окружность” на картинке”) не откроется, МР не заполнится нулями. Ура!

Рассмотрим третий случай. Наша команда была вообще говоря операционная изначально, а не управляющая. И снова, аналогично предыдущему случаю, наша схема в красной картошке предотвращает запись нулей в MP, просто инкрементируя его вместо этого. Ура!

Мы разобрались как работает устройство управления. Идем дальше.

Чтобы понять, какой бит за что отвечает в твоей микрокоманде, достаточно лишь воспользоваться шпаргалкой вентильных схем.

Бит	Мнемоника	Назначение
Чтение регистров		
00	RDDR	DR (РД) → правый вход АЛУ
01	RDCR	CR (РК) → правый вход АЛУ
02	RDIP	IP (СК) → правый вход АЛУ
03	RDSP	SP (УС) → правый вход АЛУ
04	RDAC	AC (А) → левый вход АЛУ
05	RDBR	BR (БР) → левый вход АЛУ
06	RDPS	PS (РС) → левый вход АЛУ
07	RDIR	KR (КР) → левый вход АЛУ
АЛУ		
08	COMR	Обратный код правого входа АЛУ
09	COML	Обратный код левого входа АЛУ
10	PLS1	Операция $A + B + 1$ (PLuS 1)
11	SORA	Операция И (Sum OR And)

94



ИТМО BT

Шпаргалка. Вентильные схемы

Бит	Мнемоника	Назначение
Управление коммутатором		
12	LTOL	Прямая передача младшего байта
13	LTON	Передача младшего байта в старший
14	HTOL	Передача старшего байта в младший
15	HTON	Прямая передача старшего байта
16	SEXT	Расширение знака младшего байта (sign extend)
17	SHLT	Сдвиг влево (арифметический) (SHift Left)
18	SHL0	Передача старого значения С в младший бит (для циклического сдвига влево)
19	SHRT	Сдвиг вправо (арифметический) (SHift Right)
20	SHRF	Переключатель сдвига вправо (для циклического сдвига 15 разряд)
21	SETC	Установка С
22	SETV	Установка V
23	STNZ	Установка N и Z

95

Бит	Мнемоника	Назначение
Чтение регистров		
24	WRDR	АЛУ → DR (РД)
25	WRCR	АЛУ → CR (РК)
26	WRIP	АЛУ → IP (СК)
27	WRSP	АЛУ → SP (УС)
28	WRAC	АЛУ → AC (А)
29	WRBR	АЛУ → BR (БР)
30	WRPS	АЛУ → PS (РС)
31	WRAR	АЛУ → AR (РА)
Работа с памятью		
32	LOAD	Ячейка памяти → DR (РД)
33	STOR	DR (РД) → Ячейка памяти

96

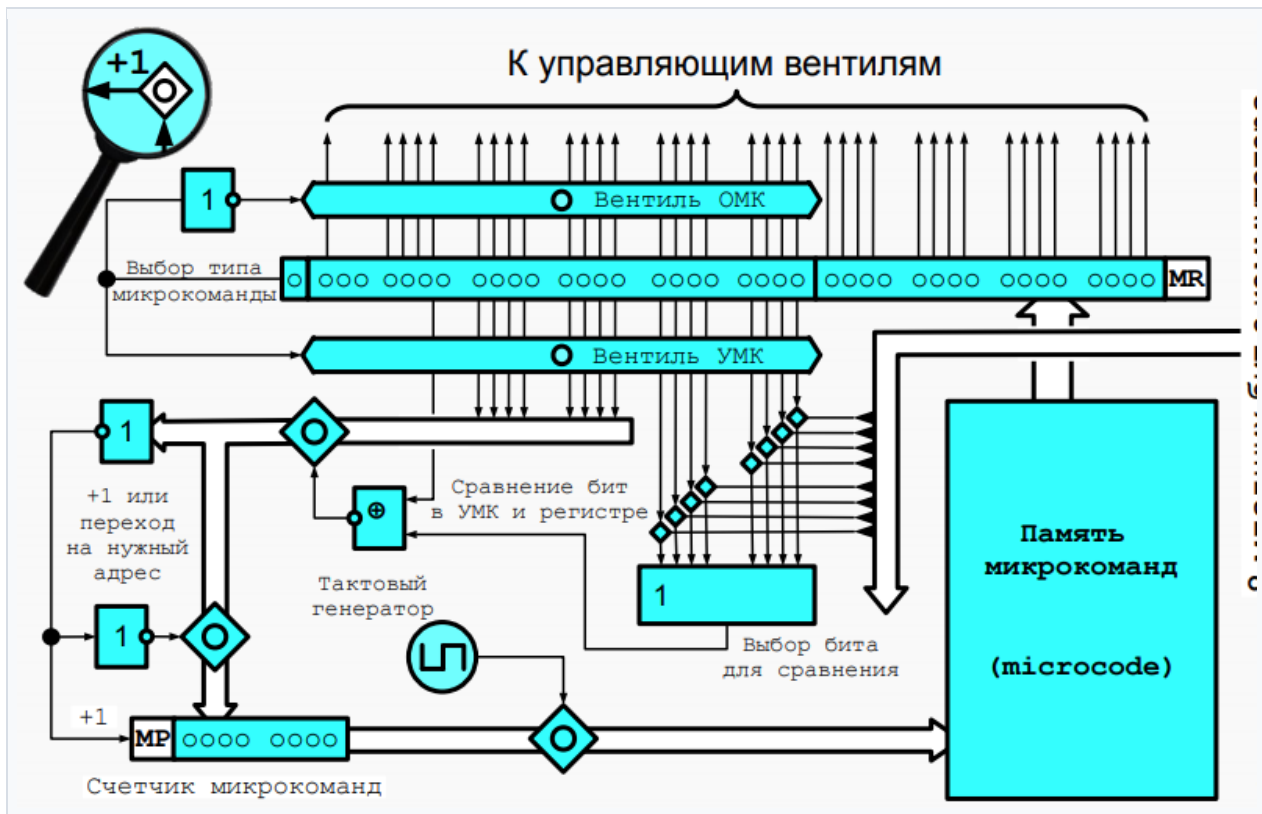
Бит	Мнемоника	Назначение
Организация ввода-вывода		
34	IO	Передача адреса и приказа на ВУ
35	IRQS	Предоставление прерывания
Резервные сигналы		
36		Зарезервирован
37		Зарезервирован
Останов БЭВМ		
38	HALT	Останов
Работа с памятью		
39	TYPE	Бит выбора ОМК/УМК

интерпретаор состоит из этого:

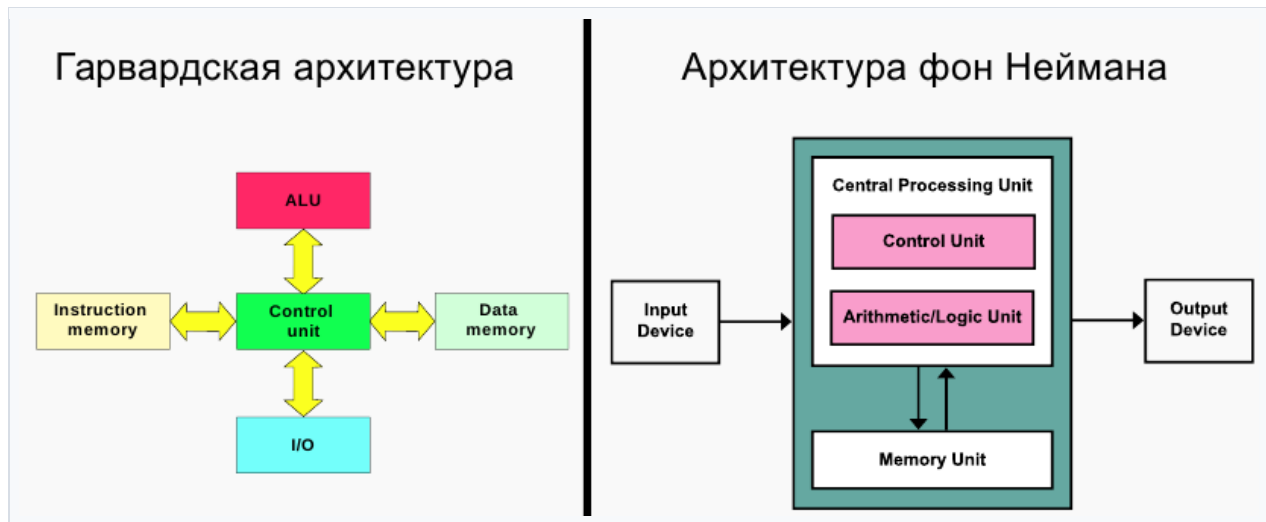
- Цикл выборки команд
- Цикл выборки адреса операнда и обработки режимов адресации
- Цикл выборки операнда
- Цикл исполнения
 - Декодирование и исполнение адресных команд
 - Декодирование и исполнение ветвлений
 - Декодирование и исполнение безадресных команд
- Декодирование и исполнение команд ввода-вывода
- Цикл прерывания
- Пультовые операции
- Свободные ячейки для:
 - Арифметической команды
 - Команды перехода
 - Безадресной команды

99

В интерпретаторе есть 256 ячеек для хранения микрокоманд. Причем горизонтальных.



23. Архитектура ЭВМ. Гарвардская и фон-Неймановская архитектура. Организация обмена архитектуры ЭВМ с использованием шин.



Архитектура фон Неймана основана на трех основных элементах: процессоре, памяти и устройствах ввода/вывода. В компьютере, спроектированном таким образом, и программа, и данные хранятся в одной памяти, к которой подключен процессор (состоящий из системы управления и арифметико-логического устройства). Он **последовательно** извлекает инструкции из памяти и выполняет соответствующие им команды, реализуя тем самым сохраненную программу. Для связи с внешним миром используются устройства ввода/вывода. В случае связи с пользователем такими устройствами будут являться монитор и клавиатура, но во многих случаях (напр., в интегральных схемах) это в первую очередь порты ввода/вывода, последовательные шины и т.п. Архитектура фон Неймана широко применяется в большинстве современных компьютеров. Своей популярностью она обязана, прежде всего, простоте и универсальности. Однако необходимо помнить, что данная модель не лишена недостатков: самым главным из них является ограниченная скорость работы: **единый путь к памяти создаёт конкуренцию между выборкой инструкций и доступом к данным; при росте частот и усложнении программ именно пропускная способность памяти/шины становится бутылочным горлышком.**

В гарвардской архитектуре память разделена на память команд и память данных, каждая со своей шиной. Такая структура повышает скорость работы всей системы. В архитектуре фон Неймана программа выполняется последовательно, а это означает, что процессор, чтобы загрузить следующую команду или данные, всегда должен перейти к следующей ячейке памяти. В гарвардской системе эти процессы выполняются параллельно (точнее независимо), т.е. процессор может загружать команды и данные, используя отдельные адресные пространства. Особенности гарвардской архитектуры позволяют часто везде, где требуется **параллельная обработка** инструкций и данных, например, в приложениях реального времени или контроллерах.

Гибрид используется в современных компьютерах. На верхнем уровне – гарвардская архитектура – L1 кэш разделен на L1I (инструкции) и L1D (данные) → параллельный доступ, чтобы повысить скорость. На нижнем уровне L2, L3 кэш, ОЗУ – Фон Нейман – общая память, где хранятся и команды, и данные, чтобы сохранить универсальность. CPU одновременно загружает инструкции из L1I и данные из L1D, повышая скорость.

Шинная архитектура

Появление ЭВМ третьего поколения связано с переходом от транзисторов к интегральным микросхемам, что позволило значительно повысить быстродействие компьютеров. Однако возникло несоответствие между скоростью процессора и медленной работой устройств ввода-вывода. Чтобы устранить эту проблему, функции обмена данными стали передавать специальным схемам — контроллерам внешних устройств. Контроллеры выполняли операции с периферией самостоятельно (например, управление дисководом) и имели собственные команды. Благодаря этому центральный процессор освобождался от рутинных задач и мог выполнять другие операции. Центральный процессор, при необходимости произвести обмен, выдает задание на его осуществление контроллеру. Дальнейший обмен информацией может протекать под руководством контроллера без участия центрального процессора. Последний получает возможность "заниматься своим делом", т.е. выполнять программу дальше (если по данной задаче до завершения обмена ничего сделать нельзя, то можно в это время решать другую...).

Для связи между отдельными функциональными узлами ЭВМ используется общая шина. Логически выделяют три «шины» сигналов: адресную (куда обратиться), данных (что передать) и управления (чтение/запись, подтверждения, ошибки, блокировки). С точки зрения логики шина — это набор параллельных линий адреса, данных и управления; с точки зрения организации системы — это хребет, по которому идут все основные потоки: запросы процессора к памяти, DMA-передачи периферии, когерентные сообщения кэшей и т. п.

Классификация шин по назначению

Тип шины	Что передаёт	Кто использует	Ключевые сигналы
Шина адреса	Адрес ячейки памяти или устройства	CPU → память, устройства	Только в одном направлении
Шина данных	Сами данные	CPU ↔ память, устройства	Двухнаправленная
Шина управления	Сигналы управления и синхронизации	CPU ↔ память, устройства	Сигналы: RD, WR, INT, RESET и др.

По уровню системы

Уровень	Назначение	Примеры
Системная шина	Соединяет CPU и контроллер памяти	Устарела, заменена на шины QPI, Infinity Fabric, UPI
Шина памяти	Связывает процессор и ОЗУ	DDR4, DDR5, LPDDR5
Шина ввода-вывода	Соединяет периферийные устройства с системой	PCI Express, USB, SATA
Внутренняя шина	Соединяет блоки внутри процессора	Внутренние межъядерные шины, Ring Bus, Mesh Interconnect

По организации обмена

Тип	Описание	Пример
Параллельная шина	Несколько бит передаются одновременно по множеству линий	FSB, старые PCI
Последовательная шина	Биты идут один за другим по одной линии, но очень быстро	PCIe, USB, SATA
Мультиплексированная	Линии попеременно передают адреса и данные	Современные системные шины

- Первое поколение — электронные лампы

Основные элементы — вакуумные лампы (диоды и триоды). Использовались электромеханические устройства ввода-вывода (перфокарты, ленты). Память — на ртутных линиях задержки, магнитных барабанах, позже — на ферритовых сердечниках. Программы вводились вручную, на машинном языке

Лебедев, 1950, МЭСМ – БЭСМ, 1953, БЭСМ – 10000 оп/с, 53КБТ.

- Второе поколение — транзисторы

Лампы заменили транзисторы — они меньше, надёжнее и потребляют меньше энергии. Использовались магнитные сердечники для ОЗУ. Появились операционные системы, компиляторы, языки высокого уровня (Фортран, Алгол, Кобол). Появились первые пакетные режимы обработки данных и векторные конвейеры.

5Э926, 1964, самодиагностика, горячая замена, 500000 оп/с – БЭСМ-6, 1965 год, +ковейерная обработка, удаленное управление по телеф. Линиями

- Третье поколение — интегральные схемы

На одной микросхеме — десятки и сотни транзисторов. Архитектура стала модульной, с контроллерами ввода-вывода и общей системной шиной. Появились многопользовательские операционные системы и виртуальная память.

Директива «Ряд», 1968 год, клонирование S/360, 1971 год – ЕС ЭВМ – Клоны PDP-11

- Четвертое поколение — сверхбольшие интегральные схемы

Микропроцессор — весь ЦП на одном кристалле! Память и периферия также на микросхемах.

Эльбрус — разработка по настоящее время

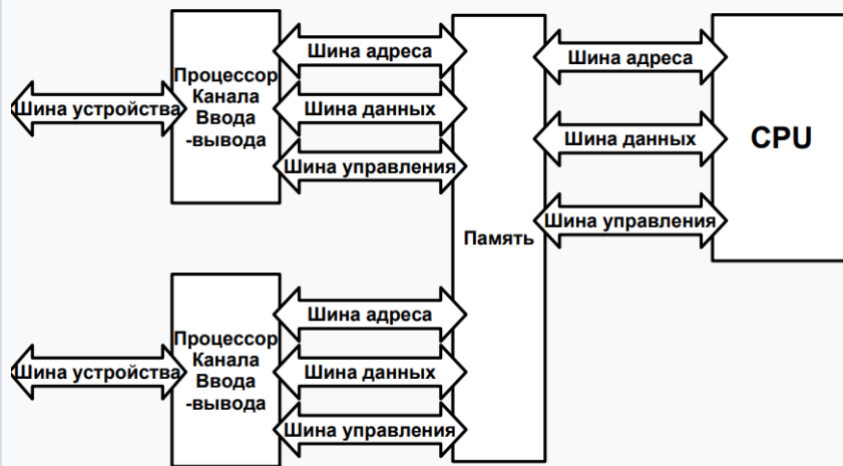


Рисунок 1 Канальная организация

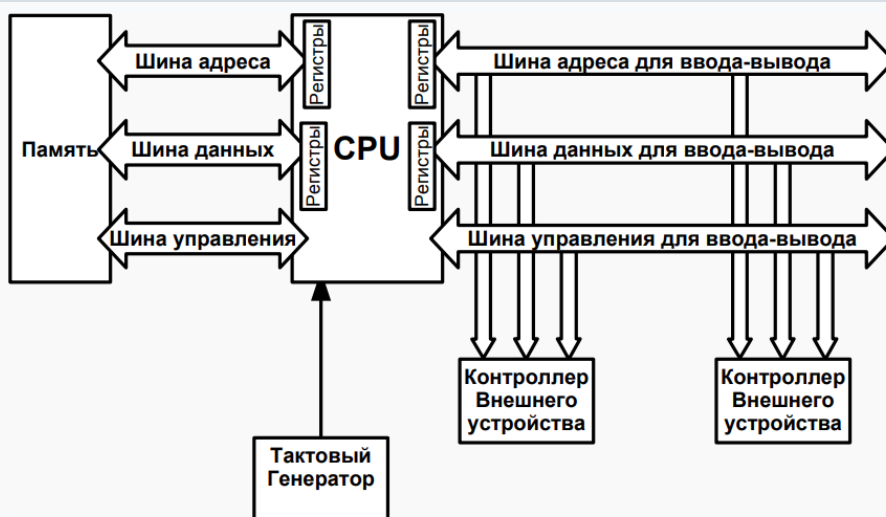


Рисунок 2 Раздельные шины

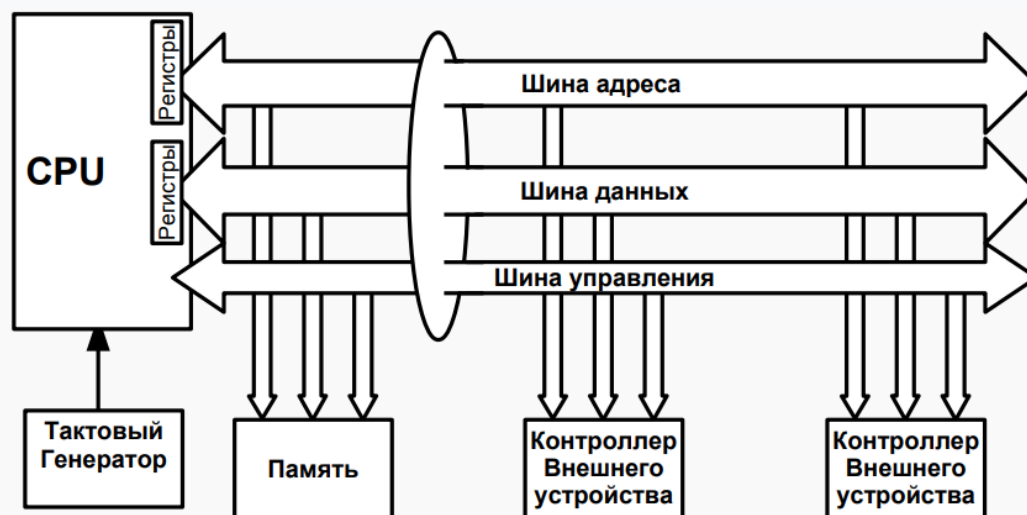
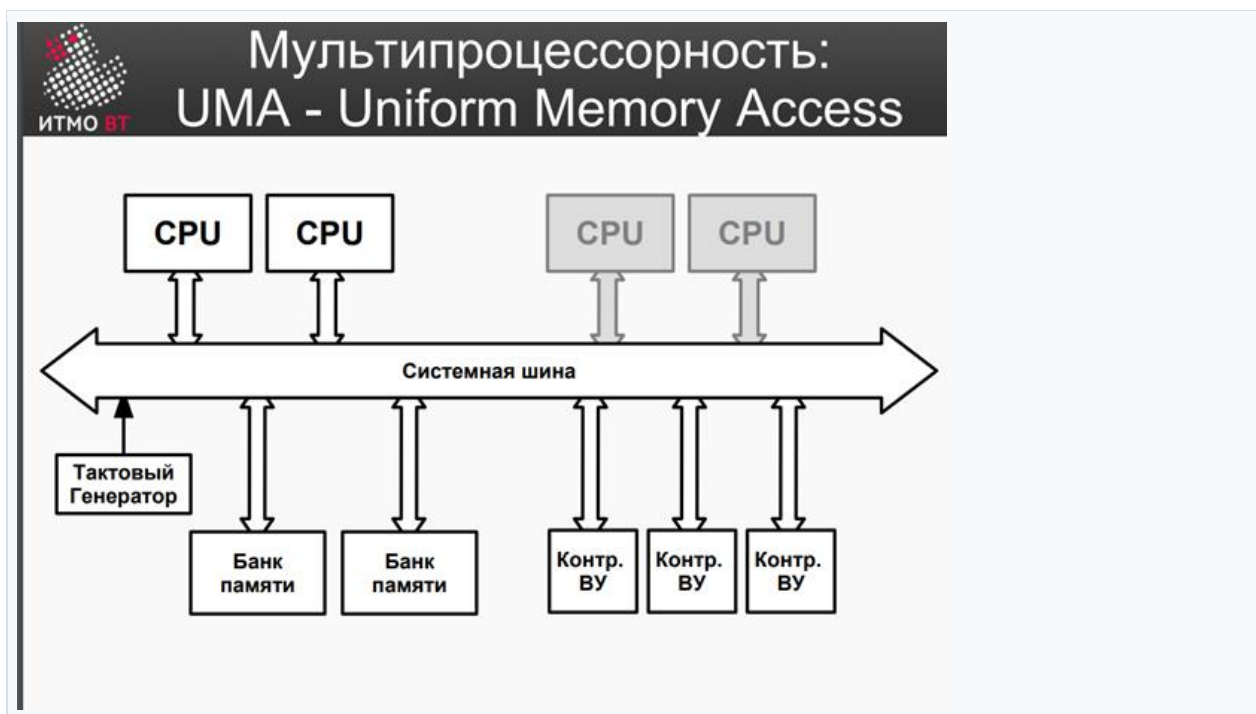


Рисунок 3 Общие шины



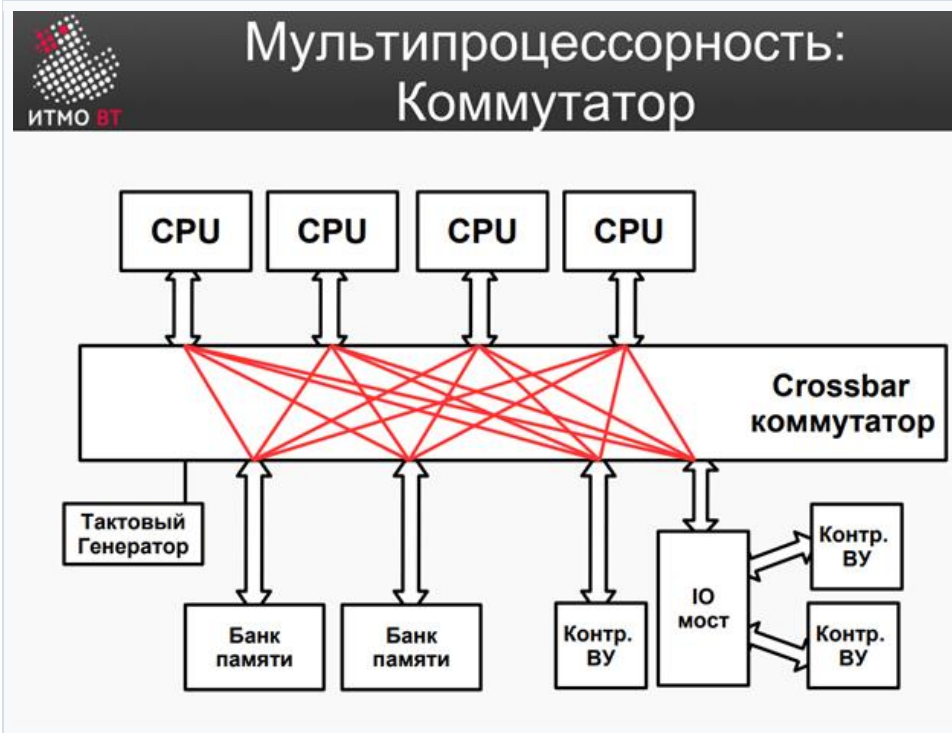
24. Архитектура многопроцессорных ЭВМ. Системный коммутатор. Архитектуры UMA и NUMA.



Зачем нужны многопроцессорные ЭВМ. Один процессор имеет предел производительности: дальше наращивать тактовую частоту мешают тепло и физика. Чтобы считать быстрее, ставят несколько процессоров, которые работают над задачей одновременно. Если задачу можно разбить на части, выполняемые параллельно, общая скорость растёт почти пропорционально числу процессоров. Но чтобы процессоры совместно решали задачу, им нужно обмениваться данными — а данные лежат в **общей памяти**. Именно организация доступа нескольких процессоров к общей памяти и есть главная инженерная проблема таких машин.

Проблема общей памяти. Память физически одна, а желающих обратиться к ней — много. Если двое процессоров одновременно захотят прочитать или записать память, кто-то должен ждать. Чем больше процессоров, тем чаще они мешают друг другу, и память превращается в «бутылочное горлышко»: процессоры простаивают в очереди вместо того, чтобы считать. Вся борьба в архитектуре многопроцессорных систем — это борьба за то, чтобы как можно больше обращений к памяти шло **одновременно и без помех**.

Общая шина и её предел. Самое простое решение — посадить все процессоры и всю память на одну **общую шину** (общий набор проводов). Это дёшево и просто, но шиной в каждый момент может пользоваться только один: пока один процессор читает память, остальные ждут своей очереди. Это как единственная узкая дорога к складу: пара машин проедет, но десяток уже встанет в пробку. Поэтому общая шина годится лишь для небольшого числа процессоров (единицы), а дальше не масштабируется.

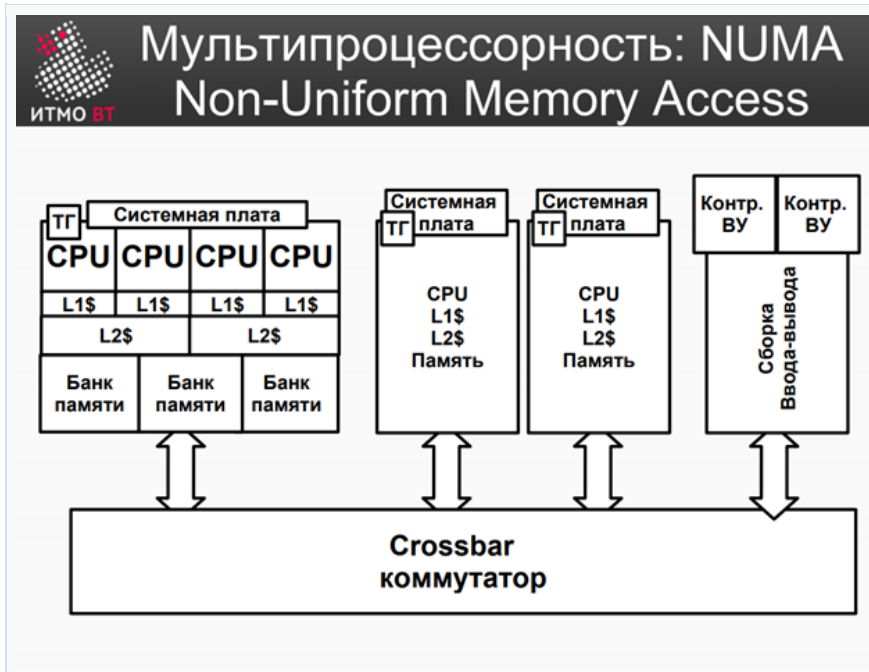


Системный коммутатор. Чтобы убрать это горлышко, вместо одной общей шины ставят **системный коммутатор (switch)** — устройство, которое динамически соединяет процессоры с модулями памяти, как телефонная станция соединяет абонентов. Память при этом делят на несколько отдельных модулей (банков). Коммутатор позволяет нескольким парам «процессор — модуль памяти» работать **одновременно**: пока процессор А обращается к модулю 1, процессор В в это же время обращается к модулю 2, и они друг другу не мешают.

Простейший вид коммутатора — **координатный (crossbar)**: представьте решётку, где по горизонтали идут линии процессоров, по вертикали — линии модулей памяти, а в каждом пересечении стоит ключ, который при необходимости замыкает связь. Он обеспечивает **максимум одновременных соединений**, но число ключей растёт как произведение числа процессоров на число модулей, поэтому для больших систем становится слишком громоздким и дорогим. Компромисс — **многоступенчатые коммутаторы**, где соединение собирается из нескольких более мелких переключателей: их дешевле строить, но путь через несколько ступеней даёт чуть большую задержку, и иногда два соединения всё же конфликтуют. Главное: коммутатор масштабируется гораздо лучше шины, ценой усложнения и удорожания.

UMA — однородный доступ к памяти. В архитектуре **UMA (Uniform Memory Access)** время обращения к любой ячейке памяти одинаково для всех процессоров — память для всех «одинаково далеко». Так устроены **симметричные мультипроцессоры (SMP)**: несколько одинаковых процессоров и общая память, к которой все обращаются на равных правах через шину или коммутатор. Достоинство — простота для программиста: всё равно, где лежат данные, доступ к ним стоит одинаково, поэтому программу не нужно подстраивать под расположение памяти. Недостаток — плохая масштабируемость: чем больше процессоров, тем сильнее они конкурируют за общую память, и эффективность падает. UMA хороша при небольшом числе процессоров.

NUMA — неоднородный доступ к памяти. В архитектуре **NUMA (Non-Uniform Memory Access)** память физически **распределена** по узлам: у каждого процессора (или группы процессоров) есть своя **локальная память** рядом и **удалённая память** соседних узлов. Обращение к локальной памяти — быстрое; обращение к удалённой идёт через межузловое соединение и заметно медленнее. Отсюда и название: время доступа **неодинаково** и зависит от того, в чьей памяти лежат данные. Аналогия: у каждого работника есть ближний личный шкаф (быстро) и общий дальний архив через коридор (медленно).



Смысл NUMA в том, что она масштабируется на **большое число процессоров**: каждый в основном работает со своей быстрой локальной памятью, и общая нагрузка не упирается в один модуль. Но за это приходится платить вниманием к **локальности данных**: программу и операционную систему нужно строить так, чтобы процессор как можно чаще обращался к своей локальной памяти и редко — к удалённой. Если данные разложены неудачно и процессор постоянно лезет в чужую память, выигрыш теряется и производительность падает.

Когерентность кэшей. В обеих архитектурах у каждого процессора есть свой **кэш** — быстрая копия части данных из памяти. Возникает опасность: два процессора держат в своих кэшах копии одной и той же ячейки, один её изменил — и у второго копия устарела. Чтобы такого не было, нужна **поддержка когерентности (согласованности) кэш-памяти**: специальные механизмы следят, чтобы все процессоры видели одно и то же значение каждой ячейки (устаревшие копии помечаются недействительными или обновляются). NUMA-систему с такой поддержкой называют **cc-NUMA** (cache-coherent NUMA); без когерентности совместная работа над общими данными была бы ненадёжной.

Ключевые различия между UMA и NUMA

- Модель UMA (совместно используемая память) использует один или два контроллера памяти. В отличие от этого, NUMA может иметь несколько контроллеров памяти для доступа к памяти.

- В архитектуре UMA используются одиночные, множественные и перекрестные шины. И наоборот, NUMA использует иерархические и древовидные типы шин и сетевых подключений.
- в UMA время доступа к памяти для каждого процессора одинаково, в то время как в NUMA время доступа к памяти изменяется по мере изменения расстояния памяти от процессора.
- Приложения общего назначения и разделения времени подходят для машин UMA. В отличие от этого, подходящее приложение для NUMA ориентировано в режиме реального времени и критично ко времени.
- Параллельные системы на основе UMA работают медленнее, чем системы NUMA.
- NUMA имеет пропускную способность больше, чем UMA.

25. Структура современных процессоров. Окружение процессора. CISC, RISC, VLIW.



Современные коммерческие процессоры

- Разрядность адреса и данных 16/32/64 бита
- Тактовые частоты 500МГц-5Гц.
- Многопроцессорные 1-100+ CPU
- Многоядерные 1-16 ядер
- От 1 ГБ до терабайтов ОЗУ
- Используют кэш-память разных уровней
- Суперскалярные
- CISC, RISC, VLIW

Суперскалярный – больше 1 команды за 1 такт тактового генератора.



CISC, RISC, VLIW

- Complex Instruction Set Computer
 - Традиционные процессоры (например Intel), отягощенные совместимостью
- Reduced Instruction Set Computer
 - Простой набор инструкций, выполнение инструкции за такт
- Very Long Instructions Word
 - Несколько инструкций, упакованных в одну команду
 - Упаковка операций в инструкцию ложится на компилятор

Intel использует cisc и это историческая архитектура в которой много мусора, который хранится ради поддержки программ на прошлых сборках

Как показала практика разница в производительности между RISC и CISC небольшая поэтому интел так и остался CISC

CISC (Complex Instruction Set Computer) — **процессор с большим набором сложных команд**. Одна команда может делать много: например, взять оба операнда прямо из памяти, выполнить операцию и записать результат обратно в память — всё за одну команду. Команды имеют **разную длину**, режимов адресации много, а реализуются команды микропрограммой.

Плюс: программа получается **компактной** — одна команда заменяет несколько, экономится память (это было критично, когда память стоила дорого), и компилятору проще.

Минус: команды разной длины и сложности **трудно конвейеризировать** — процессору тяжело выполнять их быстрым потоком, потому что разные команды занимают разное время. Управление получается сложным.

Пример: архитектура x86 (процессоры Intel и AMD для ПК).

Для CISC-архитектуры типичны:

- наличие в процессоре сравнительно небольшого числа регистров общего назначения;
- большое количество машинных команд, некоторые из них аппаратно реализуют сложные операторы ЯВУ;
- разнообразие способов адресации операндов;
- множество форматов команд различной разрядности;
- наличие команд, где обработка совмещается с обращением к памяти.

RISC

Процессор с небольшим набором простых команд. Система команд имеет упрощенный вид. Все команды одинакового формата с простой кодировкой. с памятью работают только две команды — загрузка из памяти в регистр и сохранение из регистра в память; все вычисления идут **только между регистрами**. Поэтому регистров делают **много**. Управление **аппаратное** (без микропрограммы), команда выполняется за один такт. Команда, поступающая в CPU, уже разделена по полям и не требует дополнительной дешифрации.

Плюс: простые одинаковые команды легко загнать в конвейер и выполнять быстрым непрерывным потоком — **отсюда высокое быстродействие**. Распараллеливанием команд занимается **аппаратура** процессора. CPU дешевле, потому что для выполнения инструкций нужно меньше логических элементов

Минус: Отладка программ на RISC более сложна, программа длиннее

Пример: ARM (смартфоны, планшеты, новые Mac), MIPS, RISC-V.

VLIW - архитектура процессоров с несколькими вычислительными устройствами. Характеризуется тем, что одна инструкция процессора содержит несколько операций, которые должны выполняться параллельно, базируется на RISC-архитектуре. – Несколько инструкций, упакованных в одну команду – Упаковка операций в инструкцию ложится на компилятор. Идея VLIW базируется на том, что задача эффективного планирования параллельного выполнения нескольких команд возлагается на «разумный» компилятор. Такой компилятор вначале исследует исходную программу с целью обнаружить все команды, которые могут быть выполнены одновременно,

причем так, чтобы это не приводило к возникновению конфликтов. В процессе анализа компилятор может даже частично имитировать выполнение рассматриваемой программы. На следующем этапе компилятор пытается объединить такие команды в пакеты, каждый из которых рассматривается так одна сверхдлинная команда.

Плюс: аппаратура **упрощается** — ей не нужно на лету выяснять, что можно распараллелить, всё уже расписано компилятором.

Минус: вся сложность ложится на **компилятор** (написать хороший очень трудно), а скомпилированный код **плохо переносится** между разными процессорами — он жёстко привязан к конкретному числу и набору исполнительных устройств.

Пример: Intel Itanium, многие сигнальные процессоры (DSP) и графические ускорители.

Характеристика	CISC	RISC	VLIW
Длина команды	Варируется	Единая	Единая
Расположение полей в команде	Варируется	Неизменное	Неизменное
Количество регистров	Несколько (часто специализированных)	Много регистров общего назначения	Много регистров общего назначения
Доступ к памяти	Может выполняться как часть команд различных типов	Выполняется только специальными командами	Выполняется только специальными командами -

На практике граница CISC/RISC размылась. Современные **x86-процессоры снаружи**

CISC (понимают старый сложный набор команд ради совместимости со всем накопленным софтом), но **внутри** аппаратно **разбивают каждую сложную команду на простые RISC-подобные микрооперации** и уже их прогоняют через быстрый конвейер. То есть процессор получает совместимость от CISC и скорость от RISC одновременно. А в мобильных устройствах и новых компьютерах побеждает чистый RISC (ARM) — за счёт простоты, низкого энергопотребления и хорошей конвейеризации.

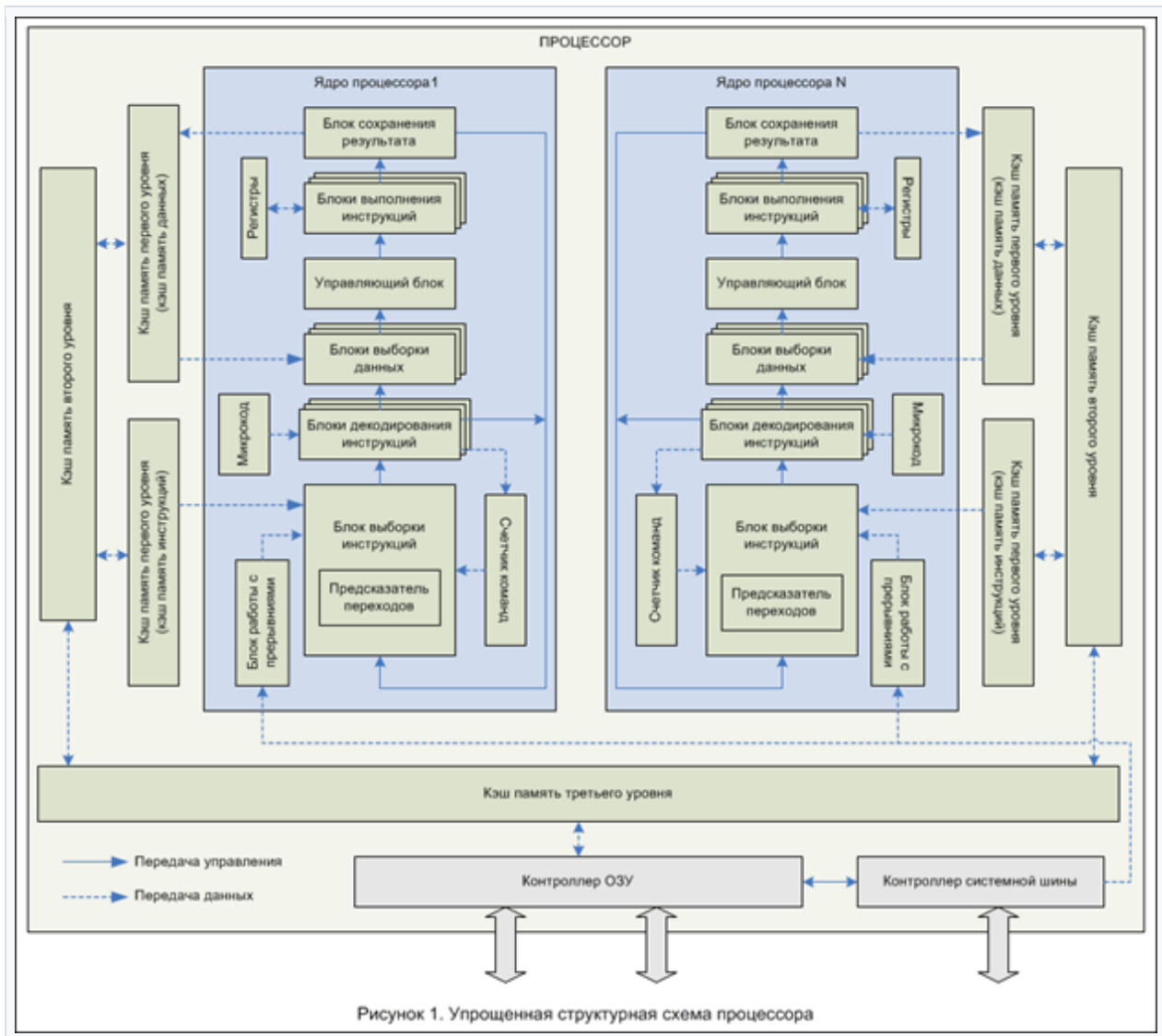
SPARC — это представитель RISC, который имеют собственную черту — регистровые окна. SPARC пошёл дальше и сделал так, что программа в каждый момент видит не все регистры сразу, а **«окно»** — небольшой набор доступных регистров. При **вызове подпрограммы** окно **сдвигается**: часть регистров вызывающей программы и часть вызываемой **перекрываются**, образуя готовый канал для передачи параметров.

Смысл в ускорении вызовов подпрограмм. Обычно при вызове приходится сохранять регистры в стек (в память), а при возврате восстанавливать — а обращения к памяти медленные. В SPARC при вызове окно просто сдвигается на новый набор физических регистров, и сохранять/восстанавливать регистры в память чаще всего **не требуется** — всё остаётся в быстрых регистрах. Параметры передаются через перекрывающуюся часть окон без копирования. Это особенно выгодно для программ с большим числом вложенных вызовов.

Оборотная сторона: физических регистров конечное число, и когда вложенных вызовов становится слишком много, окна заканчиваются — тогда происходит **переполнение окон**, и часть регистров всё-таки сбрасывается в память (а при возврате — подчитывается обратно). То есть механизм ускоряет типичный случай, но не отменяет работу с памятью полностью.

Структура современного процессора

Ядро процессора – это его основная часть, содержащая все функциональные блоки и осуществляющая выполнение всех логических и арифметических операций.



Принцип работы ядра процессора основан на цикле, описанном еще Джоном фон Нейманом в 1946 году. В упрощенном виде этапы цикла работы ядра процессора можно представить следующим образом:

1. **Блок выборки инструкций** проверяет наличие **прерываний**. Если прерывание есть, то данные регистров и счетчика команд заносятся в стек, а в счетчик команд заносится адрес команды обработчика прерываний. По окончании работы функции обработки прерываний, данные из стека будут восстановлены;
2. Блок выборки инструкций из счетчика команд считывает адрес команды, предназначенной для выполнения. По этому адресу из КЭШ-памяти или ОЗУ считывается команда. Полученные данные передаются в блок декодирования;
3. **Блок декодирования** команд расшифровывает команду, при необходимости используя для интерпретации команды записанный в ПЗУ микрокод. Если это команда перехода, то в счетчик

команд записывается адрес перехода и управление передается в блок выборки инструкций (пункт 1), иначе счетчик команд увеличивается на размер команды (для процессора с длинной команды 32 бита – на 4) и передает управление в блок выборки данных;

4. **Блок выборки данных** считывает из КЭШ-памяти или ОЗУ требуемые для выполнения команды данные и передает управление **планировщику**;

5. **Управляющий блок** определяет, какому блоку выполнения инструкций обработать текущую задачу, и передает управление этому блоку;

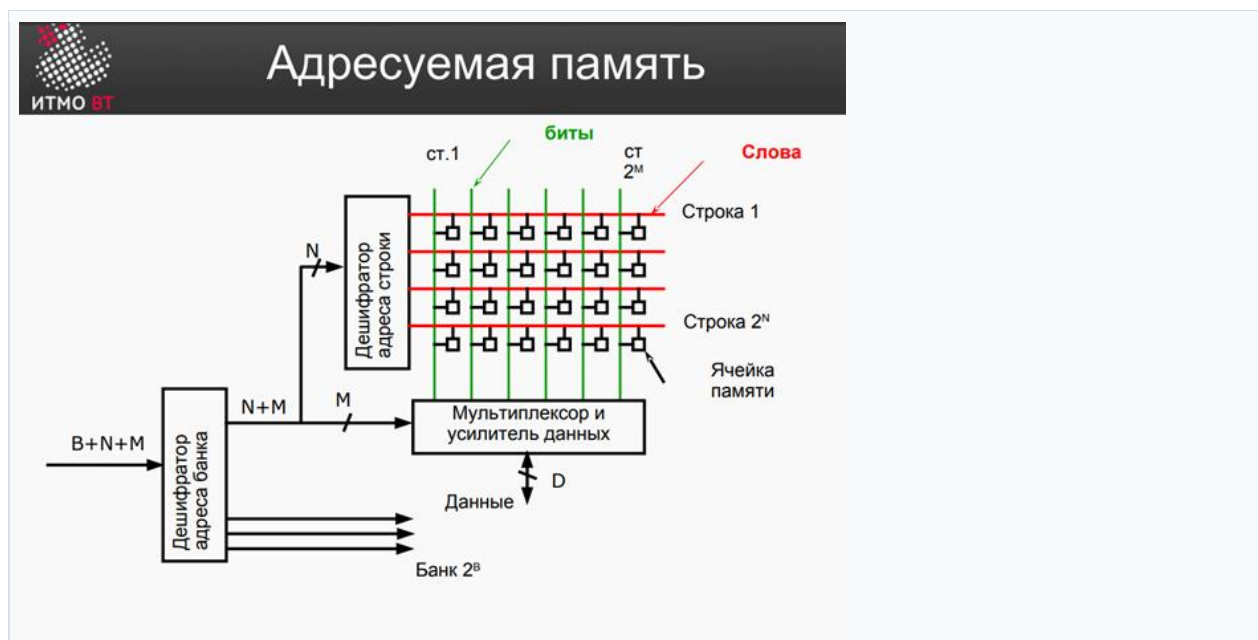
6. **Блоки выполнения инструкций** выполняют требуемые командой действия и передают управление **блоку сохранения результатов**;

7. При необходимости сохранения результатов в ОЗУ, блок сохранения результатов выполняет требуемые для этого действия и передает управление блоку выборки инструкций (пункт 1).

Описанный выше цикл называется процессом (именно поэтому процессор называется процессором). Последовательность выполняемых команд называется программой.

Скорость перехода от одного этапа цикла к другому определяется тактовой частотой процессора, а время работы каждого этапа цикла и время, затрачиваемое на полное выполнение одной инструкции, определяется устройством ядра процессора.

26. Адресуемая память, организация и временные диаграммы. Конструктивные особенности современной памяти.



Адресуемая (оперативная) память хранит данные в ячейках, к каждой из которых обращаются по адресу. Память организована как матрица ячеек. адрес поступает на дешифратор, выбирающий нужную строку (и столбец), после чего по шине данных выполняется чтение или запись под управлением сигналов "выбор микросхемы", "чтение/запись".

Адрес, который приходит на память, поступает на **дешифратор**. Дешифратор — это схема, которая по двоичному адресу активирует **ровно одну** из своих выходных линий. Часть адреса дешифрируется как **номер строки**, другая часть — как **номер столбца**, и на их пересечении выбирается нужная ячейка. После того как ячейка выбрана, по **шине данных** выполняется чтение или запись.

чтобы уменьшить число адресных выводов микросхемы, адрес часто подают **в два приёма** — сначала адрес строки, потом адрес столбца, по одним и тем же линиям. Для этого служат специальные стробирующие сигналы: **строб адреса строки (RAS)** и **строб адреса столбца (CAS)**. Сначала по линиям выставляется адрес строки и подаётся RAS — строка фиксируется внутри; затем по тем же линиям выставляется адрес столбца и подаётся CAS. Это вдвое сокращает количество адресных ножек, что важно для плотных микросхем.

Помимо адреса и данных, у памяти есть **управляющие сигналы**: выбор микросхемы (CS — разрешает работу именно этой микросхемы, когда их несколько), и сигнал, задающий операцию — чтение или запись (RD/WR).

На вход всей памяти приходит полный адрес шириной **$B + N + M$** бит (стрелка слева). Дешифратор банка берёт из него старшие **B** бит и выбирает нужный **банк** (одна из выходных линий внизу активирует свой банк). Оставшиеся **$N + M$** бит идут дальше — внутрь выбранного банка: N бит на дешифратор строки, M бит на мультиплексор столбца.

После того как дешифратор строки выбрал строку, на столбцах оказываются биты **всего слова** этой строки. Из них нужно выбрать нужные и передать на выход:

Мультиплексор по **M** линиям адреса столбца (стрелка с подписью M) выбирает нужный столбец/нужные биты из выбранной строки — то есть отбирает из строки конкретную часть.

Усилитель данных усиливает слабый сигнал, считанный с ячейки (заряд в ячейке мал, особенно в динамической памяти), до нормального логического уровня.



Временные диаграммы



Все очевидно, однако важно отметить, что линия W/R – не на шине, а внутри памяти.

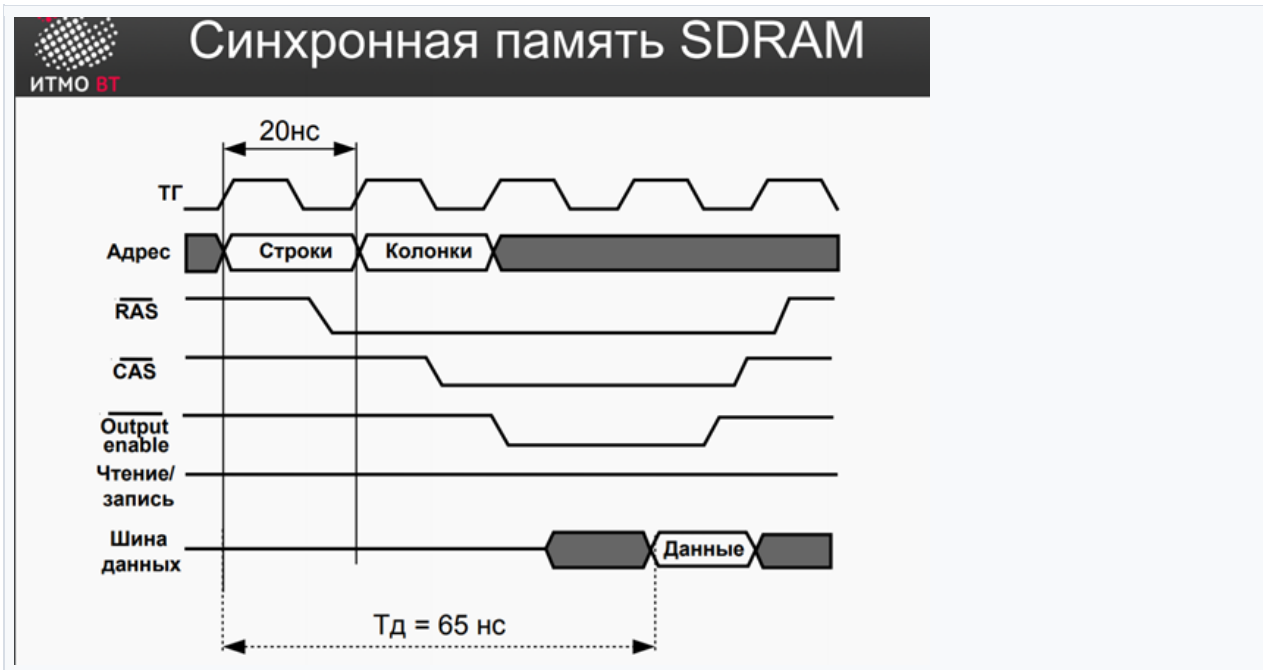
По сигналу чтения начинают срабатывать дешифраторы

T_d – время доступа, T_v – время восстановления

Временная диаграмма – это графики сигналов во времени: они показывают, в каком порядке и как долго действуют адрес, управляющие сигналы и данные. Память не выдаёт результат мгновенно – все сигналы должны быть поданы в правильной последовательности и выдержаны определённое время, и временная диаграмма как раз описывает этот «протокол» обращения.

при чтении выставляется адрес, подается сигнал выбора и чтения, по истечении времени доступа на шине данных появляются достоверные данные. Цикл записи: на адресных линиях выставляется адрес; на шину данных подаются записываемые данные; подаётся сигнал записи. По этому сигналу данные с шины заносятся в выбранную ячейку.

ключевые временные параметры - время доступа (от подачи адреса до получения данных) и время цикла (минимальный интервал между обращениями). Эти параметры определяют быстродействие памяти



Output enable – когда память читает

1. Передали адрес строки подтвердив его RAS
2. Передали адрес столбца подтвердив его CAS
3. Подключили выход памяти(Output enable)
4. Далее начинают появляться данные
5. Возврат в исходное состояние

Конструктивные особенности современной памяти

Burst mode – пакетный режим, при котором на запрос по конкретному адресу возвращаются не только данные, хранящиеся по этому адресу, но и пакет данных по нескольким последующим адресам.

SPD – При включении **контроллер памяти читает SPD** и автоматически настраивается на правильную частоту и тайминги – пользователю не нужно ничего выставлять вручную

Double Data Rate – передача данных и по фронту и по спаду. DDR использует **оба перепада** тактового сигнала: и нарастающий, и спадающий. В результате за тот же период такта передаётся **вдвое больше** данных **без повышения тактовой частоты**.

Interleaving - расслоение памяти - это размещение **соседних по адресам ячеек в разных модулях (банках)** памяти, чтобы к ним можно было обращаться **параллельно (одновременно)**

При расслоении **на два направления** ячейки с **чётными** адресами кладут в один модуль, а с **нечётными** — в другой, так что появляется возможность производить несколько обращений одновременно. Если бы все соседние ячейки лежали в одном модуле, пришлось бы ждать его освобождения перед каждым следующим обращением. А при расслоении, **пока первый модуль ещё обрабатывает чётную ячейку, второй модуль уже выдаёт нечётную** — обращения **накладываются** во времени и идут параллельно. При простых последовательных обращениях к основной памяти ячейки выбираются поочередно.

Как ускоряют современную DRAM

Синхронность (SDRAM). Память работает **синхронно с тактовым сигналом**, согласованно с процессором, что упорядочивает и ускоряет обмен по сравнению со старой асинхронной памятью.

Пакетный (блочный) доступ. Поскольку обращения к памяти обычно идут к **соседним адресам** (вспомним локальность), память отдаёт не одну ячейку, а **сразу пакет идущих подряд данных** одним обращением. Один раз указали начало строки — и быстро «выливается» целая порция.

Конвейерный доступ. Память начинает обрабатывать следующее обращение, **не дожидаясь полного завершения предыдущего**, накладывая их по времени, — это поднимает пропускную способность.

Передача по обоим фронтам такта (DDR). Данные передаются и по нарастающему, и по спадающему фронту тактового сигнала — то есть **дважды за такт**, что удваивает скорость передачи без повышения частоты. На этом принципе построены современные модули DDR.

27. Память, ориентированная на записи (блочная память). Организация дисковой памяти и памяти на магнитных лентах.

В **адресуемой памяти** (оперативной) доступна **каждая отдельная ячейка** по её адресу, и время доступа к любой ячейке одинаково. **Блочная память** устроена иначе: это класс внешних запоминающих устройств, где минимальная единица обмена — не байт и не слово, а целый **блок (запись)** данных фиксированного размера.

Так устроены **внешние запоминающие устройства** — диски и ленты. Их назначение — хранить **большие объёмы** информации **энергонезависимо** (данные сохраняются при выключении питания), в отличие от оперативной памяти, которая быстрая, но малоёмкая и энергозависимая. Платой за объём и энергонезависимость становится **медленный доступ и блочный характер** обмена. К этой группе относятся жёсткие диски (HDD), твердотельные накопители (SSD), массивы RAID, а также многие оптические диски в блочном режиме. Ключевые характеристики: латентность доступа, скорость последовательного переноса данных, IOPS (операции ввода-вывода в секунду) и надёжность (ECC, резервирование, SMART/мониторинг).

Организация дисковой памяти

HDD — жёсткий магнитный диск. Имеет **прямой (произвольный) доступ**. Головки на подвижном кронштейне могут **переместиться к любой дорожке**, а вращение подведёт любой сектор под головку. Значит, к **любому блоку можно обратиться напрямую** — нужно лишь время на позиционирование и поворот. Порядок обращений произвольный.

Магнитный диск — это одна или несколько **вращающихся пластин** с магнитным покрытием, над которыми перемещаются **головки** чтения-записи. Информация хранится в виде намагниченных участков на поверхности.

Дорожки (треки) — концентрические окружности на поверхности пластины, вдоль которых записываются данные. Головка, оставаясь на месте, читает целую дорожку за оборот.

Секторы — дорожка разбита на секторы; **сектор и есть тот самый блок** — минимальная адресуемая порция обмена с диском.

Цилиндр — если пластин несколько, то дорожки одного радиуса на всех поверхностях образуют цилиндр. Данные одного цилиндра доступны без перемещения головок (нужно только дожждаться вращения), поэтому их выгодно размещать вместе.

Чтобы добраться до нужного блока, складываются **три составляющие времени доступа** — это ключевой момент ответа:

Время позиционирования (seek time) — время перемещения головки на нужную **дорожку** (цилиндр). Это механическое перемещение и самая «дорогая» по времени операция.

Время ожидания (rotational latency) — после того как головка встала на дорожку, нужно дожждаться, пока **нужный сектор подойдёт** под головку за счёт вращения диска. В среднем это половина оборота.

Время передачи — собственно считывание/запись блока, пока он проходит под головкой.

Из-за того что для доступа к произвольному блоку нужны механические перемещения, диск — устройство **прямого (полупроизвольного) доступа**: можно обратиться к любому блоку, но с заметной задержкой, и доступ к соседним блокам быстрее, чем к разбросанным. Это резко отличает диск от оперативной памяти, где порядок обращений на время не влияет.

Плюсы: большая ёмкость при **низкой цене за гигабайт** (самое дешёвое хранилище большого объёма из «оперативных»); долгий срок хранения данных; нет ограничения на число перезаписей.

Минусы: **медленный** (особенно произвольный доступ — мешает позиционирование и вращение); **механика** — боится ударов и вибрации, изнашивается, шумит; больше потребляет и греется; задержки в миллисекундах.

SSD — твердотельный накопитель

SSD — **полностью электронное** устройство **без движущихся частей**. Данные хранятся в микросхемах **флеш-памяти (NAND)**: бит хранится как **заряд в ячейке** на транзисторе с плавающим затвором (заряд удерживается даже без питания — отсюда энергонезависимость). Ячейки объединены в **страницы** и **блоки**. Работой управляет **контроллер** накопителя — он распределяет запись по ячейкам, выполняет служебные операции и выравнивание износа.

Важная конструктивная особенность флеш-памяти: **читать и писать можно страницами, но стирать — только целыми блоками**. Ячейку нельзя перезаписать «поверх» — её сначала надо стереть. Это усложняет запись и приводит к тому, что контроллер постоянно перераспределяет данные.

Износ. Ячейки флеш-памяти имеют **ограниченное число циклов перезаписи** — после многих перезаписей ячейка изнашивается. Поэтому контроллер применяет **выравнивание износа** — равномерно распределяет записи по всем ячейкам, чтобы ни одна не изнашивалась раньше других.

Плюсы: **очень быстрый** — нет механики, доступ почти мгновенный (микросекунды), произвольный доступ так же быстр, как последовательный (нет позиционирования); **бесшумный**, стойкий к ударам; мало потребляет и греется; компактный.

Минусы: **дороже** за гигабайт, чем HDD; **ограниченный ресурс перезаписей** (хотя для обычного использования его хватает на годы); при длительном хранении **без питания** заряд в ячейках со временем может стекать (для архивов «в стол» хуже, чем магнитные носители).

Применение: там, где важна скорость — системный диск (ОС, программы), рабочие данные, ноутбуки. Сейчас это основной тип накопителя в ПК и мобильных устройствах.

в HDD, и в SSD есть кэш — это микросхема быстро памяти на плате накопителя под управлением его контроллера, она буферизует чтение (упреждающее чтение соседних блоков) и запись (отложенная запись). В **HDD** кэш сглаживает медленную механику. В **SSD** кэш дополнительно хранит **таблицу трансляции адресов**, а ещё есть **SLC-кэш** — часть флеш-памяти в быстром режиме для ускорения записи (один бит на ячейку — быстрее и надёжнее, чем обычный многобитовый режим хранения).

Магнитная лента

Лента — **гибкая полоса с магнитным покрытием**, намотанная на катушку в картридже. Для работы картридж вставляется в **лентопротяжный механизм (стример)**, который протягивает ленту мимо неподвижной головки чтения-записи. Данные записываются **строго последовательно** вдоль ленты, **блоками**, разделёнными **межблочными промежутками** (нужны для разгона и остановки механизма).

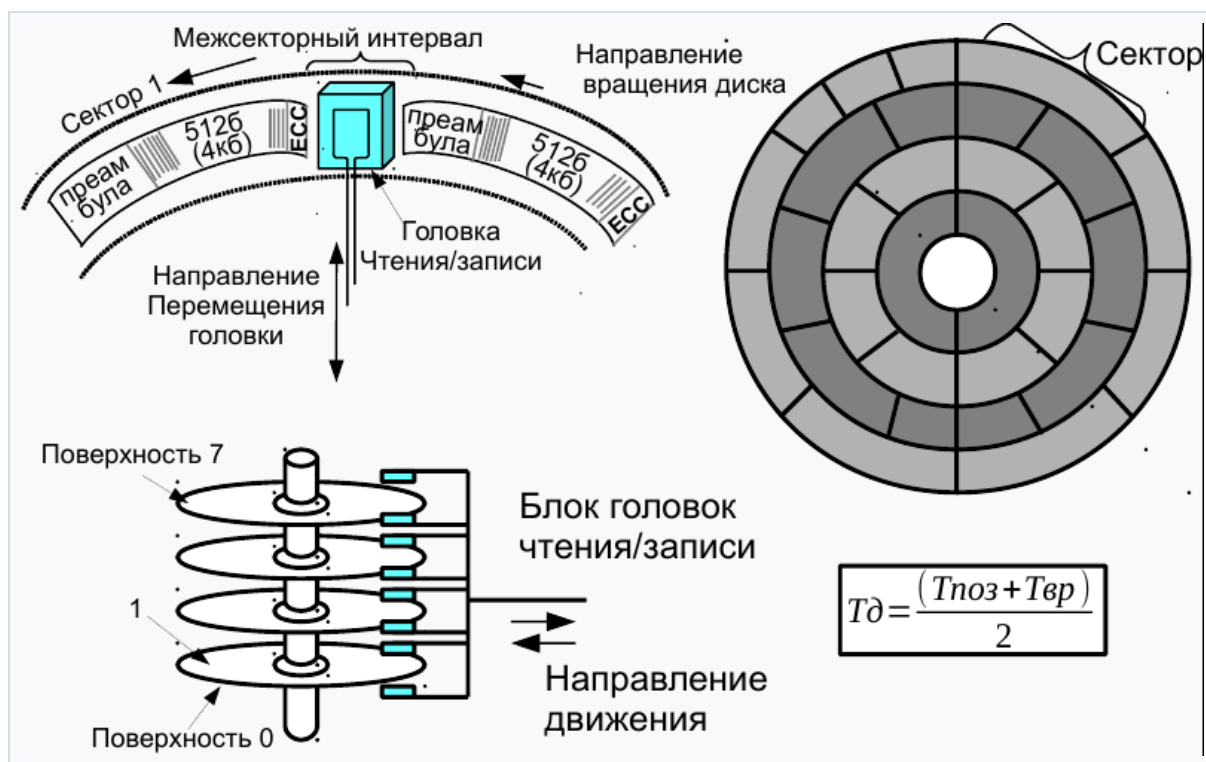
Тип доступа — последовательный: чтобы добраться до нужной записи, ленту надо физически **перемотать** до этого места.

Плюсы: самая дешёвая память за гигабайт и **очень большая ёмкость** на картридж; **долговечность хранения** — десятки лет, заряд/намагниченность держится без питания; картриджи компактны и не потребляют энергию в покое (лежат на полке).

Минусы: **последовательный доступ** — огромное время доступа к произвольной записи (перемотка); непригодна для оперативной работы с разрозненными данными; нужен отдельный механизм-стример для чтения; медленный «старт» до нужного места.

Применение: **резервное копирование и долговременное архивное хранение** больших объёмов, которые пишутся и читаются подряд и редко (бэкапы дата-центров, архивы). Именно дешевизна и ёмкость держат ленту в использовании до сих пор, несмотря на медленный доступ.

Сравнение и практические выводы простыми словами: если нужно быстро и часто читать/писать маленькие порции в произвольных местах — берём SSD (NVMe, если важна максимальная производительность). Если нужен большой объём дёшево с приемлемой скоростью крупных последовательных операций — подойдут HDD (часто в RAID). Если цель — долговременное, очень дешёвое хранение и бэкапы с редким доступом — лента (LTO и библиотека). Во всех случаях ОС и СХД стараются отправлять данные крупными блоками, сглаживать пики через буферы/кэш и параллелить запросы (очереди/интерливинг), а для надёжности — используют ECC, SMART/monitoring, RAID, регулярные бэкапы и проверку целостности.



28. Характеристики запоминающих устройств. Пирамида памяти.

Любое запоминающее устройство описывают набором показателей. Перечислю основные.

Месторасположение — где находится память относительно процессора: **процессорная** (регистры внутри процессора), **внутренняя** (оперативная память, кэш), **внешняя** (диски, ленты, флеш-накопители). Чем ближе к процессору, тем быстрее, но тем меньше объём.

Ёмкость — объём хранимой информации. Измеряется в байтах с **метрическими** (кило-, мега-, гига- — степени 1000) и **двоичными** (киби-, меби-, гоби- — степени 1024) множителями.

Единица пересылки — минимальная порция обмена. Для оперативной памяти это **слово**, для кэша — **строка кэша** (блок), для дисков и лент — **блок (сектор)**. Это важная характеристика: адресуемая память обменивается словами, а блочная — целыми блоками.

Метод доступа — как находят данные: **произвольный (адресный)** — по адресу, время одинаково для любой ячейки (ОЗУ); **прямой** — к любому блоку, но с задержкой на позиционирование (диск); **последовательный** — только перемоткой по порядку (лента); **ассоциативный** — по содержимому/тегу (кэш).

Быстродействие — насколько быстро память работает (детально ниже).

Физический тип — на каком принципе построена: **полупроводниковая** (триггеры в SRAM, конденсаторы в DRAM, заряд в плавающем затворе у флеш), **магнитная** (диски, ленты), **оптическая** (CD/DVD).

Стоимость хранения единицы информации (цена за гигабайт) и **энергозависимость** — теряются ли данные при выключении питания. Полупроводниковая оперативная память энергозависима; магнитная и флеш — энергонезависимы.

Временные характеристики быстродействия

Быстродействие раскрывается через несколько параметров — их надо называть отдельно, это часто спрашивают.

Время доступа — интервал **от момента выставления адреса до момента установления связи с ячейкой**(получения достоверных данных). Это основная характеристика отзывчивости памяти.

Время цикла (длительность цикла обращения) — **минимальный интервал между двумя последовательными обращениями** к памяти. Может быть больше времени доступа, потому что после обращения памяти иногда нужно время на восстановление, прежде чем она готова принять следующий запрос.

Время чтения и время записи — длительности соответствующих операций (могут различаться).

Время восстановления — время, нужное памяти, чтобы прийти в готовность после обращения.

Скорость передачи (пропускная способность) — сколько данных память отдаёт в единицу времени при потоковом обмене. Важно: время доступа (задержка одного обращения) и пропускная способность (поток подряд) — **разные** вещи; современная память берёт именно пропускной способностью за счёт пакетного доступа, расслоения и DDR.

Главное противоречие — почему нельзя сделать «всё сразу»

Теперь центральная мысль, из которой следует пирамида. Между характеристиками есть **фундаментальное противоречие**:

Чем быстрее память — тем она дороже и тем меньше её объём. И наоборот: чем больше и дешевле память — тем она медленнее.

Это не случайность, а следствие физики и стоимости. Быстрая память (SRAM на триггерах) требует много транзисторов на бит → дорого и малоёмко. Дешёвая ёмкая память (DRAM, магнитные носители) либо медленнее по природе, либо механическая. **Идеальной памяти — одновременно быстрой, большой и дешёвой — не существует.** Поэтому нельзя построить компьютер на одной только быстрой памяти (слишком дорого и мало) или на одной только дешёвой (слишком медленно).

Решение — пирамида (иерархия) памяти

Раз одна память не может быть всем сразу, память строят **иерархически — в виде пирамиды из нескольких уровней**, сочетая быстрые малые и медленные большие.

 Пирамида памяти					
	Объем	Тд	*	Тип	Управл.
CPU	100-1000 б.	<1нс	1с	Регистр	компилятор
L1 Cache	32-128Кб	1-4нс	2с	Ассоц.	аппаратура
L2-L3 Cache	0.5-32Мб	8-20нс	19с	Ассоц.	аппаратура
Основная память	0.5Гб-4Тб	60-200нс	50-300с	Адресная	программно
SSD	128Гб-1Тб/drive	25-250мкс	5д	Блочн.	программно
Жесткие диски	0.5Тб-4Тб/drive	5-20мс	4м	Блочн.	программно
Магнитные ленты	1-6Тб/к	1-240с	200л	Последов.	программно

29. Ассоциативная память, Кэш-память. Влияние промахов кэш-памяти на производительность.

Процессоры всегда работали быстрее, чем память. Процессоры и память совершенствовались параллельно, поэтому это несоответствие сохранялось. Поскольку на микросхему можно помещать все больше и больше транзисторов, разработчики процессоров использовали эти преимущества для создания конвейеров и супер-скалярной архитектуры, что еще больше повышало скорость работы процессоров. Разработчики памяти обычно использовали новые технологии для увеличения емкости, а не скорости, что еще больше усугубляло проблему. На практике такое несоответствие в скорости работы приводит к следующему: после того как процессор дает запрос памяти, должно пройти много циклов, прежде чем он получит слово, которое ему нужно. Чем медленнее работает память, тем дольше процессору приходится ждать, тем больше циклов должно пройти.

Есть два пути решения этой проблемы. Самый простой из них — начать считывать информацию из памяти, когда это необходимо, и при этом продолжать выполнение команд, но если какая-либо команда попытается использовать слово до того, как оно считалось из памяти, процессор должен приостанавливать работу. Чем медленнее работает память, тем чаще будет возникать такая проблема и тем больше будет проигрыш в работе. Например, если отсрочка составляет 10 циклов, весьма вероятно, что одна из 10 следующих команд попытается использовать слово, которое еще не считалось из памяти.

Другое решение проблемы — сконструировать машину, которая не приостанавливает работу, но следит, чтобы программы-компиляторы не использовали слова до того, как они считаются из памяти. Однако это не так просто осуществить на практике. Часто при выполнении команды загрузки машина не может выполнять другие действия, поэтому компилятор вынужден вставлять пустые команды, которые не производят никаких операций, но при этом занимают место в памяти. В действительности при таком подходе простаивает не аппаратное, а программное обеспечение, но снижение производительности при этом такое же.

На самом деле эта проблема не технологическая, а экономическая. Инженеры знают, как построить память, которая будет работать так же быстро, как и процессор, но при этом ее приходится помещать прямо на микросхему процессора (поскольку информация через шину поступает очень медленно). Установка большой памяти на микросхему процессора делает его больше и, следовательно, дороже, и даже если бы стоимость не имела значения, все равно существуют ограничения в размерах процессора, который можно сконструировать. Таким образом, приходится выбирать между быстрой памятью небольшого размера и медленной памятью большого размера. Мы бы предпочли память большого размера с высокой скоростью работы по низкой цене. Маленькая память с высокой скоростью работы называется кэш-памятью (от французского слова *cacher* «прятать»). Основная идея кэш-памяти проста: в ней находятся слова, которые чаще всего используются. Если процессору нужно какое-нибудь слово, сначала он обращается к кэш-памяти. Только в том случае, если слова там нет, он обращается к основной памяти. Если значительная часть слов находится в кэш-памяти, среднее время доступа значительно сокращается. Таким образом, успех или неудача зависит от того, какая часть слов находится в кэш-памяти. Давно известно, что программы не обращаются к памяти наугад. Если программе нужен доступ к адресу А, то скорее всего после этого ей понадобится доступ к адресу,

расположенному поблизости от А. Практически все команды обычной программы (за исключением команд перехода и вызова процедур) вызываются из последовательных участков памяти. Кроме того, большую часть времени программа тратит на циклы, когда ограниченный набор команд выполняется снова и снова. Точно так же при манипулировании матрицами программа скорее всего будет обращаться много раз к одной и той же матрице, прежде чем перейдет к чему-либо другому.

То, что при последовательных отсылках к памяти в течение некоторого промежутка времени используется только небольшой ее участок, называется принципом локальности. Этот принцип составляет основу всех систем кэш-памяти. Идея состоит в следующем: когда определенное слово вызывается из памяти, оно вместе с соседними словами переносится в кэш-память, что позволяет при очередном запросе быстро обращаться к следующим словам. Если слово считывается или записывается k раз, компьютеру понадобится сделать 1 обращение к медленной основной памяти и $k-1$ обращений к быстрой кэш-памяти. Чем больше k , тем выше общая производительность.

Одной из самых эффективных технологий одновременного увеличения пропускной способности и уменьшения времени ожидания является применение нескольких блоков кэш-памяти. Основная технология — **введение отдельной кэш-памяти для команд и отдельной для данных** (разделенной кэш-памяти). Такая кэш-память имеет несколько преимуществ. Во-первых, операции могут начинаться независимо в каждой кэш-памяти, что удваивает пропускную способность системы памяти. Именно по этой причине в микроархитектуре Mic-1 нам понадобились два отдельных порта памяти: особый порт для каждой кэш-памяти. Отметим, что каждая кэш-память имеет независимый доступ к основной памяти.

В настоящее время многие системы памяти гораздо сложнее этих. Между разделенной кэш-памятью и основной памятью часто помещается кэш-память второго уровня. Вообще говоря, может быть три и более уровней кэш-памяти, поскольку требуются более продвинутые системы. **Прямо на микросхеме центрального процессора находится небольшая кэш-память для команд и небольшая кэш-память для данных**, обычно от 16 до 64 Кбайт. Есть еще кэш-память второго уровня, которая расположена не на самой микросхеме процессора, а рядом с ним в том же блоке. Эта кэш-память обычно не является разделенной и содержит смесь данных и команд. Ее размер — от 512 Кбайт до 1 Мбайт. Кэш-память третьего уровня находится на той же плате, что и процессор, и обычно состоит из статического ОЗУ в несколько мегабайтов, которое функционирует гораздо быстрее, чем динамическое ОЗУ основной памяти. Обычно все содержимое кэш-памяти первого уровня находится в кэш-памяти второго уровня, а все содержимое кэш-памяти второго уровня находится в кэш-памяти третьего уровня.

Ассоциативная память (АП) или ассоциативное запоминающее устройство (АЗУ) - специальный вид машинной памяти, используемый в приложениях очень быстрого поиска. Известна также под терминами «память, адресуемая по содержимому», «ассоциативное запоминающее устройство», «контентно-адресуемая память» или «ассоциативный массив», хотя последний термин чаще используется в программировании для обозначения структуры данных

В ассоциативной памяти элементы выбираются не по адресу, а по содержимому. Поясним последнее понятие более подробно. Для памяти с адресной организацией было введено понятие минимальной адресуемой единицы (MAE) как порции данных, имеющей индивидуальный адрес. Введем аналогичное понятие для ассоциативной памяти, и будем эту минимальную единицу

хранения в ассоциативной памяти называть строкой ассоциативной памяти (СтрАП). Каждая СтрАП содержит два поля: **поле тега** (англ. tag — ярлык, этикетка, признак) и **поле данных**. Запрос на чтение к ассоциативной памяти словами можно выразить следующим образом: выбрать строку (строки), у которой (у которых) тег равен заданному значению. Особо отметим, что при таком запросе возможен один из трех результатов: имеется в точности одна строка с заданным тегом; имеется несколько строк с заданным тегом; нет ни одной строки с заданным тегом. Поиск записи по признаку — это действие, типичное для обращений к базам данных, и поиск в базе зачастую является ассоциативным поиском. Для выполнения такого поиска следует просмотреть все записи и сравнить заданный тег с тегом каждой записи. Это можно сделать и при использовании для хранения записей обычной адресуемой памяти (и понятно, что это потребует достаточно много времени — пропорционально количеству хранимых записей!). Об ассоциативной памяти говорят тогда, когда ассоциативная выборка данных из памяти поддержана аппаратно.

Принцип работы кэша процессора

Итак, мы разобрались с назначением кэша процессора, а теперь рассмотрим базовые принципы работы кэша, которые позволяют ему решать свою основную задачу.

Кэш состоит из контроллера и собственно кэшпамяти. Кэшконтроллер управляет работой кэшпамяти, то есть загружает в нее нужные данные из оперативной памяти и возвращает, когда нужно, модифицированные процессором данные в оперативную память. Архитектурно кэшконтроллер расположен между процессором и оперативной памятью. Перехватывая запросы к оперативной памяти, кэшконтроллер определяет, имеется ли копия затребованных данных в кэше. Если такая копия там есть, то это называется кэшпопаданием (cache hit) — в таком случае данные очень быстро извлекаются из кэша (существенно быстрее, чем из оперативной памяти). Если же требуемых данных в кэше нет, то говорят о кэшпромахе (cache miss) — тогда запрос данных переадресуется к оперативной памяти.

Для достижения наивысшей производительности кэшпромахи должны происходить как можно реже (в идеале — отсутствовать). Учитывая, что по емкости кэшпамять намного меньше оперативной памяти, добиться этого не такто просто. А потому основная задача кэшконтроллера заключается в том, чтобы загружать кэшпамять действительно нужными данными и своевременно удалять из нее данные, которые больше не понадобятся. Важно понимать, что кэш всегда «полон», так как оставлять часть кэшпамяти пустой нерационально. Новые данные попадают в кэш только путем вытеснения (замещения) каких либо старых данных.

Загрузка кэша данными реализуется на основе так называемой стратегии кэширования, а выгрузка данных — на основе политики замещения.

30. Предназначение и организация виртуальной памяти. Сегментно-страничная организация. Устройство управления памятью (MMU), буфер трансляции (TLB).

Виртуальная память создаёт для каждой программы иллюзию **большого, непрерывного, собственного адресного пространства**, не зависящего ни от реального объёма оперативной памяти, ни от других программ. Программа работает с **виртуальными адресами**, а аппаратура переводит их в **физические адреса** реальной памяти. Это даёт три ключевых преимущества:

- **Выполнение программ, превышающих объём ОЗУ.** Не все данные программы должны находиться в оперативной памяти одновременно: редко используемые части хранятся на **диске**, а в память подгружаются по мере надобности. Программа «видит» большое пространство, хотя физической памяти меньше.
- **Изоляция и защита процессов.** Каждый процесс имеет своё виртуальное пространство и **не может обратиться к памяти другого процесса** — аппаратура трансляции просто не даст. Это защищает программы друг от друга и от случайной или злонамеренной порчи данных.
- **Упрощение размещения программ.** Программу не нужно загружать в конкретное место памяти и заботиться о том, чтобы оно было свободно: виртуальные адреса транслируются в любые свободные физические, причём не обязательно непрерывные. Это снимает проблему фрагментации физической памяти.

Страничная организация

В большинстве современных операционных систем виртуальная память организуется с помощью страничной адресации. Оперативная память делится на страницы: области памяти фиксированной длины, которые являются минимальной единицей выделяемой памяти. Исполняемый процессором пользовательский поток обращается к памяти с помощью адреса виртуальной памяти, который делится на номер страницы и смещение внутри страницы. Процессор преобразует номер виртуальной страницы в адрес соответствующей ей физической страницы при помощи буфера ассоциативной трансляции (TLB). Если ему не удалось это сделать, то требуется дозаполнение буфера путём обращения к таблице страниц, что может сделать либо сам процессор, либо операционная система. Если страница была выгружена из оперативной памяти, то операционная система подкачивает страницу с жёсткого диска в ходе обработки события. При запросе на выделение памяти операционная система может «сбросить» на жёсткий диск страницы, к которым давно не было обращений. Критические данные (например, код запущенных и работающих программ, код и память ядра системы) обычно находятся в оперативной памяти (исключения существуют, однако они не касаются тех частей, которые отвечают за обработку аппаратных прерываний, работу с таблицей страниц и использование файла подкачки).

Сегментная организация виртуальной памяти

В сегментной организации память делится на **сегменты переменной длины по логическому смыслу**: отдельный сегмент кода, сегмент данных, сегмент стека. Адрес состоит из **номера сегмента** и **смещения** внутри него, а **таблица сегментов** хранит базовый адрес и длину каждого сегмента. При загрузке процесса часть сегментов помещается в оперативную память, а часть сегментов размещается в дисковой памяти. Во время загрузки система создает таблицу сегментов

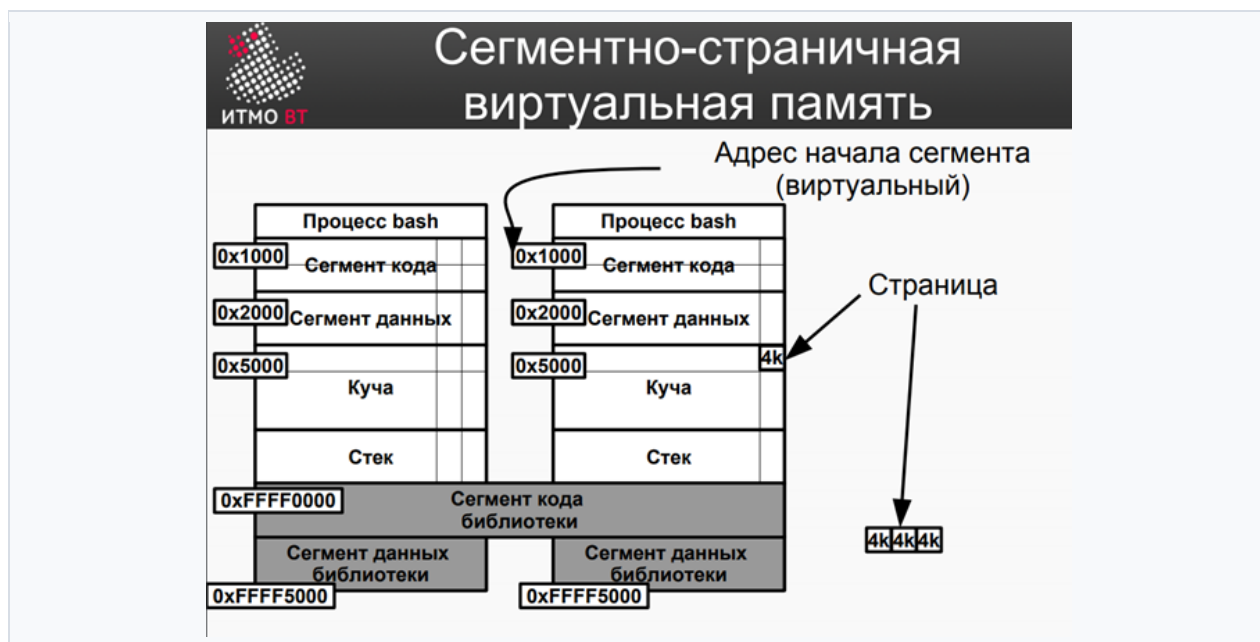
процесса, в которой для каждого сегмента указывается начальный физический адрес сегмента в оперативной памяти, размер сегмента, правила доступа, признак модификации, признак обращения к данному сегменту за последний интервал времени и некоторая другая информация. Если виртуальные адресные пространства нескольких процессов включают один и тот же сегмент, то в таблицах сегментов этих процессов делаются ссылки на один и тот же участок оперативной памяти, в который данный сегмент загружается в единственном экземпляре. Система с сегментной организацией функционирует аналогично системе со страничной организацией: время от времени происходят прерывания, связанные с отсутствием нужных сегментов в памяти, при необходимости освобождения памяти некоторые сегменты выгружаются, при каждом обращении к оперативной памяти выполняется преобразование виртуального адреса в физический. Кроме того, при обращении к памяти проверяется, разрешен ли доступ требуемого типа к данному сегменту. Виртуальный адрес при сегментной организации памяти может быть представлен парой (g, s) , где g — номер сегмента, а s — смещение в сегменте. Физический адрес получается путём сложения начального физического адреса сегмента, найденного в таблице сегментов по номеру g , и смещения s . Недостатком данного метода распределения памяти является фрагментация на уровне сегментов и более медленное по сравнению со страничной организацией преобразование адреса.

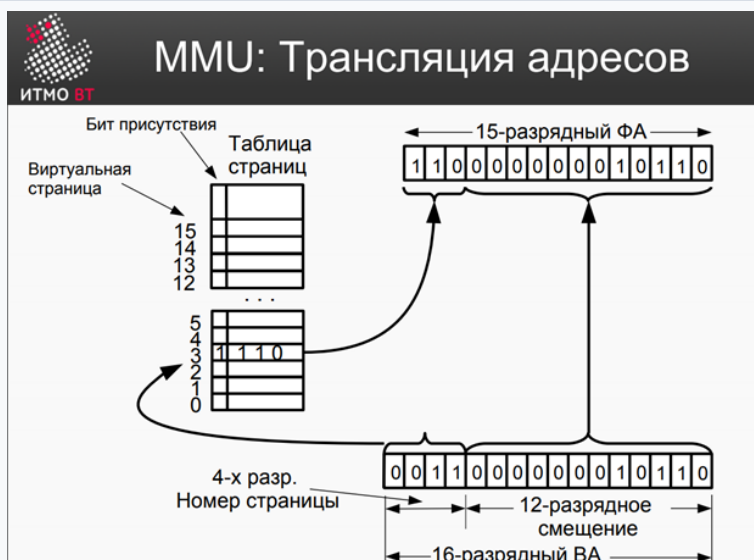
Сегментно-страничная организация

Сегментно-страничная организация **объединяет оба подхода**, беря достоинства каждого.

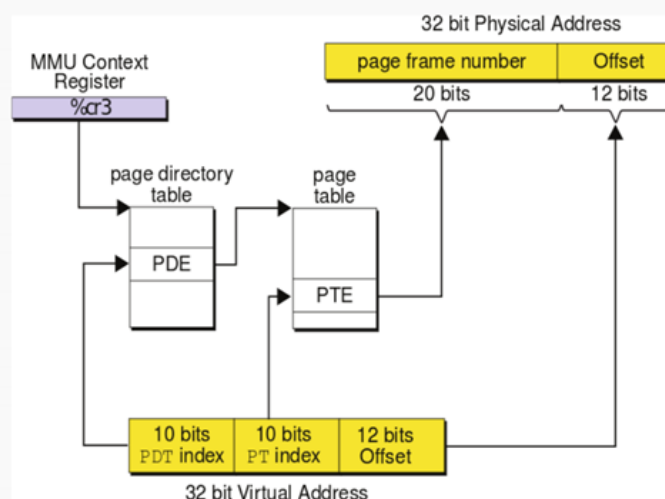
Программа делится на **сегменты** (логическая структура и защита), а каждый сегмент дополнительно делится на **страницы** (удобство размещения и подкачки).

Трансляция идёт в два этапа: по **номеру сегмента** находится таблица страниц этого сегмента, затем по **номеру страницы** — физический кадр, и добавляется **смещение**. В итоге сочетаются логическое деление и защита от сегментов с отсутствием внешней фрагментации и удобной подкачкой от страниц. Это компромисс, вобравший плюсы обеих схем.

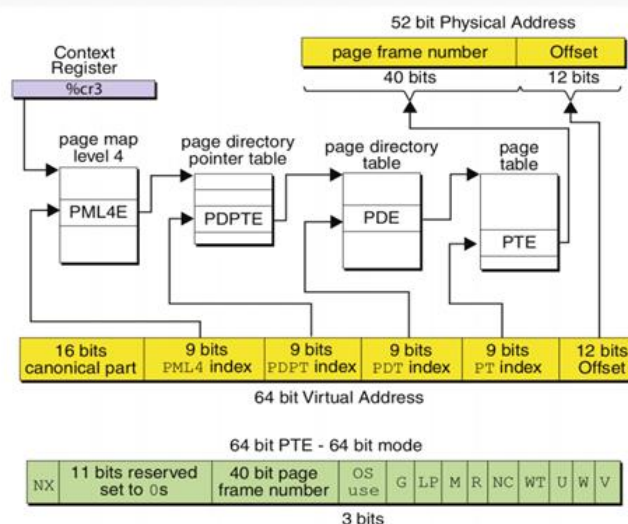




Архитектура x86: ММУ



Архитектура x64



Устройство управления памятью (MMU)

Трансляцию при **каждом** обращении к памяти нужно выполнять очень быстро, поэтому её делает **аппаратура** — **устройство управления памятью (MMU, Memory Management Unit)**, встроенное в процессор.

MMU **аппаратно преобразует виртуальный адрес в физический** по таблицам страниц: перехватывает каждый адрес, который программа выставляет, обращается к таблицам, получает физический адрес и направляет обращение в память. MMU также **проверяет права доступа** (можно ли читать/писать эту страницу) и **обнаруживает отсутствие страницы**, вызывая страничную ошибку для обработки операционной системой. Без аппаратного MMU трансляция при каждом обращении была бы недопустимо медленной.

Буфер трансляции (TLB)

Здесь возникает проблема производительности, и её решает TLB — это важная часть ответа, прямо связанная с ассоциативной памятью.

Таблицы страниц **лежат в оперативной памяти**. Значит, чтобы перевести один виртуальный адрес, MMU должен сначала обратиться к таблице в ОЗУ — то есть **каждое** обращение к памяти превратилось бы в **два** (сначала за переводом, потом за самими данными), а при многоуровневых таблицах — и больше. Это резко замедлило бы работу.

Решение — **буфер ассоциативной трансляции (TLB, Translation Lookaside Buffer)**: маленькая **быстрая ассоциативная память** внутри MMU, которая **кэширует последние переводы** «номер виртуальной страницы → номер физического кадра». Работает он на принципе ассоциативной памяти (поиск по тегу — номеру страницы) и опирается на локальность: программа обращается к одним и тем же страницам многократно.

Как это ускоряет:

Попадание в TLB (TLB hit) — нужный перевод уже есть в буфере, физический адрес получается **за один такт**, без обращения к таблицам в ОЗУ. Это подавляющее большинство случаев.

Промех в TLB (TLB miss) — перевода в буфере нет; тогда выполняется обращение к **таблицам страниц** в памяти (аппаратно или с помощью ОС), найденный перевод **заносятся в TLB**, и следующие обращения к этой странице будут попаданиями.

То есть TLB играет для трансляции адресов ту же роль, что кэш для данных: прячет медленное обращение к таблицам в памяти за быстрым ассоциативным поиском, делая виртуальную память практически бесплатной по времени в типичном случае.

31. Сетевые технологии, Понятие сети ЭВМ, классификация компьютерных сетей. Сообщение и пакет. Модель взаимодействия открытых систем.

Компьютерная сеть – это совокупность компьютеров, между которыми возможен информационный обмен без промежуточных носителей информации.

Программные компоненты состоят из сетевых ОС и сетевых приложений (почтовые программы, сетевые БД).

Все устройства, подключаемые к сети (т.е. аппаратные компоненты) можно разделить на три функциональные группы:

- рабочие станции;
- серверы сети;
- коммуникационные узлы.

Рабочая станция (workstation) – это ПК, подключенный к сети, на котором пользователь сети выполняет свою работу. Каждая рабочая станция обрабатывает свои локальные файлы и использует свою ОС.

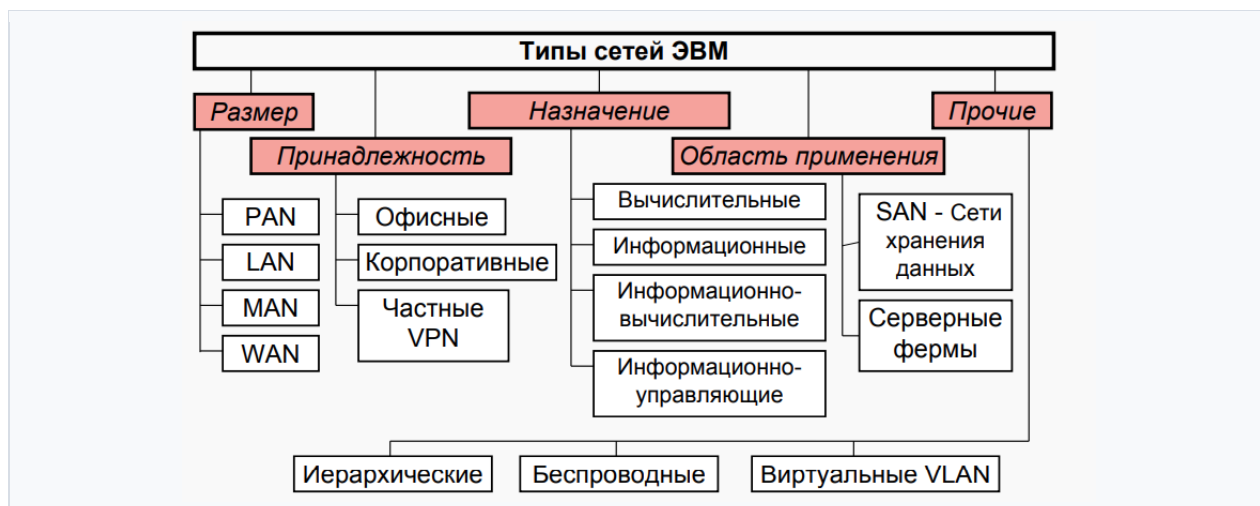
Сервер сети (server) – это компьютер, подключенный к сети и предоставляющий пользователям сети определённые услуги, например, хранение данных общего пользования, обработку запросов к СУБД, печать заданий, удалённую обработку заданий и т.д. Можно выделить следующие группы серверов: файловый сервер, сервер БД, сервер прикладных программ, факс сервер и др.

К коммуникационным узлам относятся следующие устройства:

- повторители;
- коммутаторы (мосты);
- маршрутизаторы;
- шлюзы.

Это устройства, необходимые для соединения различных сетей друг с другом.

Вычислительная сеть создается для обеспечения потенциального доступа к любому ресурсу сети для любого пользователя сети.

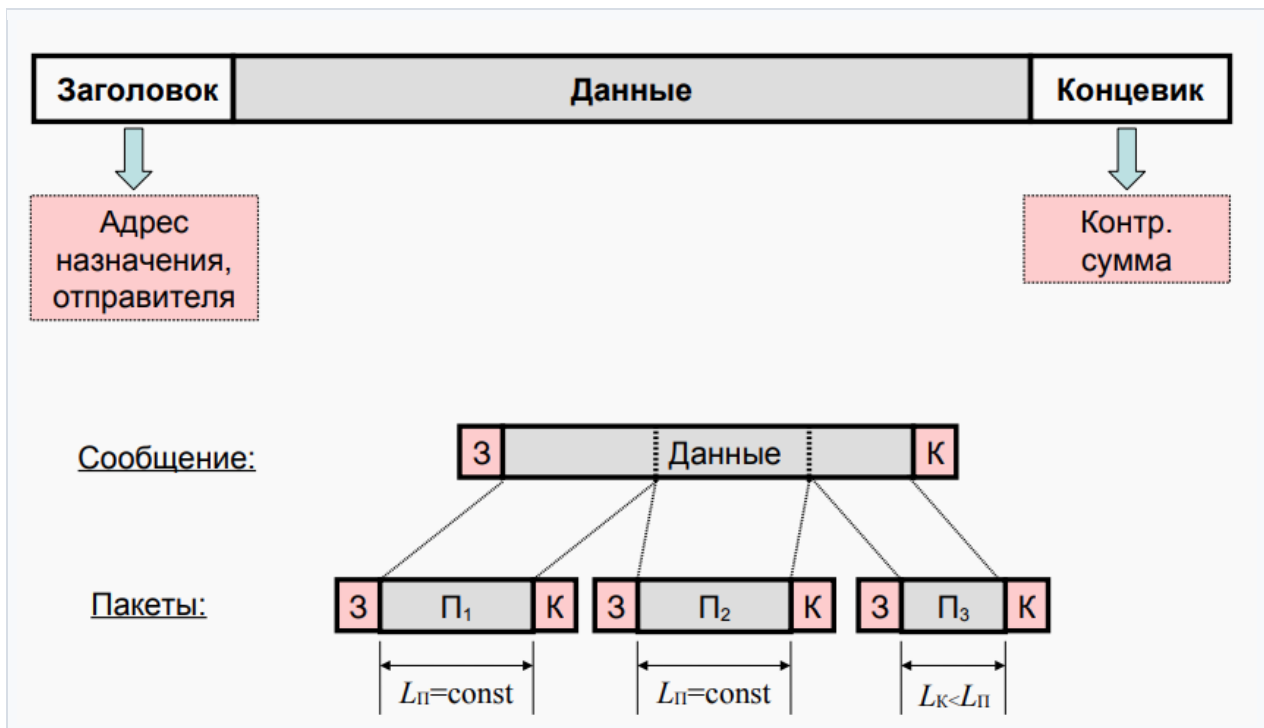


Коммутация сообщений – разбиение информации на сообщения, каждый из которых состоит из заголовка и информации.

Это способ взаимодействия, при котором создается логический канал, путем последовательной передачи сообщений через узлы связи по адресу, указанному в заголовке сообщения.

Коммутация пакетов - это особый способ коммутации узлов сети, который специально создавался для наилучшей передачи компьютерного трафика (пульсирующего трафика).

При коммутации пакетов все передаваемые пользователем сети сообщения разбиваются в исходном узле на сравнительно небольшие части, называемые пакетами. Необходимо уточнить, что сообщением называется логически завершенная порция данных - запрос на передачу файла, ответ на этот запрос, содержащий весь файл, и т. п. Сообщения могут иметь произвольную длину, от нескольких байт до многих мегабайт. Напротив, пакеты обычно тоже могут иметь переменную длину, но в узких пределах, например от 46 до 1500 байт (Ethernet). Каждый пакет снабжается заголовком, в котором указывается адресная информация, необходимая для доставки пакета узлу назначения, а также номер пакета, который будет использоваться узлом назначения для сборки сообщения.



Состоит она из 7 уровней и каждый уровень выполняет определенную ему роль и задачи.

Разберем, что делает каждый уровень снизу вверх: 1) Физический уровень (Physical Layer): определяет метод передачи данных, какая среда используется (передача электрических сигналов, световых импульсов или радиоэфир), уровень напряжения, метод кодирования двоичных сигналов. 2) Канальный уровень (Data Link Layer): он берет на себя задачу адресации в пределах локальной сети, обнаруживает ошибки, проверяет целостность данных. Если слышали про MAC-адреса и протокол «Ethernet», то они располагаются на этом уровне. 3) Сетевой уровень (Network Layer): этот уровень берет на себя объединения участков сети и выбор оптимального пути (т.е. маршрутизация). Каждое сетевое устройство должно иметь уникальный сетевой адрес в сети. Думаю, многие слышали про протоколы IPv4 и IPv6. Эти протоколы работают на данном уровне. 4)

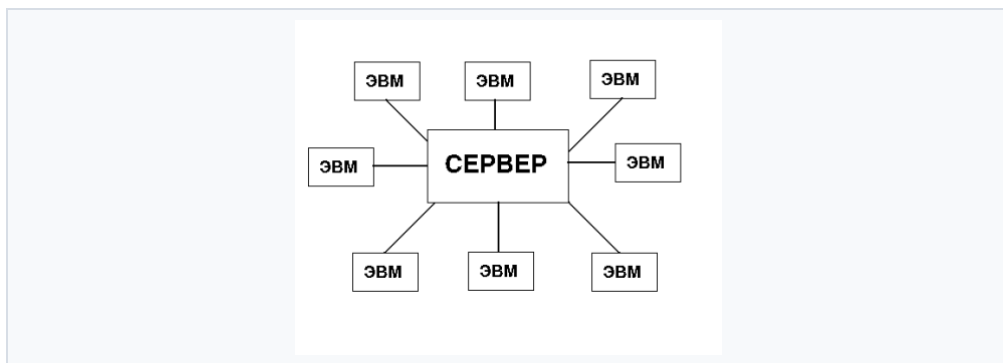
Транспортный уровень (Transport Layer): Этот уровень берет на себя функцию транспорта. К примеру, когда вы скачиваете файл с Интернета, файл в виде сегментов отправляется на Ваш компьютер. Также здесь вводятся понятия портов, которые нужны для указания назначения к конкретной службе. На этом уровне работают протоколы TCP (с установлением соединения) и UDP (без установления соединения).5) Сеансовый уровень (Session Layer): Роль этого уровня в установлении, управлении и разрыве соединения между двумя хостами. К примеру, когда открываете страницу на веб-сервере, то Вы не единственный посетитель на нем. И вот для того, чтобы поддерживать сеансы со всеми пользователями, нужен сеансовый уровень.6) Уровень представления (Presentation Layer): Он структурирует информацию в читабельный вид для прикладного уровня. Например, многие компьютеры используют таблицу кодировки ASCII для вывода текстовой информации или формат jpeg для вывода графического изображения.7) Прикладной уровень (Application Layer): Наверное, это самый понятный для всех уровень. Как раз на этом уровне работают привычные для нас приложения — e-mail, браузеры по протоколу HTTP, FTP и остальное.

Под сетью ЭВМ понимают соединение двух и более ЭВМ с целью совместного использования их ресурсов (процессоров, устройств памяти, устройств ввода/вывода, данных). По степени охвата территории различают сети:

локальные (местные) - в пределах одного учреждения, помещения (или при максимальном удалении ЭВМ не более 1км.)

региональные - внутри населенного пункта, района

национальные - внутри государства



глобальные

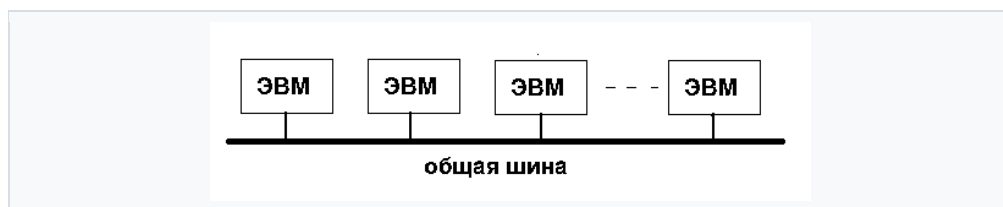
По степени доступности различают корпоративные и общедоступные сети.

По топологии:

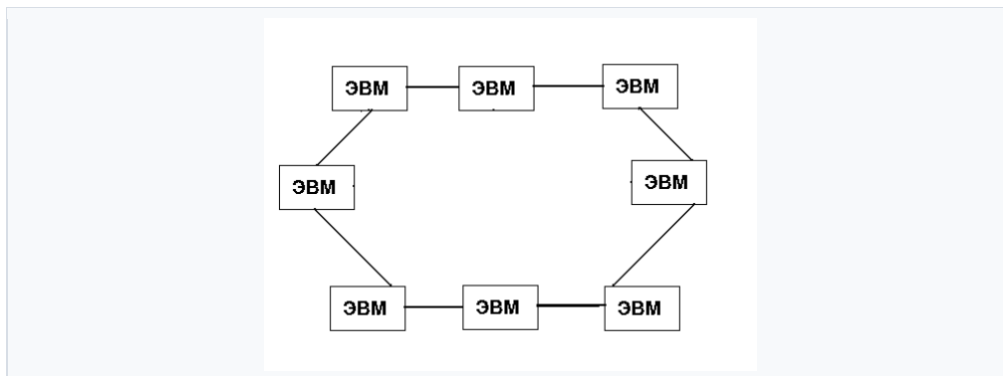
звездообразная топология

При таком способе обмен данными между ЭВМ осуществляется через более мощную ЭВМ - сервер. Недостатком такого соединения является низкая живучесть сети - выход из строя сервера означает прекращение функционирования сети. Однако, простота и дешевизна реализации сделала эту структуру популярной в локальных сетях.

топология с общей шиной

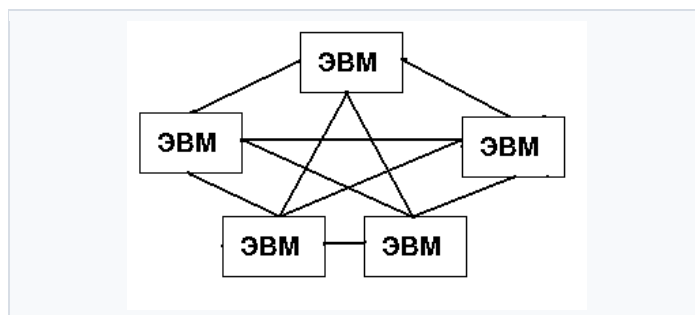


При этом способе обмен данными происходит через общую шину, которую используя механизм прерывания может "захватывать" тот или иной компьютер. Характерной особенностью здесь является отсутствие сервера. Очень часто используется в локальных сетях, а уж в "домашних" повсеместно.



кольцевая топология

В этой структуре каждая ЭВМ используя механизм прерывания работает в качестве ретранслятора. Обратите внимание, живучесть сети повышена - при одиночном обрыве связи между соседними ЭВМ сеть продолжает функционировать



Соединение ЭВМ "каждая с каждой" позволяют получить сеть самую дорогую, но и обладающую максимальной живучестью.

Сообщение – цифровые данные определенного формата, предназначенные для передачи.

Пакет – это определенным образом оформленный блок данных, передаваемый по сети. Часто состоит из заголовка и полезной нагрузки.

Уровень (layer)	Тип данных (PDU)	Функции	Примеры	
Host layers	7. Прикладной	Данные	Доступ к сетевым службам	HTTP, FTP, POP3, WebSocket

Уровень (layer)	Тип данных (PDU)	Функции	Примеры	
	6. Представления		Представление и шифрование данных	ASCII, EBCDIC
	5. Сеансовый		Управление сеансом связи	RPC, PAP, L2TP
	4. Транспортный	Сегменты/Дейтаграммы	Прямая связь между конечными пунктами и надёжность	TCP, UDP, SCTP, PORTS
Media layers	3. Сетевой	Пакеты	Определение маршрута и логическая адресация	IPv4, IPv6, IPsec, AppleTalk
	2. Канальный	Биты/Кадры (frame)	Физическая адресация	PPP, IEEE 802.22, Ethernet, DSL, ARP, сетевая карта.
	1. Физический	Биты	Работа со средой передачи, сигналами и двоичными данными	кабель (USB, «витая пара», коаксиальный, оптоволоконный), радиоканал

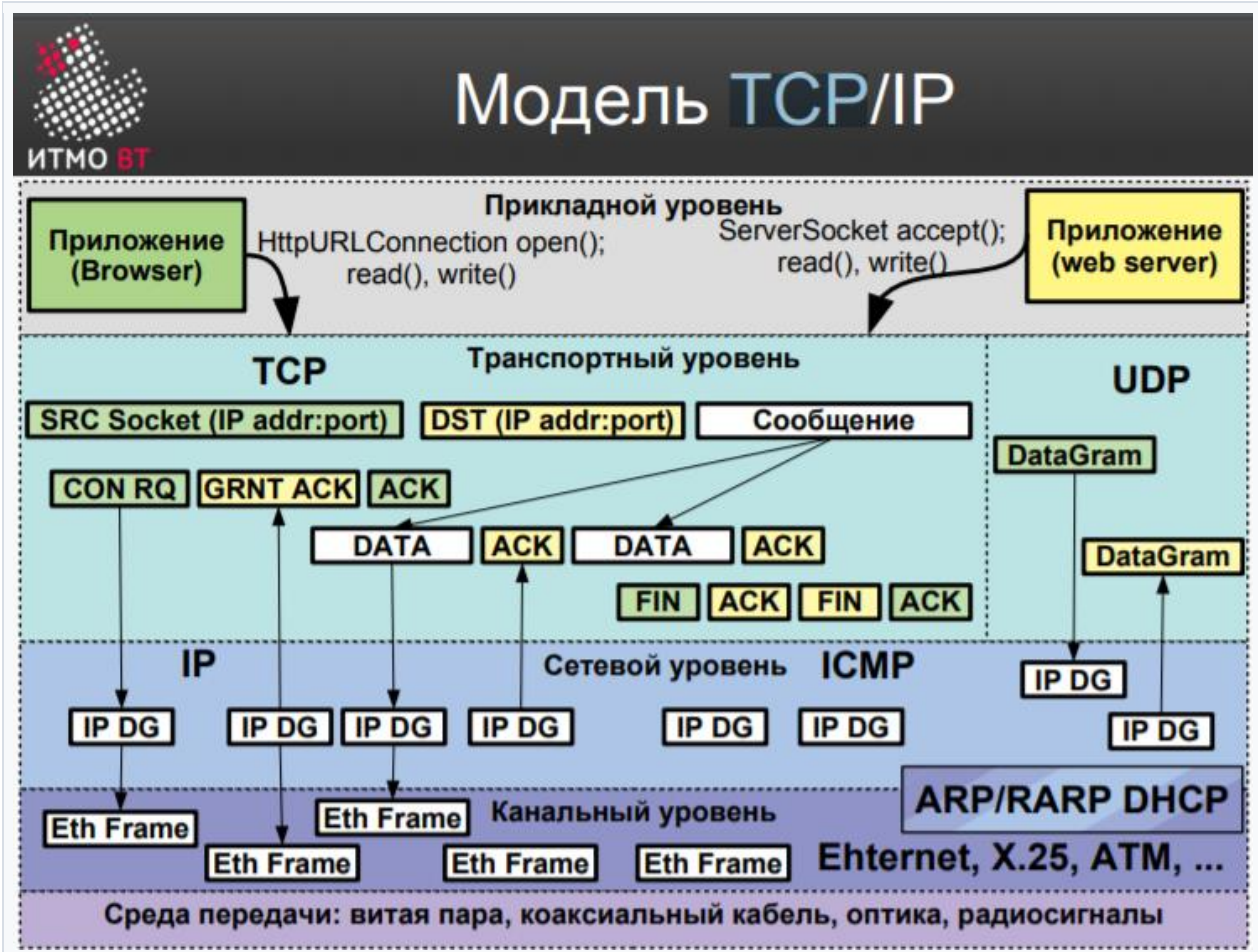
32. Модель TCP/IP: передающая среда, канальный и сетевой уровень. Адресация, передача и маршрутизация пакетов.

TCP/IP — сетевая модель передачи данных, представленных в цифровом виде. Модель описывает способ передачи данных от источника информации к получателю. В модели предполагается прохождение информации через четыре уровня, каждый из которых описывается правилом (протоколом передачи). Наборы правил, решающих задачу по передаче данных, составляют стек протоколов передачи данных, на которых базируется Интернет. Название TCP/IP происходит из двух важнейших протоколов семейства — Transmission Control Protocol (TCP) и Internet Protocol (IP), которые были первыми разработаны и описаны в данном стандарте.

Канальный уровень (англ. Data Link layer), также уровень передачи данных[1] — второй уровень сетевой модели OSI, предназначенный для передачи данных узлам, находящимся в том же сегменте локальной сети. Также может использоваться для обнаружения и, возможно, исправления ошибок, возникших на физическом уровне. Примерами протоколов, работающих на канальном уровне, являются: Ethernet для локальных сетей (многоузловой), Point-to-Point Protocol (PPP), HDLC и ADCCP для подключений точка-точка (двухузловой). Канальный уровень отвечает за доставку кадров (frame) между устройствами, подключенными к одному сетевому сегменту. Кадры канального уровня не пересекают границ сетевого сегмента. Кадры передаются последовательно с обработкой кадров подтверждения, отсылаемых обратно получателем[1]. Функции межсетевой маршрутизации и глобальной адресации осуществляются на более высоких уровнях модели OSI, что позволяет протоколам канального уровня сосредоточиться на локальной доставке и адресации. Заголовок кадра содержит аппаратные адреса отправителя и получателя, что позволяет определить, какое устройство отправило кадр и какое устройство должно получить и обработать его. В отличие от иерархических и маршрутизируемых адресов, аппаратные адреса одноуровневые. Это означает, что никакая часть адреса не может указывать на принадлежность к какой-либо логической или физической группе.

Сетевой уровень (англ. Network layer) — 3-й уровень сетевой модели OSI, предназначается для определения пути передачи данных. Отвечает за трансляцию логических адресов и имён в физические, определение кратчайших маршрутов, коммутацию и маршрутизацию, отслеживание неполадок и заторов в сети. На этом уровне работает такое сетевое устройство, как маршрутизатор. В пределах семантики иерархического представления модели OSI Сетевой уровень отвечает на запросы обслуживания от Транспортного уровня и направляет запросы обслуживания на Канальный уровень. Максимальная длина пакета сетевого уровня может быть ограничена командой `ip mtu`.

Передающая среда Коаксиальный кабель (устарел) – «толстый» - 10Base-5 — до 500м – «тонкий» - 10Base-2 — до 50м Витая пара 10Base-T, 100Base-T, – Категория 3: от 10 до 100 Мбит/с 100BASE-T4 (100м). – Категория 5е: 100 Мбит/с (2 пары), 1Гбит/с на (4пары) – Категория 6: 10 Гбит/с (55м) – Категория 7а: 40Гбит/с (50м), 100Гбит/с (15м) Оптика (10BASE-F,100BASE-SX,10GBASE-ER...) – ST (Straight Tip) – SC (Standard Connector) – LC (Lucent Connector) – Лазер находится в SFP (Small Plugin Factor) – ~500 м (Multi-mode fiber), ~80км (Single Mode) Wireless (802.11 - WiFi, 802.16 - WiMAX, 3G, 4G) – 2.4, 5, 60 GHZ – До 15 Гбит/с



Уровни стека TCP/IP [\[править | править код \]](#)

Стек протоколов TCP/IP включает в себя четыре уровня^[5]:

- **Прикладной уровень** (Application Layer),
- **Транспортный уровень** (Transport Layer),
- **Межсетевой уровень** (Сетевой уровень^[6]) (Internet Layer),
- **Канальный уровень** (Network Access Layer).

Передающая среда(физический уровень)

Физический уровень описывает физические свойства (например, электромеханические характеристики) среды и сигналов, переносящих информацию. Это физические характеристики кабелей и разъемов, уровни напряжений и электрического сопротивления и.т.д., в том числе, например, спецификация кабеля «неэкранированная витая пара»

Коаксиальный кабель(устарел) (До 500 м)

Витая пара (от 10 Мбит/с до 100 Гбит/с)

Оптика (~500 м(Multi-mode fiber), ~80км (Single mode))

Wireless (До 15 Гбит/с)

Канальный уровень Ethernet

Канальный уровень обеспечивает перенос данных по физической среде, обмен происходит кадрами. Уровень работает с применяемым в Ethernet физическими адресами, которые «вшиты» в сетевые адаптеры их производителями. Пример канального уровня: проверка доступности канала связи, если он общий для нескольких абонентов(Wireless).

Управление доступом к среде (физическому уровню) передачи,

Физическая адресация узлов (MAC)

Обеспечение сервиса для протоколов более высокого уровня (Service Access Point)

Упорядочивание кадров (фреймов), буферизация

Учет топологии сети

Управление потоком данных

Определение способа взаимодействия источника и приемника (связь с установлением соединения или нет)

Ethernet

7 байт	1 байт	6 байт	6 байт	2 байта	46 – 1500 байт	4 байта
Преамбула (101010...)	SFD (10101011)	Адрес получателя	Адрес источника	Тип	данные	Контрольная сумма

Преамбула (Preamble) не несет полезной информации. Генерируемый при её передаче в физической среде сигнал извещает принимающие устройства о необходимости быть готовым к приему.

SFD (Start of Frame Delimiter) разделитель начала кадра тоже не несет полезной информации (в первых версиях разделитель считался частью преамбулы). Он позволяет приемнику точно определить момент начала передачи полезных данных.

Эти два элемента кадра явным образом предназначены для обеспечения правильной работы нижнего – физического уровня.

Адрес получателя DA (Destination Address) содержит уникальный MAC – адрес устройства, которому адресован данный кадр или специальный адрес для широковещательной рассылки пакетов.

MAC-адреса

Этот адрес всегда состоит из 6 байт или 48 бит. Для его записи обычно используется шестнадцатеричная форма XX:XX:XX:XX:XX:XX.

Значения первых двух бит в первом байте - признак уникального адреса, остальные байты задают адрес конкретного сетевого адаптера. Уникальность адресации адаптеров обеспечивается специ-альным соглашением, по которому каждому производителю аппаратуры выделяется свое

значение (одно или несколько) кода OUI (Organizationally Unique Identifier – уникальный идентификатор организации) – 22 бита из байтов 1-3. Байты 4-6 заполняются изготовителем – на нем лежит ответственность за их уникальность (эта информация может рассматриваться как серийный номер платы).

Сетевой уровень IP

В отличие от канального уровня, имеющего дело с физическими адресами, сетевой уровень работает с логическими адресами. Он предназначается для определения пути передачи данных. Отвечает за трансляцию логических адресов и имён в физические, определение кратчайших маршрутов, коммутацию и маршрутизацию, отслеживание неполадок и заторов в сети. Одна из основных функций сетевого уровня – маршрутизация, объединение нескольких разнородных локальных сетей в одну сеть.

К протоколам сетевого уровня относятся IP и ICMP (Internet Control Message Protocol).

IP-адрес – уникальный сетевой адрес узла в компьютерной сети, построенной на основе стека протоколов TCP/IP

На сетевом уровне сам термин *сеть* наделяют специфическим значением. В данном случае под сетью понимается совокупность компьютеров, соединённых между собой в соответствии с одной из стандартных типовых топологий и использующих для передачи данных один из протоколов канального уровня, определённый для этой топологии.

Внутри сети доставка данных обеспечивается соответствующим канальным уровнем, а вот доставкой данных между сетями занимается сетевой уровень, который и поддерживает возможность правильного выбора маршрута передачи сообщения даже в том случае, когда структура связей между составляющими сетями имеет характер, отличный от принятого в протоколах канального уровня.

Сети соединяются между собой специальными устройствами, называемыми маршрутизаторами. *Маршрутизатор* – это устройство, которое собирает информацию о топологии межсетевых соединений и на её основании пересылает пакеты сетевого уровня в сеть назначения. Чтобы передать сообщение от отправителя, находящегося в одной сети, получателю, находящемуся в другой сети, нужно совершить некоторое количество *транзитных передач между сетями*, *waaxopoe* (от *hop* – прыжок), каждый раз выбирая подходящий маршрут. Таким образом, маршрут представляет собой последовательность маршрутизаторов, через которые проходит пакет.

Адресация, передача и маршрутизация пакетов

Маршрутизация пакетов проводится с помощью IP адресов. Маршрутизация осуществляется в посылающих TCP/IP конечных устройствах и в маршрутизаторах. В обоих случаях на интернет-уровне конечные устройства и в маршрутизаторы должны принимать решение куда отсылать пакет. Для принятия такого решения интернет-уровень получает информацию из специальных таблиц маршрутизации. Записи в таблице создаются автоматически при инициализации этого TCP/IP, хотя это может быть сделано и вручную. При доставке IP пакетов возможны два варианта доставки:

1. **Прямая доставка** (Direct delivery), в случае которой одно IP конечное устройство посылает пакеты другому устройству, принадлежащему одному и тому же сектору, используя MAC адрес принимающего устройства.

2. Косвенная доставка (Indirect delivery) производится через промежуточные устройства или маршрутизаторы к цели, которая не относится к тому же сектору локальной сети. В этом случае посылающий компьютер адресует пакет данных, используя MAC адрес маршрутизатора.

Более подробно ниже

Маршрутизация - процесс выбора пути для передачи пакета в сети. Под путем понимается последовательность маршрутизаторов, через которые проходит пакет по пути к узлу-назначению. IP-маршрутизатор – это специальное устройство, предназначенное для объединения сетей и обеспечивающее определение пути прохождения пакетов в составной сети.

Передача и адресация(надо дополнить)

Прямая маршрутизация

Обычно мы ассоциируем маршрутизаторы с устройствами, которые действительно выполняют маршрутизацию, однако любое поддерживающее протокол IP устройство способно выполнять эту функцию. На рисунке узел 10 присоединен непосредственно к сети 10, и способен маршрутизировать пакеты к любому другому узлу в сети 10.

Узлу 10.1.1.1 необходимо передать пакет узлу 10.2.2.2. Первое, что он делает - определяет, находится ли IP-адрес получателя в одной с ним сети. Для этого сравнивает свой номер сети 10 с номером сети получателя 10. Делает вывод, что узел-получатель находится в одном с ним сегменте сети.

С помощью протокола ARP(Address Resolution Protocol) определяет MAC-адрес узла-получателя и посылает пакет по этому адресу.

Косвенная маршрутизация

В следующем примере предполагается, что узел 10.1.1.1 имеет пакет, который нужно отправить узлу **172.16.0.1**.

Изучение адреса показывает, что узел назначения находится не на одной с передающим узлом сети. Узел 10.1.1.1 сконфигурирован так, что любые пакеты, требующие косвенной маршрутизации, передаются его шлюзу по умолчанию - маршрутизатору 1.

Чтобы доставить пакет маршрутизатору 1, узлу 10.1.1.1 необходим MAC-адрес маршрутизатора 10.3.3.3. Если MAC-адрес узлу 10.1.1.1 неизвестен, он отправляет ARP-запрос, чтобы его получить. Затем пакет, предназначенный для 172.16.0.1 отправляется маршрутизатору 1.

Маршрутизатор 1 осознает, что он подсоединен к сети 172.16. и полагает, что узел 172.16.0.1 должен быть частью этой сети. Маршрутизатор 1 реализует свою собственную процедуру прямой маршрутизации и посылает ARP-запрос, ища узел назначения.

При отправке пакета от узла 10.1.1.1 до узла 192.168.1.1 узел-отправитель сравнит номер своей сети с номером сети назначения и выяснит, что получатель находится в другой сети. Пакет будет отправлен шлюзу по умолчанию, в данном случае Маршрутизатору 1. Пусть в таблице маршрутизатора нет записи о сети 192.168.1, но шлюзом по умолчанию является маршрутизатор 2, тогда он перешлет пакет ему, а тот доставит пакет до получателя.

Если же узел 10.1.1.1 попытается отправить пакет узлу 192.164.1.1, то этот пакет будет перенаправлен маршрутизатору 1. Но так как он ничего не знает про сеть 192.164.1, то будет

выполнено одно из самых главных правил маршрутизации: **если пакет получен, и таблица маршрутизации не содержит информации о его сети назначения, пакет следует отбросить. (И отправить ICMP сигнал об ошибке отправителю)**

В этой таблице в столбце

Адрес сети назначения – указываются адреса всех сетей, которым данный маршрутизатор может передавать пакеты

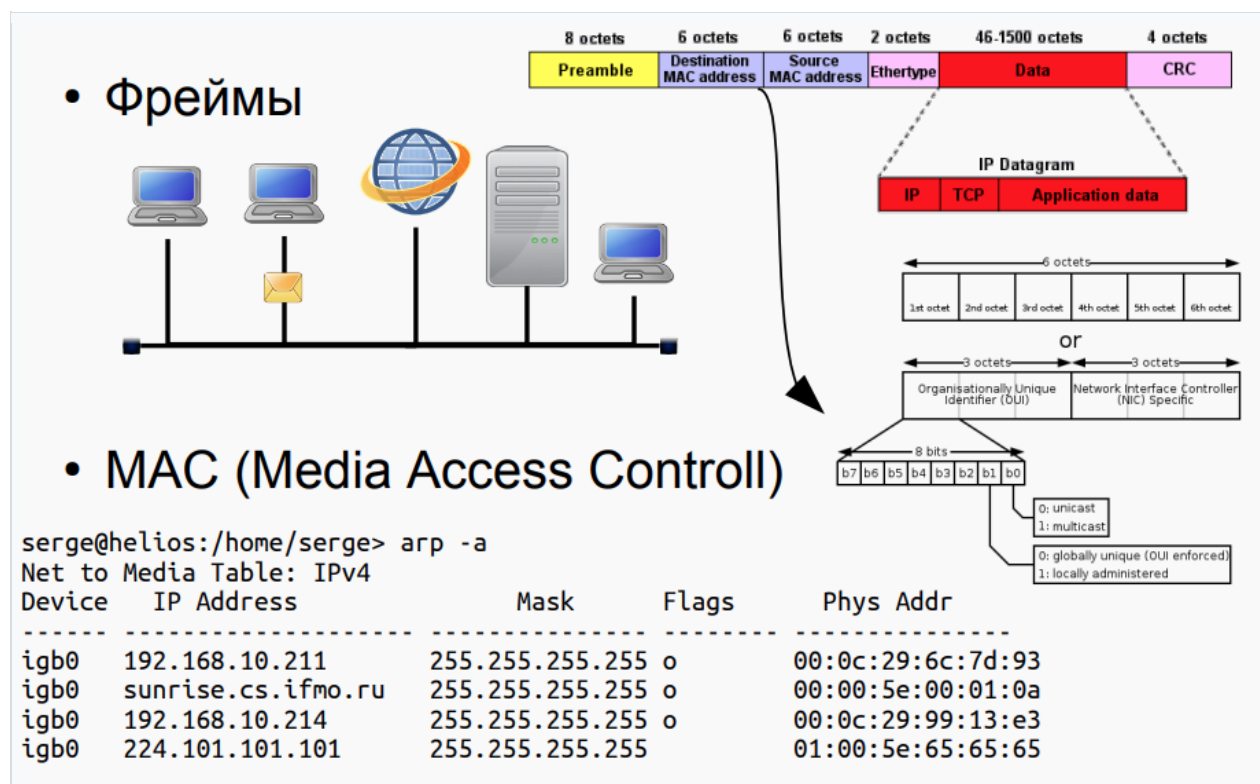
Следующий маршрутизатор в пути – IP-адрес удаленного маршрутизатора, которому необходимо послать пакет для доставки его по назначению

Номер выходного порта – по которому следует отправить пакет

Расстояние до сети назначения – это любая метрика, используемая в соответствии с заданным в сетевом пакете классом сервиса. Это может быть количество хопов(переходов), время прохождения пакета по линиям связи, надежность линий связи или другая величина, отражающая качество данного маршрута по отношению к конкретному классу сервиса. Если маршрутизатор поддерживает несколько классов сервиса пакетов, то таблица маршрутов составляется и применяется отдельно для каждого вида сервиса (критерия выбора маршрута)

Адресация пакетов Каждый абонент (узел) локальной сети должен иметь свой уникальный адрес (идентификатор или MAC-адрес), для того чтобы ему можно было адресовать пакеты. Существуют две основные системы присвоения адресов абонентам сети (точнее, сетевым адаптерам этих абонентов). 1. Первая система сводится к тому, что при установке сети каждому абоненту пользователь присваивает индивидуальный адрес по порядку, к примеру, от 0 до 30 или от 0 до 254. Присваивание адресов производится программно или с помощью переключателей на плате адаптера. При этом требуемое количество разрядов адреса определяется из неравенства: $2^n > N_{\max}$ где n – количество разрядов адреса, а $N_{\max}$ – максимально возможное количество абонентов в сети. Например, восемь разрядов адреса достаточно для сети из 255 абонентов. Один адрес (обычно 1111....11) отводится для широковещательной передачи, то есть он используется для пакетов, адресованных всем абонентам одновременно. Именно такой подход применен в известной сети Arpanet. Достоинства данного подхода – малый объем служебной информации в пакете, а также простота аппаратуры адаптера, распознающей адрес пакета. Недостаток – трудоемкость задания адресов и возможность ошибки (например, двум абонентам сети может быть присвоен один и тот же адрес). Контроль уникальности сетевых адресов всех абонентов возлагается на администратора сети. 2. Второй подход к адресации был разработан международной организацией IEEE, занимающейся стандартизацией сетей. Именно он используется в большинстве сетей и рекомендован для новых разработок. Идея этого подхода состоит в том, чтобы присваивать уникальный сетевой адрес каждому адаптеру сети еще на этапе его изготовления. Если количество возможных адресов будет достаточно большим, то можно быть уверенным, что в любой сети по всему миру никогда не будет абонентов с одинаковыми адресами. Поэтому был выбран 48-битный формат адреса, что соответствует примерно 280 триллионам различных адресов. Понятно, что столько сетевых адаптеров никогда не будет выпущено. * Младшие 24 разряда кода адреса называются OUA (Organizationally Unique Address) – организационно уникальный адрес. Именно их присваивает каждый из зарегистрированных производителей сетевых адаптеров. Всего возможно свыше 16 миллионов комбинаций, то есть каждый изготовитель может выпустить 16 миллионов сетевых адаптеров. * Следующие 22 разряда

кода называются OUI (Organizationally Unique Identifier) – организационно уникальный идентификатор. IEEE присваивает один или несколько OUI каждому производителю сетевых адаптеров. Это позволяет исключить совпадения адресов адаптеров от разных производителей. Всего возможно свыше 4 миллионов разных OUI, это означает, что теоретически может быть зарегистрировано 4 миллиона производителей. Вместе OUA и OUI называются UAA (Universally Administered Address) – универсально управляемый адрес или IEEE-адрес. * Два старших разряда адреса управляющие, они определяют тип адреса, способ интерпретации остальных 46 разрядов. Старший бит I/G (Individual/Group) указывает на тип адреса. Если он установлен в 0, то индивидуальный, если в 1, то групповой (многоточечный или функциональный). Пакеты с групповым адресом получают все имеющие этот групповой адрес сетевые адаптеры. Причем групповой адрес определяется 46 младшими разрядами. Второй управляющий бит U/L (Universal/Local) называется флажком универсального/местного управления и определяет, как был присвоен адрес данному сетевому адаптеру. Обычно он установлен в 0. Установка бита U/L в 1 означает, что адрес задан не производителем сетевого адаптера, а организацией, использующей данную сеть. Это случается довольно редко.



Ethernet фрейм определяет формат сообщения данных, передаваемых по сети. Формат сообщения содержит несколько полей информации, в том числе данных, подлежащих передаче. Блок данных определяется как фактические данные, которые будут отправлены, и может содержать от 46 до 1500 байт двоичной информации. Длина блока данных определяется и включается в сообщении в качестве поля для приемника, чтобы определить, какая часть сообщения представляет собой данные.

MAC-адрес является шести байтовым двоичным номером набора, который включает в себя информацию источника и назначения для узлов. MAC-адрес включен в каждое сообщение и

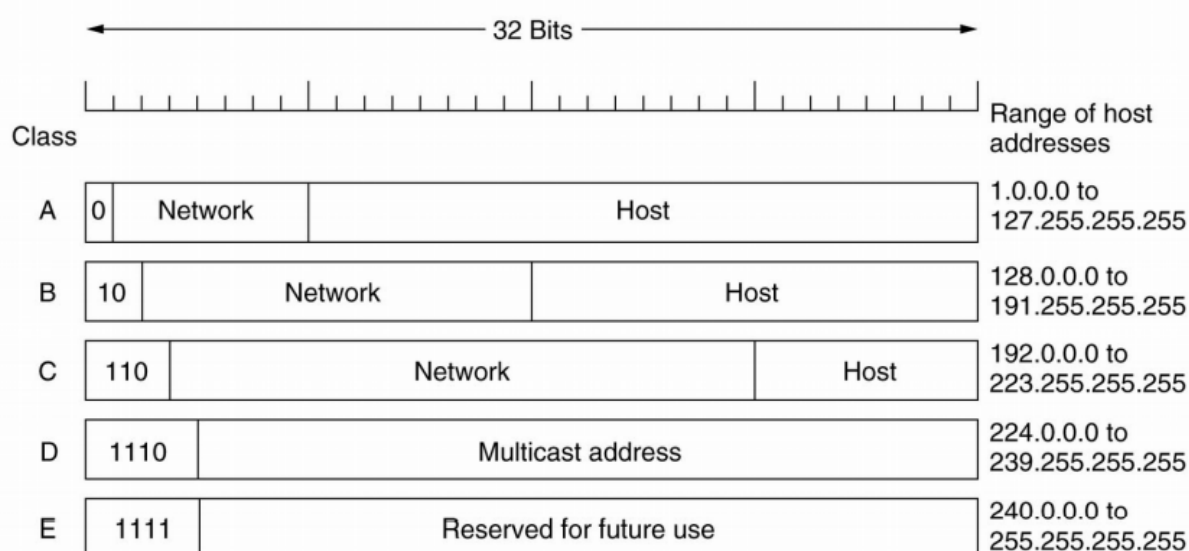
передается через сеть, а каждый узел сети Ethernet имеет уникальный MAC-адрес. MAC-адрес (Media Access Control – управление доступом к носителю) – это уникальный идентификатор, присваиваемый каждой единице оборудования компьютерных сетей.

Сетевой уровень предназначен для доставки пакетов от источника в пункт назначения, вероятно через множество физических сетей (линий связи). Сетевой уровень гарантирует, что каждый пакет будет доставлен от его исходной точки к его конечному пункту назначения. Некоторые обязанности сетевого уровня включают логическую адресацию и маршрутизацию.

Сетевой уровень предназначен для доставки отдельных пакетов от хоста источника до хоста пункта назначения.

Internet-протокол (IP). IP – механизм передачи, используемый в соответствии с протоколами TCP/IP. Это ненадежное без установления соединения обслуживание с лучшими намерениями. Термин "с лучшими намерениями" означает, что IP не обеспечивает проверки ошибок или выбор оптимального маршрута. IP учитывает ненадежность основных уровней и "прилагает все усилия", чтобы передать информацию к пункту назначения, но без гарантий. IP транспортирует данные в пакетах, называемых дейтаграммами, каждая из которых транспортируется отдельно. Дейтаграммы могут перемещаться по различным маршрутам и могут прибыть не в исходной последовательности или оказаться продублированными. IP не сохраняет порядок и список маршрутов и не имеет никаких средств для того, чтобы исправить дейтаграммы, однажды прибывшие в пункт назначения. Ограниченные функциональные возможности IP нельзя рассматривать как слабость. IP обеспечивает только функции передачи. Пользователь может добавить те средства, которые необходимы для данного приложения, и таким образом обеспечить максимальную эффективность.

• Адресация в IPv4 сетях



Маршрутизация (Routing) – это процесс по определению/вычислению лучшего маршрута движения для данных в сетях связи. Есть еще второе определение – это передача пакетов данных от отправителя к получателю.

Функцию роутинга могут выполнять:

Аппаратные средства — маршрутизаторы. Самый оптимальный вариант, позволяющий обрабатывать большие потоки данных и работает он быстрее.

Настроенные компьютеры с несколькими сетевыми интерфейсами и установленным на них специализированным и настроенным ПО. Обычно используется если конфигурация будет не слишком сложная.

Таблица маршрутизации — это файл-электронная таблица или база данных, которая располагается на маршрутизаторе или специально настроенном компьютере. В ней описывается соответствие адресов назначения с интерфейсами, через которые необходимо производить отправку данных до следующего маршрутизатора.

Таблица содержит:

Адрес сети или узла

Маску подсети назначения

Сетевой шлюз или по-другому, адрес маршрутизатора на который будут направлены данные

Интерфейс, с которого можно достучаться до шлюза

Метрика (не всегда), т.е. показатель, который задает предпочтительность пути.

Может заполняться как вручную, так и автоматически.

33. Модель TCP/IP: выделение адресов (DHCP), сервисы имен, транспортный и прикладной уровни.

DHCP — протокол автоматизации назначения IP-адреса клиенту. Он широко используется в современных сетях. В статье рассмотрим принципы работы, процесс DORA, основные опции и другие аспекты протокола. DHCP — протокол прикладного уровня модели TCP/IP, служит для назначения IP-адреса клиенту. Это следует из его названия — Dynamic Host Configuration Protocol. IP-адрес можно назначать вручную каждому клиенту, то есть компьютеру в локальной сети. Но в больших сетях это очень трудозатратно, к тому же, чем больше локальная сеть, тем выше возрастает вероятность ошибки при настройке. Поэтому для автоматизации назначения IP был создан протокол DHCP.

Принцип работы DHCP

Из вступления ясно, какие функции предоставляет DHCP, но по какому принципу он работает? Получение адреса проходит в четыре шага. Этот процесс называют DORA по первым буквам каждого шага: Discovery, Offer, Request, Acknowledgement.

Discovery, или поиск

Изначально клиент находится в состоянии инициализации (INIT) и не имеет своего IP-адреса. Поэтому он отправляет широковещательное (broadcast) сообщение DHCPDISCOVER на все устройства в локальной сети. В той же локальной сети находится DHCP-сервер. DHCP-сервер — это, например, маршрутизатор или коммутатор, существуют также выделенные DHCP-серверы.

Offer, или предложение

DHCP-сервер отвечает на поиск предложением, он сообщает IP, который может подойти клиенту. IP выделяются из области (SCOPE) доступных адресов, которая задается администратором.

Request, или запрос

Клиент получает DHCPOFFER, а затем отправляет на сервер сообщение DHCPREQUEST. Этим сообщением он принимает предлагаемый адрес и уведомляет DHCP-сервер об этом. Широковещательное сообщение почти полностью дублирует DHCPDISCOVER, но содержит в себе уникальный IP, выделенный сервером. Таким образом, клиент сообщает всем доступным DHCP-серверам «да, я беру этот адрес», а сервера помечают IP как занятый.

Acknowledgement, или подтверждение

Сервер получает от клиента DHCPREQUEST и окончательно подтверждает передачу IP-адреса клиенту сообщением DHCPACK. Это широковещательное или прямое сообщение утверждает не только владельца IP, но и срок, в течение которого клиент может использовать этот адрес.

Система доменных имен (DNS) является одной из фундаментальных технологий современной интернет-среды и представляет собой распределенную систему хранения и обработки информации о доменных зонах. Она необходима, в первую очередь, для соотнесения IP-адресов устройств в сети и более удобных для человеческого восприятия символьных имен. DNS состоит из распределенной базы имен, чья структура напоминает логическое дерево, называемое пространством имен домена. Каждый узел в этом пространстве имеет свое уникальное имя. Это логическое дерево «растет» из корневого домена, который является самым верхним уровнем

иерархии DNS и обозначается символом – точкой. А уже от корневого элемента ответвляются поддоменные зоны или узлы (компьютеры).

Сопоставление имен

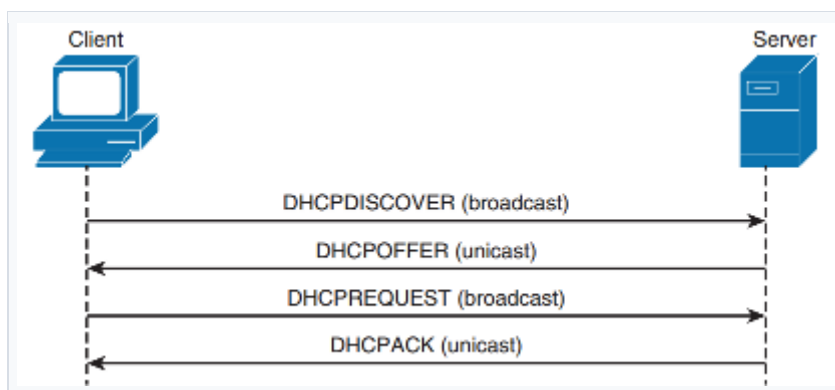
Давайте взглянем, как происходит сопоставление имен и IP-адресов. Предположим, пользователь набирает в строке браузера `www.1cloud.ru` и нажимает Enter. Браузер посылает запрос DNS-серверу сети, а сервер, в свою очередь, либо отвечает сам (если ответ ему известен), либо пересылает запрос одному из высокоуровневых доменных серверов (или корневому). Затем запрос начинает свое путешествие – корневой сервер пересылает его серверу первого уровня (поддерживающего зону `.ru`). Тот – серверу второго уровня (`1cloud`) и так далее, пока не найдется сервер, который точно знает запрошенное имя и адрес, либо знает, что такого имени не существует. После этого запрос начинает движение обратно. Чтобы наглядно объяснить, как это работает, ребята из `dnssimple` подготовили красочный комикс, который вы можете найти по ссылке. Также стоит пару слов сказать про процедуру обратного сопоставления – получение имени по предоставленному IP-адресу. Это происходит, например, при проверках сервера электронной почты. Существует специальный домен `in-addr.arpa`, записи в котором используются для преобразования IP-адресов в символьные имена. Например, для получения DNS-имени для адреса `11.22.33.44` можно запросить у DNS-сервера запись `44.33.22.11.in-addr.arpa`, и тот вернёт соответствующее символьное имя.

Транспортный уровень:

- TCP – Transmission Control Protocol
- Надежный, управляет перепосылкой данных
- Организует виртуальное соединение между гнездами (Socket=IP:port) на двух системах
- HTTP, FTP, SSH, SMTP, ...
- UDP – User Datagramm Protocol
- Послал сообщение и забыл
- Контроль надежности оставлен разработчику
- Максимальная скорость передачи
- SNMP, TFTP, DHCP, DNS, ...
- Протоколы разрабатывает программист

```
public static void main (String args[]) throws Exception {
    URL u = new URL("https://se.ifmo.ru/documents");
    URLConnection c = u.openConnection();
    BufferedReader s = new BufferedReader( new InputStreamReader(c.getInputStream()));
    String line = null;
    while ((line = s.readLine())!=null) {
        System.out.println(line);
    }
}
```

TCP/IP сеть позволяет пользовательским ЭВМ при подключении к сети в автоматическом режиме получить IP-адрес, маску, шлюз и стандартный DNS-сервер сети посредством DHCP сервера. Согласно протоколу DHCP, клиент отправляет широковещательный(его получают все узлы сети) запрос DHCPDISCOVER, чтобы найти DHCP сервер. Сервер, в ответ на данный запрос, отвечает предложением(DHCPOFFER) выдать адрес для клиента. Если клиент хочет воспользоваться этим DHCP сервером, то в ответ на DHCPOFFER он посылает DHCPREQUEST, который тоже является широковещательным, чтобы уведомить все остальные DHCP сервера, что их услуги не требуются. После получения DHCPREQUEST, сервер резервирует за клиентом IP адрес и подтверждает право его использования пакетом DHCPACK.



Для удобства пользования сетью модель TCP/IP предоставляет возможность создавать текстовые алиасы, называемые доменными именами, для хостов посредством сервиса DNS(Domain name system). Клиент, когда ему требуется получить IP-адрес, соответствующий имеющемуся доменному имени, отправляет запрос к DNS-серверу, и если у сервера есть информация о соответствии, то сервер отвечает на запрос и дает клиенту IP-адрес хоста. Если у сервера нет информации о соответствии, то он обращается к вышестоящему DNS-серверу и, кроме того, чтобы отправить полученную информацию клиенту, кэширует её и в следующий раз даст клиенту информацию не обращаясь к вышестоящим серверам. Существуют DNS-сервера самого высокого уровня, которые называются корневыми.

Транспортный уровень модели OSI (равно как и стека протоколов TCP/IP) обеспечивает следующие возможности:

Отслеживание индивидуальных сеансов общения между приложениями на передающем и принимающем устройствах. Эта функция относится только к протоколу TCP. Перед началом передачи полезных данных узел, инициирующий соединение открывает сессию с узлом получателем, чтобы убедиться, что получатель существует и готов принимать данные. Далее все полезные данные передаются в рамках установленной сессии и после завершения такой передачи сессия закрывается.

Сегментация данных (разбиение больших порций данных на сегменты для индивидуальной отсылки по сети, сборка этих сегментов после получения)

Идентификация приложений, передающих и принимающих данные. Для идентификации приложений используются номера портов: каждое приложение, желающее работать с сетью сообщает операционной системе о своих планах и регистрирует за собой какой-то номер порта.

Впоследствии, когда на компьютер придут данные, транспортный уровень заглянет в поле «номер порта получателя» и передаст эти данные соответствующему приложению.

Прикладной уровень (application layer) — это в действительности просто набор разнообразных протоколов, с помощью которых пользователи сети получают доступ к разделяемым ресурсам, таким как файлы, принтеры или гипертекстовые веб-страницы, а также организуют свою совместную работу, например, по протоколу электронной почты. Единица данных, которой оперирует прикладной уровень, обычно называется сообщением.

34. Интерфейсы ввода-вывода. Контроллеры внешних устройств. Уровни стандартизации, сопряжения с системной шиной, циклы обмена. Регистры контроллера.

Интерфейсы ввода-вывода

Интерфейсы определяют конкретные детали обмена (Частота, набор каналов передачи, способ кодирования, команды, представление данных, набор данных и последовательность...)

Требуется аппаратная и программная реализация. Нуждаются в точной спецификации и стандартизации (стороны обмена должны однозначно интерпретировать детали обмена)

Уровни стандартизации

Логическое подключение (все алгоритмы неким образом понимаются системой)

Физические параметры сигналов (напряжение, ток)

Конструктивные особенности (физическая реализация разъёмов, портов и т.п.)

Сопряжение с системной шиной происходит через контроллер ВУ. Два уровня сопряжения: Процессор <-> Контроллеры; Контроллеры <-> ВУ

Управляющие сигналы позволяют организовать 2 режима обмена информацией с контроллерами ВУ:

Программно-управляемый (асинхронный и по прерыванию)

Прямой доступ в память

Сопряжение с системной шиной

При использовании для обмена с ВУ команд ввода-вывода адрес (номер) ВУ передается по шине адреса. По этой же шине передаются и адреса ячеек памяти. Информация на шине адреса имеет смысл адреса ВУ только при наличии специальных управляющих сигналов (например, «Ввод из ВУ», «Вывод в ВУ», которые инициируются соответствующими командами ввода-вывода).

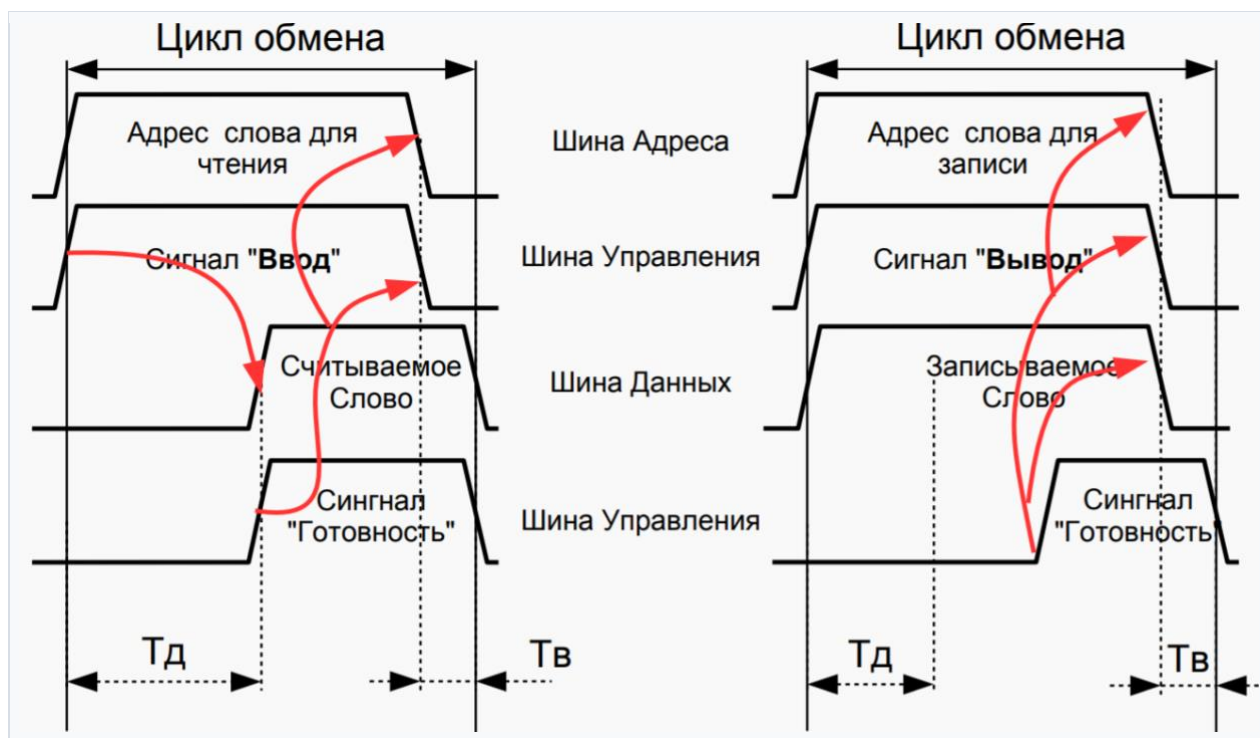
При реализации обмена с ВУ по аналогии с обращениями к памяти нет необходимости в спец. сигналах, указывающих, что на шине адреса находится адрес ВУ. Для этих адресов отведена часть адресного пространства, в контроллерах организована селекция адресов ВУ. Но остается необходимость передавать в ВУ приказ на ввод/вывод информации. Для этого есть линии управляющей шины «Чтение» и «Запись». Они обеспечивают обмен информацией микропроцессора с модулями памяти.

Циклы обмена

Операция вывод: микропроцессор выставляет на линиях адресной шины адрес (номер) ВУ, на линиях шины данных – значения разрядов выводимого слова данных и единичным сигналом по линии «Вывод в ВУ» указывает тип операции. Адресуемый контроллер ВУ принимает данные, пересылает их в ВУ и единичным сигналом по линии «Готовность ВУ» сообщает процессору, что данные приняты в ВУ и можно снять информацию с шин адреса и данных, а также сигнал «Вывод в ВУ».

Операция ввод: микропроцессор выставляет на линиях адресной шины адрес (номер) ВУ, и единичным сигналом на линии «Ввод из ВУ» указывает тип операции. По сигналу «Ввод из ВУ»

контроллер адресуемого ВУ считывает слово данных из ВУ, выставляет на линиях шины данных значения разрядов считанного слова и единичным сигналом на линии «Готовность ВУ» сообщает об этом процессору. Приняв данные, процессор снимает сигналы с шины адреса и линии «Ввод из ВУ».



Регистры контроллера

Рассмотрим типичные структуры контроллеров ВУ:

В основе контроллера несколько регистров (для временного хранения передаваемой информации). Каждый регистр имеет свой адрес (такие регистры называют портами ввода-вывода).

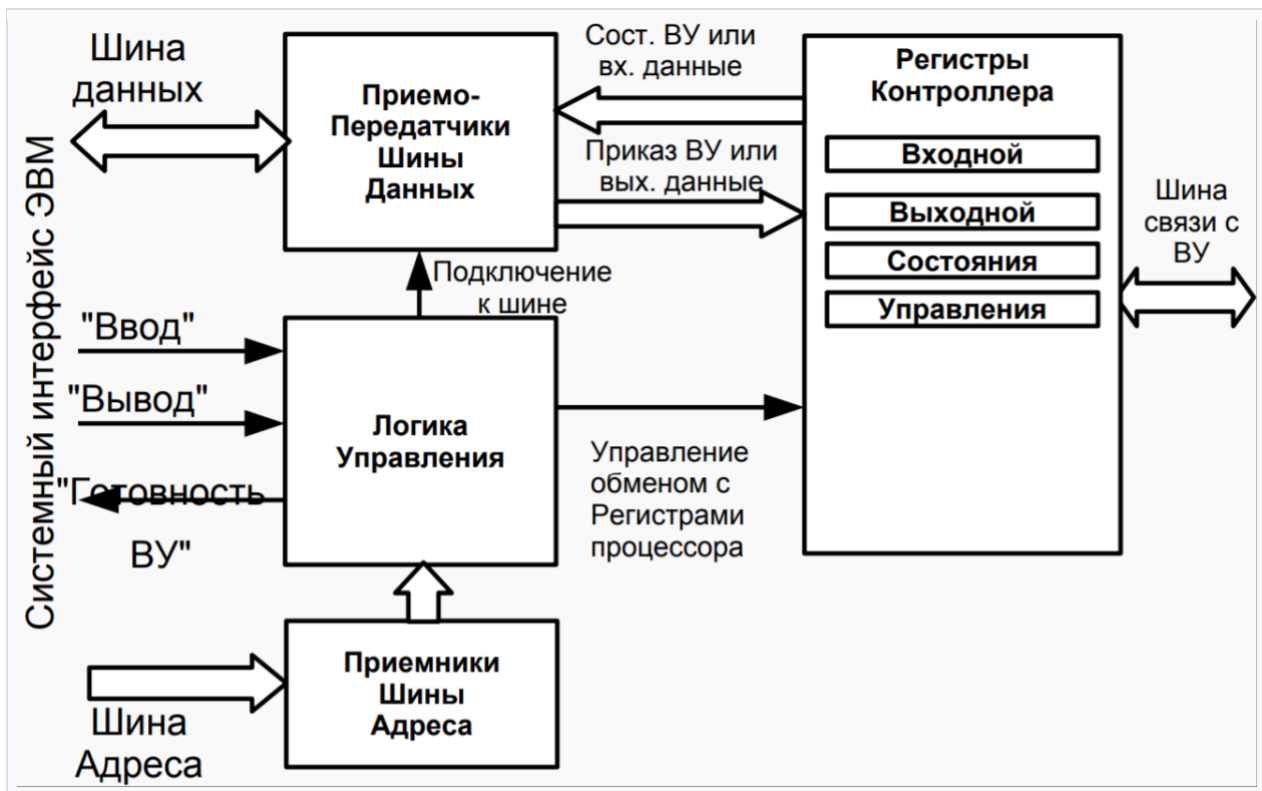
Регистр входных данных работает в режиме чтения, регистр выходных данных в режиме записи.

Регистр состояния в режиме чтения и содержит информацию о состоянии ВУ (включено/выключено, готово/не готово к обмену данными).

Регистр управления в режиме записи, служит для приема из микроЭВМ приказов для ВУ.

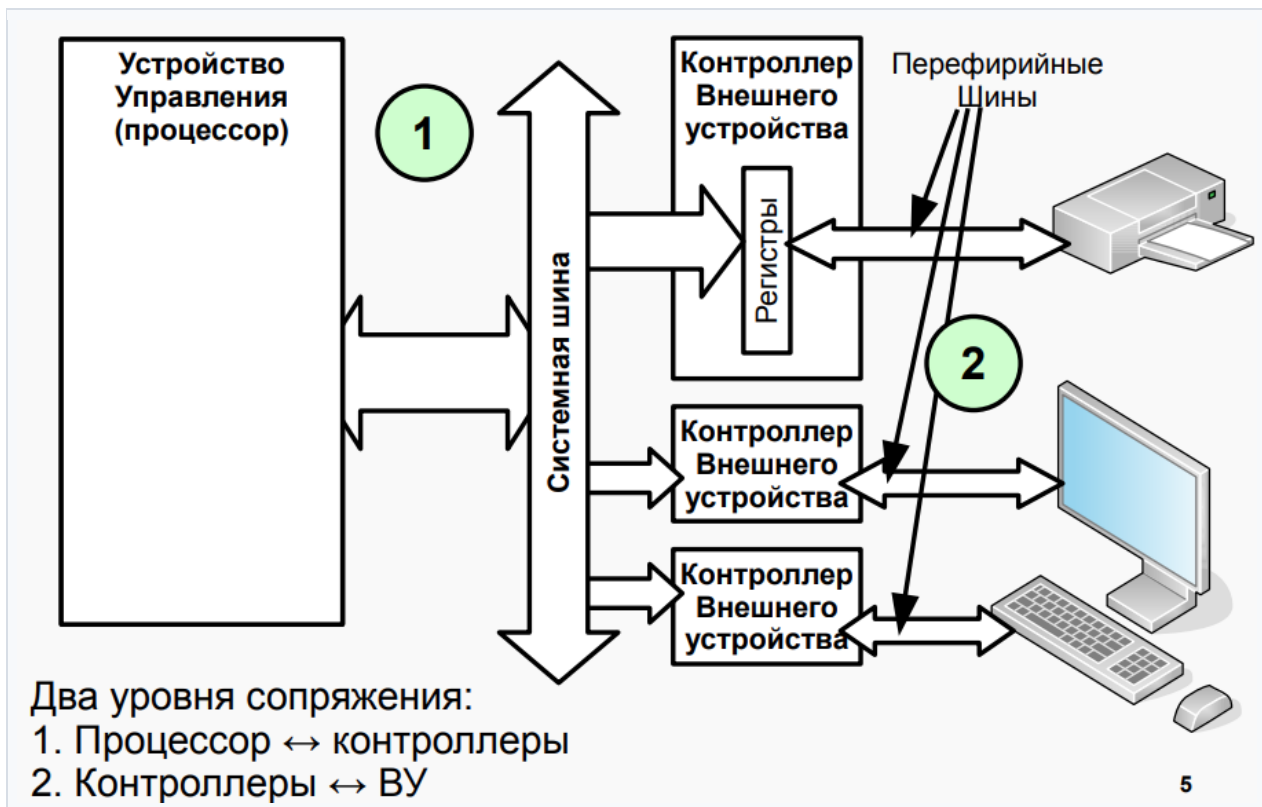
Логика управления выполняет селекцию адресов регистров контроллера, прием, обработку и формирование управляющего сигнала системного интерфейса, обеспечивая обмен информацией между регистрами контроллера и шиной данных системного интерфейса.

Приемопередатчики шин адреса и данных – для физического подключения электронных схем контроллера к шинам системного интерфейса.



Интерфейсы:

- Определяет конкретные детали обмена
- Аппаратная и/или программная реализация
- Нуждаются в точной спецификации и/или стандартизации
- Логическое подключение
- Конструктивные особенности



Контроллеры внешних устройств

Подключение любого внешнего устройства к микроЭВМ осуществляется через контроллер ВУ. Способы организации контроллеров определяются двумя факторами:

Форматами данных и режимами работы конкретных ВУ (существование самых разнообразных контроллеров от простейших до очень сложных)

Типом системного интерфейса микроЭВМ (определяет способ организации электронных схем контроллеров ВУ, обеспечивающих связь с шинами интерфейса, в первую очередь - схем распознавания адресов ВУ)

Для разных типов микроЭВМ разработаны контроллеры, обеспечивающие:

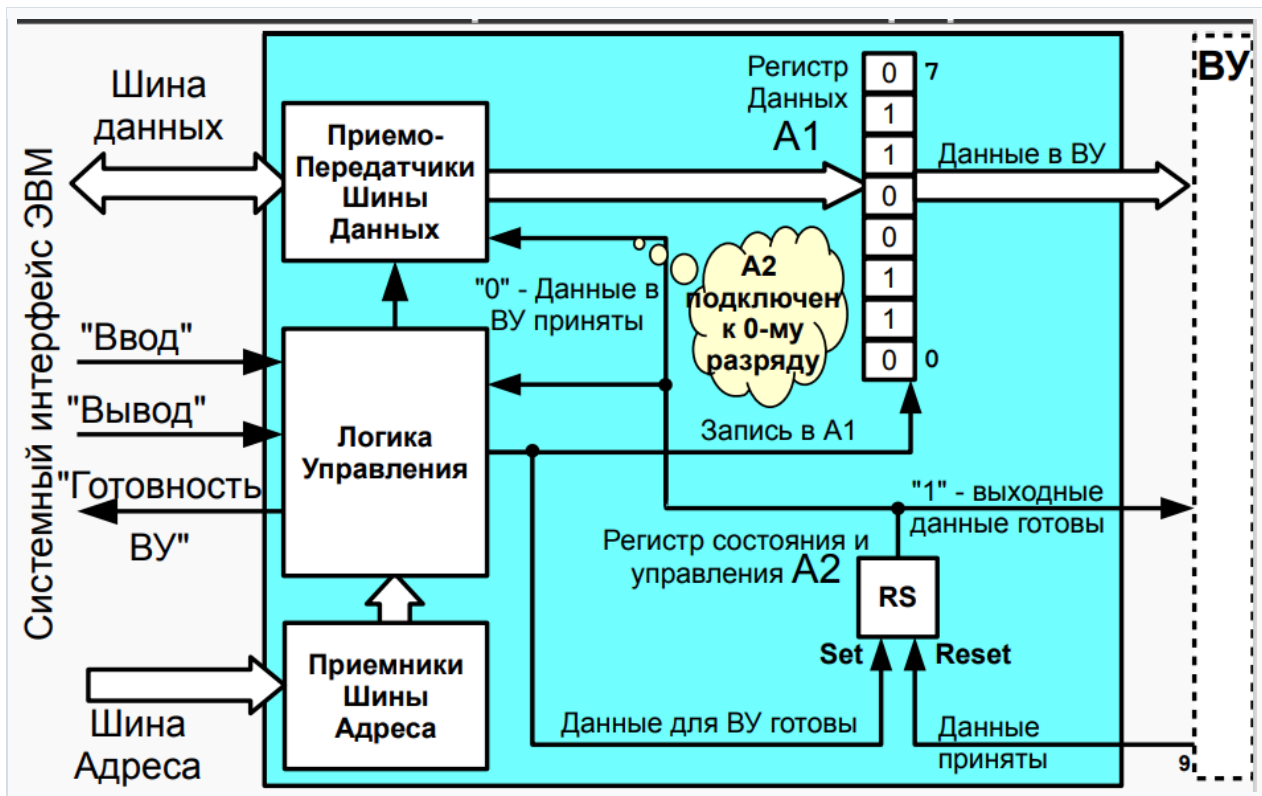
Связь с ВУ по стандартному параллельному (ИРПР) каналу передачи данных;

Преобразование информации из аналоговой в цифровую с заданной точностью;

Преобразование информации из дискретной формы представления в аналоговую в заданных диапазонах изменения аналоговых величин.

Существуют также программируемые контроллеры, режимы работы которых устанавливаются специальными командами микроЭВМ или определяются программами обмена с ВУ. Такие контроллеры необходимо настраивать на конкретный режим обмена (синхронный, асинхронный, с сигналами прерывания или без них). Настройка контроллеров производится программным путем с помощью спец. команд.

35. Параллельная передача данных. Контроллеры параллельной передачи и приема.



Параллельная передача данных в ВУ под управлением программы асинхронного обмена:

Процессор микроЭВМ проверяет готовность ВУ к приему данных

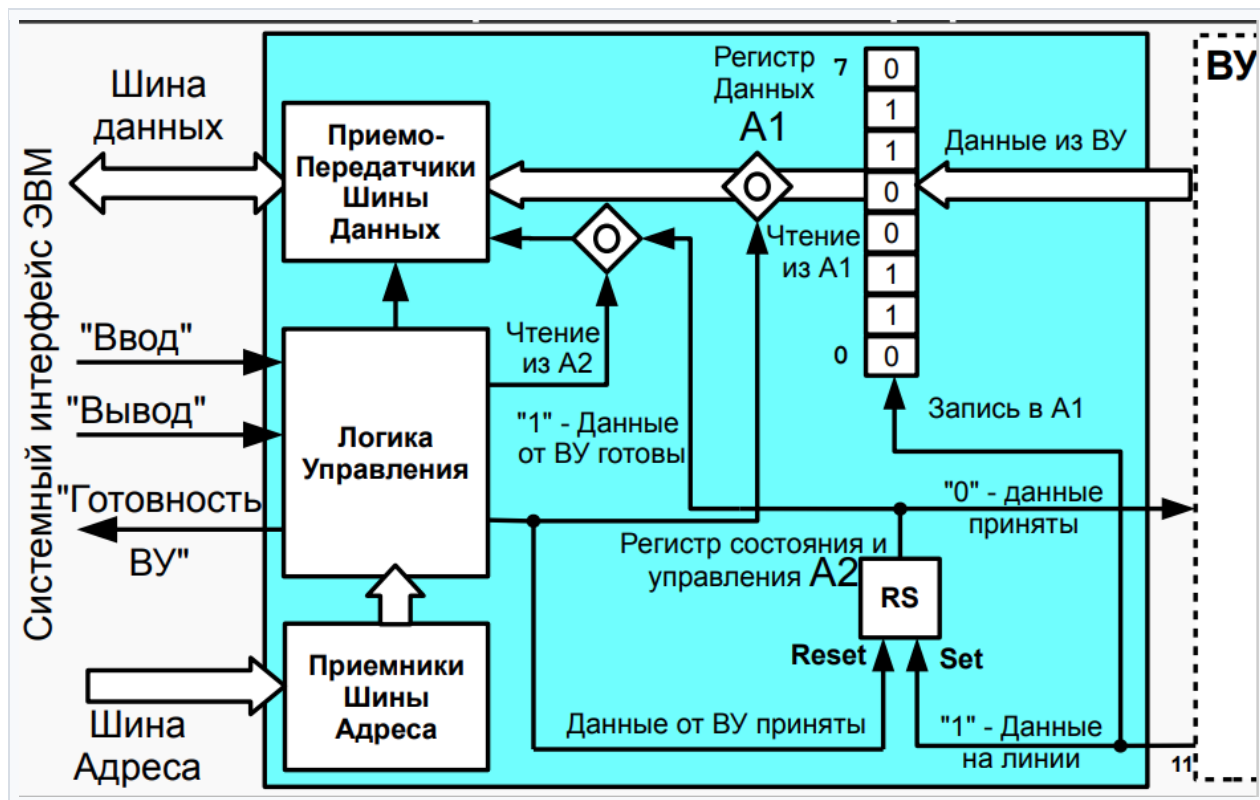
Если ВУ готово к приему данных (логический 0 в регистре A2), то данные передаются из шины данных системного интерфейса в регистр данных A1 контроллера и далее в ВУ. Иначе повторяется пункт 1.

В шине связи с ВУ используются 2 управляющих сигнала. Для формирования управляющего сигнала «Выходные данные готовы» и приема из ВУ управ. сигнала «Данные приняты» в контроллере используется одноразрядный адресуемый регистр состояния и управления A2. Одновременно с записью очередного байта данных из шины данных сист. интерфейса в адресуемый регистр данных контроллера A1 в регистр состояния и управления записывается логическая единица (формируется управляющий сигнал «Выходные данные готовы»).

ВУ, приняв байт данных, управ. сигналом «Данные приняты» обнуляет регистр состояния. При этом формируется:

Управ. сигнал сист. интерфейса «Готовность ВУ»

Признак готовности ВУ к обмену, передаваемый в процессор по одной из линий шины данных



Алгоритм асинхронного ввода:

Процессор проверяет наличие данных в регистре данных контроллера A1

Если данные готовы (логическая 1 в регистре A2), то они передаются из регистра данных A1 в шину данных системного интерфейса и далее в регистр процессора или ячейку памяти микроЭВМ. Иначе повторяется пункт 1.

Для формирования управляющего сигнала «Данные приняты» и приема из ВУ управ. сигнала «Данные от ВУ готовы» в контроллере используется одноразрядный адресуемый регистр состояния и управления A2.

ВУ записывает в регистр данных контроллера A1 очередной байт данных и управ. сигналом «Данные от ВУ готовы» устанавливает в единицу регистр состояния и управления A2. При этом формируется:

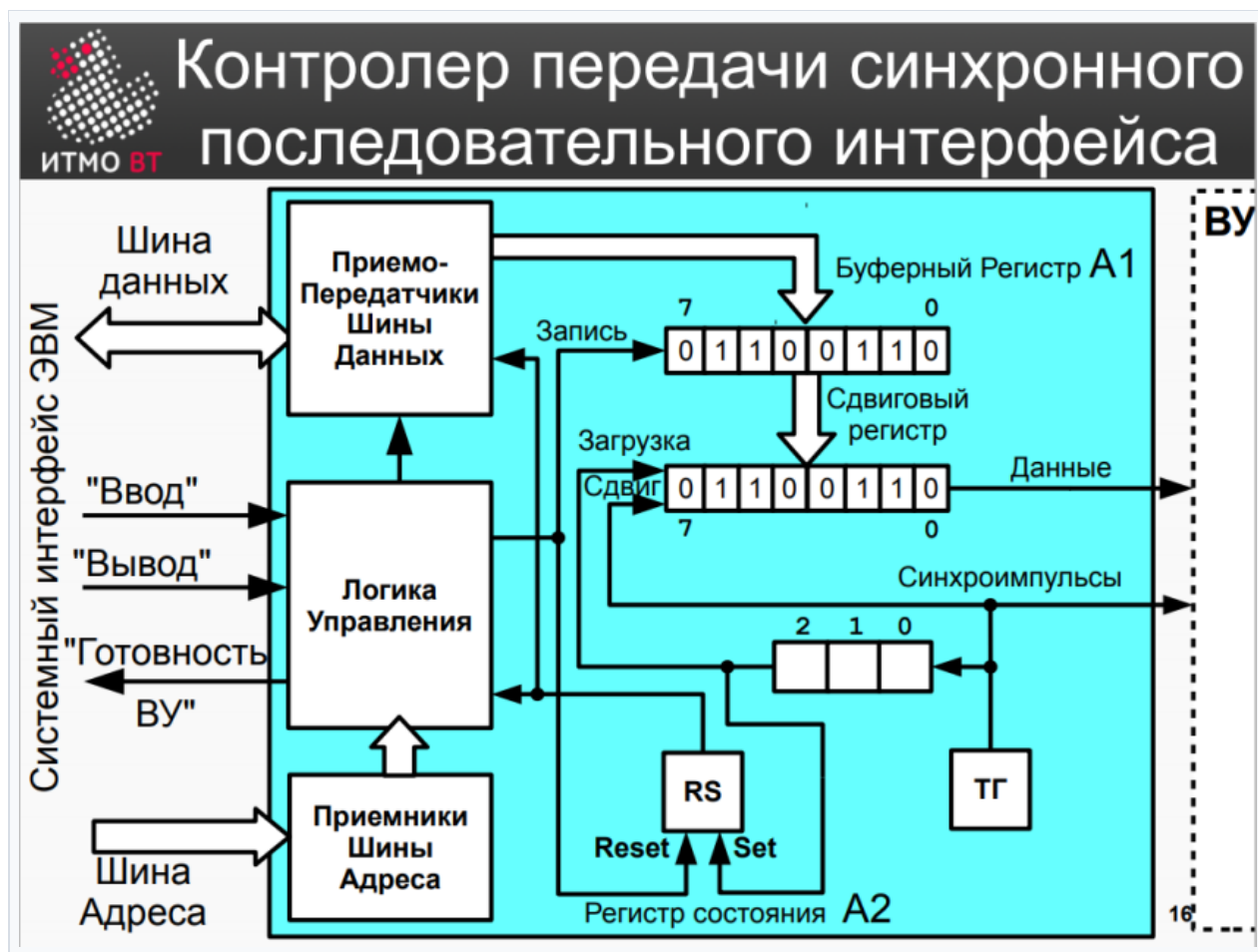
Управ. сигнал сист. интерфейса «Готовность ВУ»

Признак готовности ВУ к обмену, передаваемый в процессор по одной из линий шины данных

Так контроллер извещает процессор о готовности данных в регистре A1. Процессор читает байт данных из регистра данных контроллера и обнуляет регистр состояния и управления A2. При этом формируется управляющий сигнал «Данные приняты».

Для сопряжения регистров контроллера с шинами адреса и данных сист. интерфейса используются приемники шины адреса и приемопередатчики шины данных

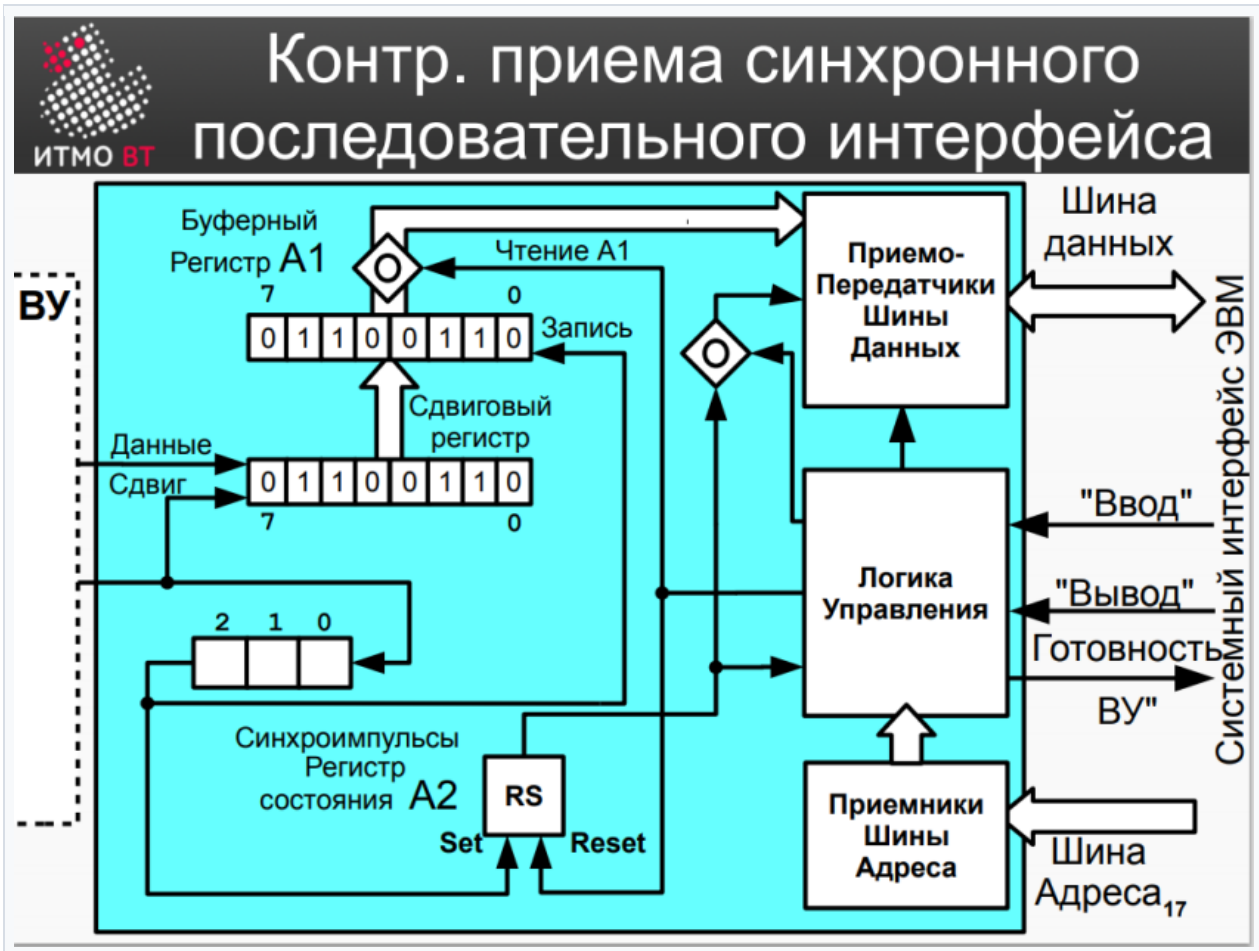
36. Синхронные последовательные интерфейсы. Контроллеры последовательной передачи и приема.



8-ми разрядный буферный регистр контроллера A1 - для временного хранения байта данных до его загрузки в сдвиговый регистр. Запись байта данных в буферный регистр происходит при наличии 1 в регистре состояния A2. Содержимое этого регистра передается в процессор по одной из линий шины данных и используется для формирования управ. сигнала «Готовность ВУ». При записи очередного байта в регистр A1 обнуляется регистр A2.

В сдвиговом регистре происходит преобразование данных из параллельного формата в последовательный и передача их в линию связи. По очередному тактовому импульсу содержимое сдвигового регистра сдвигается на 1 разряд вправо и в линию связи «Данные» выдается значение очередного разряда. Одновременно со сдвигом по линии «Синхронизация» передается тактовый импульс.

Количество переданных в линию тактовых сигналов (переданных бит) подсчитывается счетчиком тактовых импульсов. Как только его содержимое равно 7 (передано 8 бит информации) формируется управляющий сигнал «Загрузка» и происходит запись в сдвиговый регистр очередного байта из буферного. Устанавливается в 1 регистр состояния. Следующим тактовым импульсом счетчик будет сброшен в 0 и начнется очередной цикл выдачи 8 бит из сдвигового регистра в линию связи.



Буферный регистр контроллера A1 - для временного хранения байта , поступившего из сдвигового регистра. Чтение байта данных из буферного регистра происходит при наличии 1 в регистре состояния A2.

Данные, поступающие из линии связи в последовательном коде преобразуются в параллельный с помощью сдвигового регистра. Линия «Данные» подключается в контроллере к последовательному входу сдвигового регистра, а линия «Синхронизация» - на управ. вход «Сдвиг» и на вход счетчика тактовых импульсов. По тактовому импульсу по линии «Синхронизация» производится сдвиг содержимого сдвигового регистра на 1 разряд влево и запись очередного бита данных из линии «Данные» в младший разряд этого регистра. Одновременно увеличивается на 1 счетчик тактовых импульсов. Как только он становится равным 7 формируется управ. сигнал «Запись» и происходит запись в буферный регистр байта из сдвигового. Устанавливается в 1 регистр состояния.

При передаче байта данных из буферного регистра в шину данных регистр состояния обнуляется (т.е. в сдвиговый регистр принимается очередной байт информации).

37. Асинхронный обмен. Принципы деления частоты, формат кадра.

При реализации асинхронного обмена интервал между командами передачи данных задается самим внешним устройством. Контроллеры этих устройств снабжаются регистром состояния, который информирует ЭВМ о готовности устройства к обмену информацией. Обмен происходит по готовности ВУ.

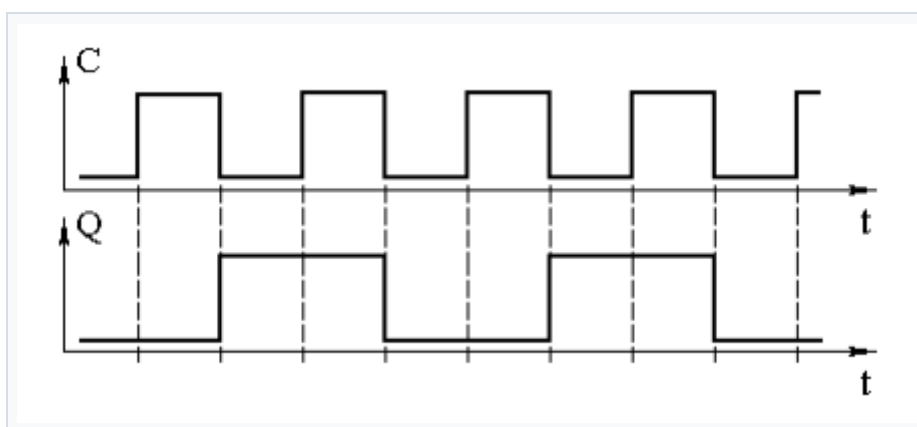
Преимущества:

не нужно знать время выполнения операции на ВУ

ВУ всегда успеет выполнить обработку данных перед началом следующей операции обмена

Недостаток: ЭВМ не выполняет никаких полезных действий во время ожидания момента готовности ВУ

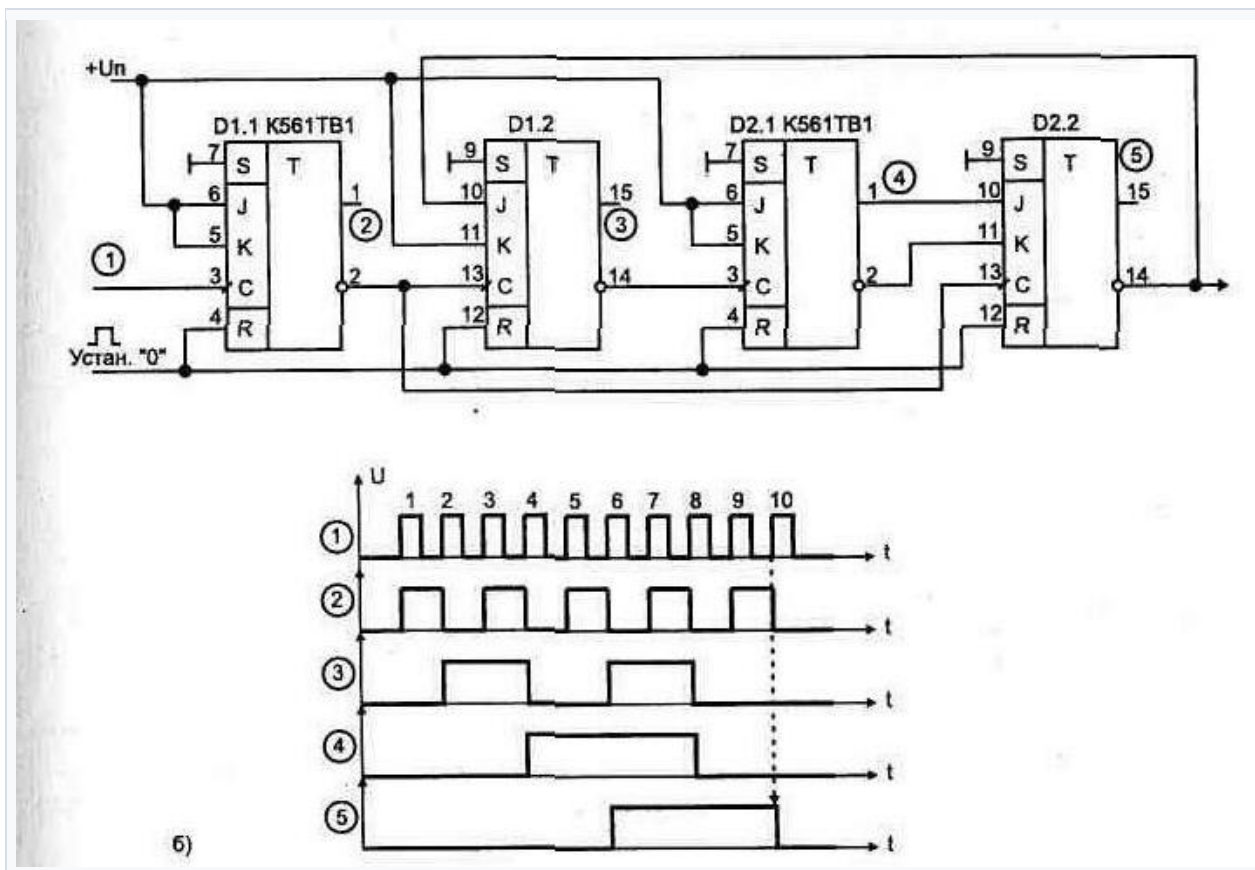
При асинхронной передаче данных, со временем может происходить рассинхронизация генераторов тактов передатчика и приемника, в результате чего данные могут быть переданы с искажениями, или не быть переданы вовсе. Одним из способов решения этой проблемы является деление тактовой частоты генераторов. Принцип его заключается в том, что исходная тактовая частота делится с некоторым коэффициентом, чаще всего кратным степени двойки. Таким образом увеличивается область совпадения фаз и время передачи данных, что увеличивает точность передачи. В этом примере частоту поделили на 2. На примере понять, как работает деление частоты, можно в некст вопросе.



Очень часто необходимо использовать триггер для деления частоты входной последовательности импульсов на два, т. е. производить переключение триггера в новое состояние каждым входным импульсом (фронтом или спадом). Такой триггер называют счетным, или Т-триггером.



Для деления частоты на степень двойки используется каскад из Т-триггеров:



Формат кадра – количество бит стартовых бит, количество бит информации, количество стоповых бит. В кадре может содержаться дополнительная информация, такая, как длина кадра и тд.

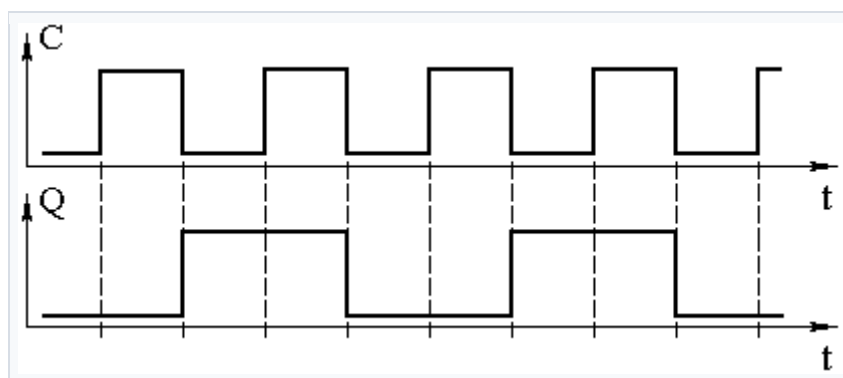
Асинхронный обмен

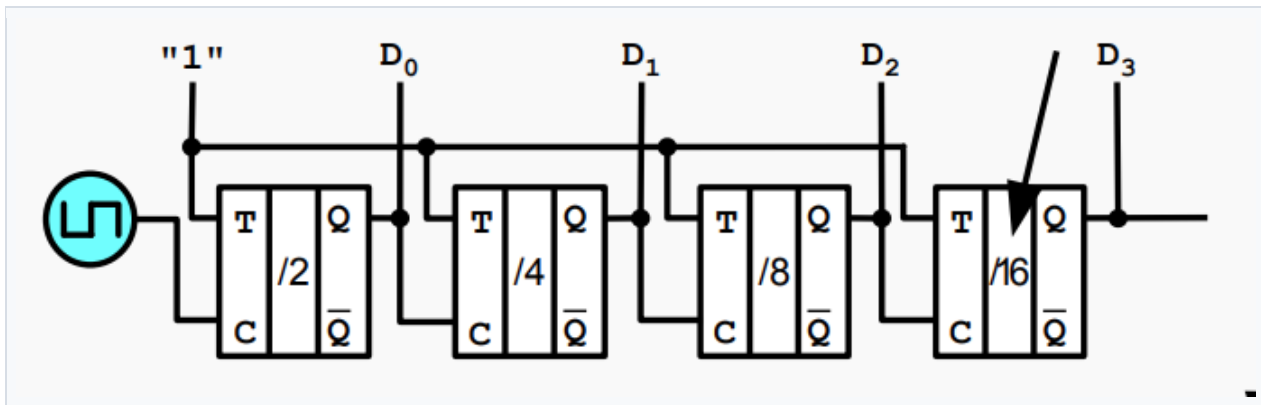
Обмен происходит по готовности ВУ, из этого вытекают следующие:

Недостаток:

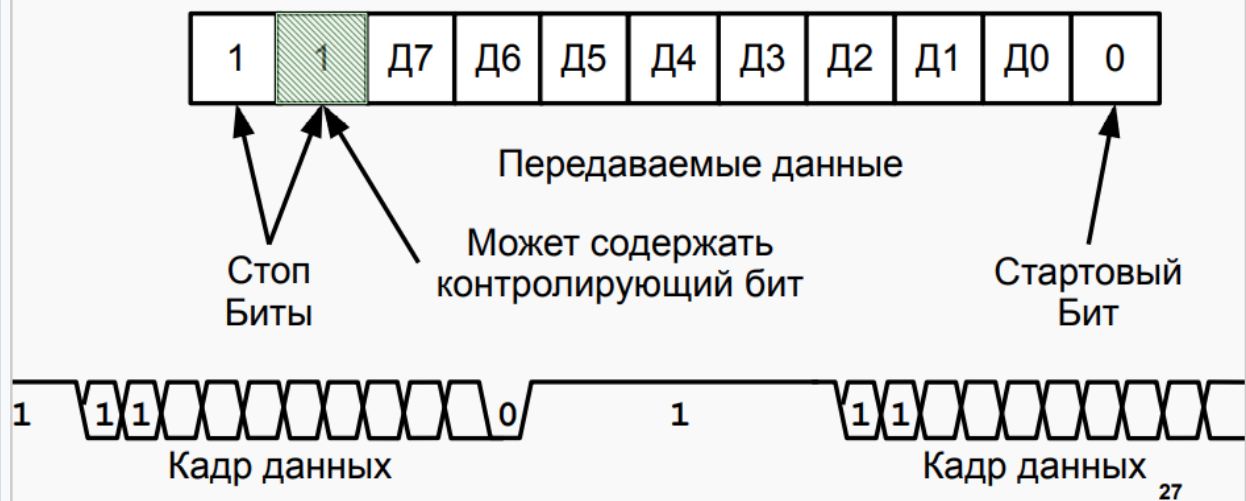
Принципы деления частоты

При асинхронной передаче данных, со временем может происходить рассинхронизация генераторов тактов передатчика и приёмника, в результате чего данные могут быть переданы с искажениями, или не быть переданы вовсе.





- Пример параметров настройки последовательного асинхронного порта 9600,8,n,1



38. Контроллер передачи асинхронного последовательного интерфейса.

Процесс передачи

После передачи предыдущих байтов данных в Регистр Состояния A2 записывается 1, что информирует процессор о готовности контроллера к приему следующего байта данных и передаче его по линии связи в ВУ. Он же запрещает формирование импульсов со схемы выработки импульсов сдвига – делителя частоты тактового генератора на 16 (счетчик по mod 16). Счетчик импульсов сдвига (счетчик по mod 10) находится в нулевом состоянии, и его единичный выходной сигнал поступает на вентиль И, подготавливая цепь выработки сигнала загрузки сдвигового регистра.

Процессор, выполняя команду «Вывод», выставляет передаваемый байт на шине данных и формирует управляющий сигнал системного интерфейса «Вывод».

По сигналу «Вывод» в контроллере происходит запись передаваемого байта в буферный регистр A1, сброс регистра состояния A2 и формирование на вентиле И сигнала «Загрузка».

Передаваемый бит переписывается в разряды 1..8 сдвигового регистра, в 0 разряд записывается ноль – стартовый бит, а в разряды 9 и 10 единицы – стоповые биты.

Снимается сигнал «Сброс» с делителя частоты, он начинает накапливать импульсы генератора тактовой частоты и в момент приема шестнадцатого тактового импульса срабатывает импульс сдвига (так реализовано деление частоты).

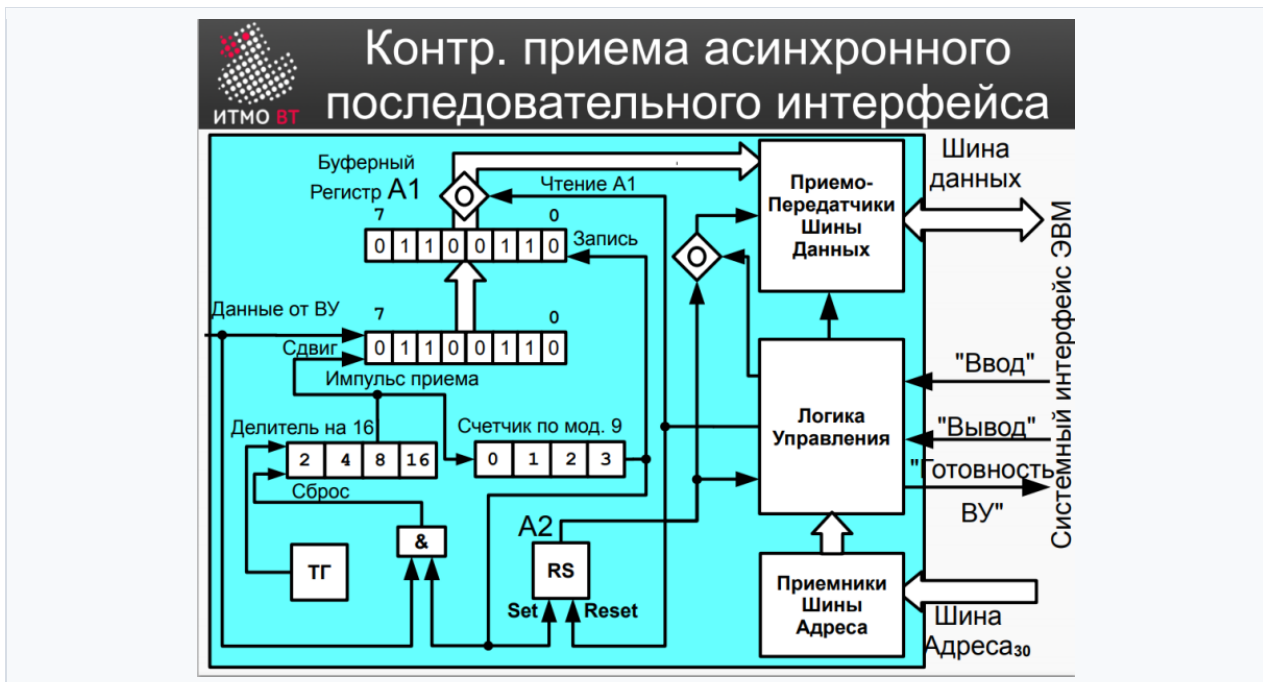
На шине «Данные» поддерживается 0 (значение стартового бита) до тех пор, пока не будет выработан первый импульс сдвига (время передачи 1 бита). Импульс сдвига изменит состояние счетчика импульсов сдвига и переписет в нулевой разряд сдвигового регистра первый информационный бит передаваемого байта данных. Значение этого бита будет поддерживаться на линии «Данные» до следующего импульса сдвига.

Аналогично передаются остальные информационные биты, первый стоповый бит, и, наконец второй стоповый бит, при передаче которого счетчик импульсов сдвига снова установится в нулевое состояние. Это приведет к записи 1 в регистр состояния A2. Единичный сигнал с выхода регистра A2 запретит формирование импульсов сдвига, и информирует о готовности к приему нового байта данных.

После завершения передачи очередного кадра (стартового бита, информационного бита и 2х-стоповых битов), на линии передачи данных поддерживается значение второго стопового бита – единицы



39. Контроллер приема асинхронного последовательного интерфейса.



На линии данных от ВУ находится единица, что запрещает работу делителя частоты генератора тактовых импульсов.

При обнаружении нулевого сигнала на линии «Данные» (смена стопового бита на стартовый), снимается сигнал «Сброс» с делителя частоты, он начинает накапливать импульсы генератора тактовой частоты.

Когда на счетчике накопится значение 8 (половина времени передачи бита), он выдаст сигнал, поступающий на входы сдвигового регистра и счетчика импульсов сдвига. Таким образом уменьшается вероятность искажения данных.

Последующие сдвиги происходят после прохождения 16-ти тактовых импульсов.

При приеме в сдвиговый регистр 9-го бита кадра (8-го инф. бита), из него выдвинется стартовый бит, и, следовательно в сдвиговом регистра будет размещен информационный байт. В этот момент счетчик импульсов сдвига придет в нулевое состояние и на его выходе будет выработан единичный сигнал, это приведет к тому, что:

содержимое сдвигового регистра будет переписано в буферный регистр

в триггер A2 запишется 1 и он будет информировать процессор об окончании приема очередного байта

вентиль И подготовится к выработке сигнала «Сброс». Он сформируется после прихода первого стопового бита

Получив сигнал готовности (1 в триггере А2), процессор выполнит команду «Ввод». При этом вырабатывается сигнал системного интерфейса «Ввод», по которому производится пересылка принятого байта данных из БР в процессор (сигнал «Чтение») и сброс триггера А2

40. Организация прямого доступа к памяти. Контроллер ПДП.

Этот тип обмена реализуется полностью аппаратно и управляется контроллером ПДП.

Организация ПДП

Особенности ПДП

Возможность начальной загрузки программ в основную память микроЭВМ из устройства ввода.

Обеспечивает возможность использования в микроЭВМ быстродействующих внешних запоминающих у-в.

Для экономии ресурсов контроллер ПДП не имеет свои ресурсы, а подключается к шинам данных (ШД) и адреса (ША) системного интерфейса ЭВМ, что делает невозможным одновременное использование шин контроллером ПДП и процессором.

Эта проблема решается двумя способами:

Захват цикла

Простой захват цикла. Передача происходит в те машинные циклы, в которых процессор не обменивается данными с памятью. Пометка таких циклов выполняется либо с помощью спец. указывающего цикла, либо такие циклы выбираются с помощью спец. селектирующих схем, что усложняет конструкцию процессоров. При таком способе организации обмена ПДП не снижает производительности микроЭВМ, но обмен возможен только в случайные моменты времени одиночными байтами или словами.

Захват цикла с принудительным отключением процессора от шин системного интерфейса. Для его реализации такого режима ПДП системный интерфейс (СИ) дополняется двумя линиями для передачи управляющих сигналов «Требование ПДП» (ТПДП) и «Предоставление ПДП» (ППДП).

ТПДП формируется контроллером ПДП.

Получив сигнал ТПДП, процессор приостанавливает выполнение очередной команды, не дожидаясь её завершения, выдает в системный интерфейс ППДП и отключается от шин СИ

Контроллер ПДП получает управления над шинами СИ и осуществляет обмен одним байтом или словом данных с памятью микроЭВМ.

Контроллер ПДП возвращает управление СИ процессору.

Блокировка процессора. Отличается от «Захвата цикла» тем, что управление СИ передается контроллеру ПДП не на время обмена одним байтом, а на время обмена блоком данных.

Контроллер ПДП ввода данных из ВУ в режиме «Захват цикла»

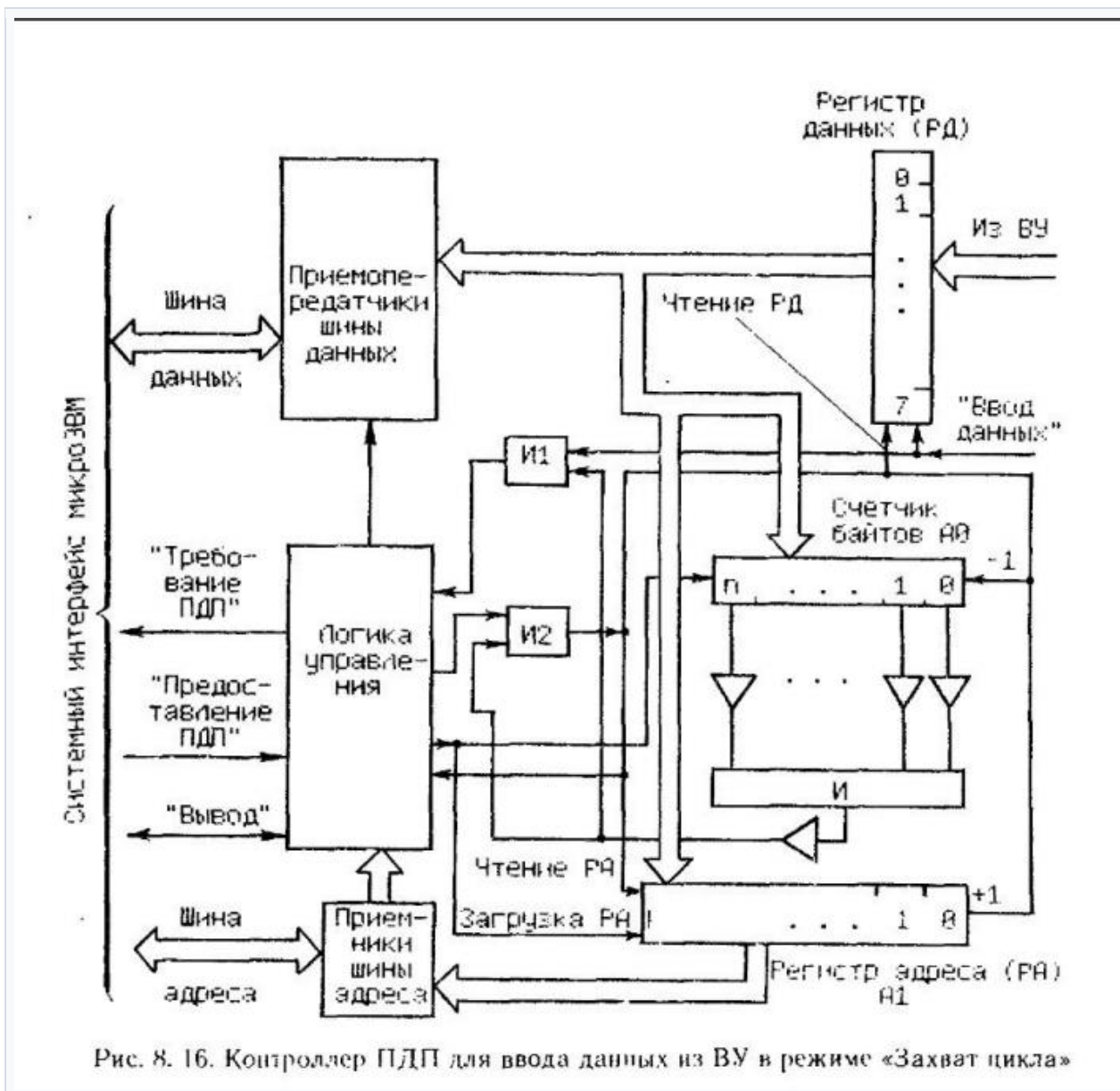


Рис. 8. 16. Контроллер ПДП для ввода данных из ВУ в режиме «Захват цикла»

Процессор загружает в СК контроллера количество принимаемых байтов, а в РА контроллера начальный адрес области памяти для вводимых данных.

Байты данных из ВУ поступают в РД контроллера, при этом каждый байт сопровождается управляющим сигналом из ВУ «Ввод данных», который обеспечивает запись байта в РД контроллера. По этому же сигналу при ненулевом состоянии СК контроллер формирует сигнал ТПДП.

По ответному сигналу процессора ППДП контроллер выставляет на ША и ЩД содержимое своих РА и РД.

Формируя приказ «Вывод», контроллер ПДП обеспечивает запись байта данных из своего регистра данных в память микроЭВМ.

По тому же сигналу ППДП содержимое СК декрементируется, а содержимое РА обновляется. Как только СК станет равным нулю, контроллер прекратит формирование сигналов ТПДП.

Прямой доступ к памяти ([англ.](#) *direct memory access, DMA*) — режим обмена данными между устройствами [компьютера](#) или же между устройством и [основной памятью](#), в котором [центральный процессор](#) (ЦП) не участвует. Так как данные не пересылаются в ЦП и обратно, скорость передачи увеличивается.

Этот тип обмена реализуется полностью аппаратно и управляется контроллером ПДП.

Организация ПДП

Особенности ПДП

1. Возможность начальной загрузки программ в основную память микроЭВМ из устройства ввода.
2. Обеспечивает возможность использования в микроЭВМ быстродействующих внешних запоминающих у-в.

Для экономии ресурсов контроллер ПДП не имеет свои ресурсы, а подключается к шинам данных (ШД) и адреса (ША) системного интерфейса ЭВМ, что делает невозможным одновременное использование шин контроллером ПДП и процессором.

Эта проблема решается двумя способами:

1. Захват цикла
 - а. Простой захват цикла.
Передача происходит в те машинные циклы, в которых процессор не обменивается данными с памятью. Пометка таких циклов выполняется либо с помощью спец. указывающего цикла, либо такие циклы выбираются с помощью спец. селектирующих схем, что усложняет конструкцию процессоров.
При таком способе организации обмена ПДП не снижает производительности микроЭВМ, но обмен возможен только в случайные моменты времени одиночными байтами или словами.
 - б. Захват цикла с принудительным отключением процессора от шин системного интерфейса.
Для его реализации такого режима ПДП системный интерфейс (СИ) дополняется двумя линиями для передачи управляющих сигналов «Требование ПДП» (ТПДП) и «Предоставление ПДП» (ППДП).

1. ТПДП формируется контроллером ПДП.

2. Получив сигнал ТПДП, процессор приостанавливает выполнение очередной команды, не дожидаясь её завершения, выдает в системный интерфейс ППДП и отключается от шин СИ
3. Контроллер ПДП получает управления над шинами СИ и осуществляет обмен одним байтом или словом данных с памятью микроЭВМ.
4. Контроллер ПДП возвращает управление СИ процессору.

2. Блокировка процессора

Отличается от «Захвата цикла» тем, что управление СИ передается контроллеру ПДП не на время обмена одним байтом, а на время обмена блоком данных.

Контроллер ПДП ввода данных из ВУ в режиме «Захват цикла»

1. Процессор загружает в СК контроллера количество принимаемых байтов, а в РА контроллера начальный адрес области памяти для вводимых данных.
2. Байты данных из ВУ поступают в РД контроллера, при этом каждый байт сопровождается управляющим сигналом из ВУ «Ввод данных», который обеспечивает запись байта в РД контроллера. По этому же сигналу при ненулевом состоянии СК контроллер формирует сигнал ТПДП.
3. По ответному сигналу процессора ППДП контроллер выставляет на ША и ШД содержимое своих РА и РД.
4. Формируя приказ «Вывод», контроллер ПДП обеспечивает запись байта данных из своего регистра данных в память микроЭВМ.
5. По тому же сигналу ППДП содержимое СК декрементируется, а содержимое РА обновляется. Как только СК станет равным нулю, контроллер прекратит формирование сигналов ТПДП

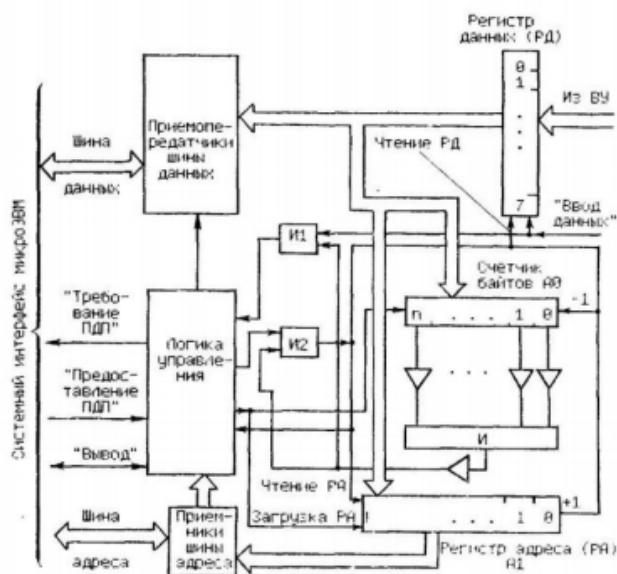


Рис. 8. 16. Контроллер ПДП для ввода данных из ВУ в режиме «Захват цикла»